

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**NGHIÊN CỨU VÀ PHÁT TRIỂN ROBOT TRỢ LÝ ẢO
ỨNG DỤNG TRÍ TUỆ NHÂN TẠO HỖ TRỢ CHĂM SÓC
VÀ GIÁM SÁT NGƯỜI CAO TUỔI TẠI NHÀ**

Ngành: Kỹ thuật Máy tính

HỘI ĐỒNG: 10L

GVHD: TS. Lê Trọng Nhân

GVPB: PGS. TS. Trần Văn Hoài

—o0o—

SVTH 1: Lê Đăng Khoa (2211601)

SVTH 2: Vũ Trọng Khánh (2211541)

SVTH 3: Lương Văn Khanh (2211486)

Tp. Hồ Chí Minh, tháng 12/2025

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đề án với đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*” là công trình nghiên cứu do chính chúng em thực hiện dưới sự hướng dẫn khoa học của TS. Lê Trọng Nhân. Toàn bộ nội dung của đề án được xây dựng trên cơ sở quá trình học tập, nghiên cứu nghiêm túc, có sự tìm hiểu, phân tích và tổng hợp từ nhiều nguồn tài liệu khác nhau.

Chúng em xin cam đoan rằng các số liệu, kết quả thực nghiệm, mô hình đề xuất, hình ảnh minh họa và các nội dung trình bày trong đề án là trung thực, phản ánh đúng quá trình nghiên cứu và thực hiện đề tài.

Trong quá trình thực hiện đề án, chúng em có tham khảo và sử dụng các tài liệu, công trình nghiên cứu, sách, bài báo khoa học, cũng như các nguồn thông tin hợp pháp khác. Tất cả các tài liệu tham khảo đều đã được trích dẫn đầy đủ, chính xác và tuân thủ đúng các quy định về trích dẫn học thuật. Chúng em xin cam đoan không sao chép, không đạo văn, không sử dụng trái phép kết quả nghiên cứu của các tác giả khác dưới bất kỳ hình thức nào.

Chúng em xin hoàn toàn chịu trách nhiệm trước Khoa, Nhà trường và Hội đồng chấm đề án về tính trung thực, chính xác và tính pháp lý của toàn bộ nội dung được trình bày trong đề án này. Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến bản quyền, đạo đức học thuật hoặc tính xác thực của dữ liệu, chúng em xin hoàn toàn chịu trách nhiệm theo các quy định hiện hành.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

LỜI CẢM ƠN

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Lê Trọng Nhân, người đã trực tiếp hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài “*Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà*”. Trong quá trình thực hiện đồ án, thầy đã tận tình chỉ dẫn về phương pháp nghiên cứu, định hướng triển khai nội dung, cũng như thường xuyên góp ý, chỉnh sửa và giải đáp những khó khăn mà chúng em gặp phải. Sự hướng dẫn tận tâm và những ý kiến chuyên môn quý báu của thầy đã giúp chúng em từng bước hoàn thiện đề tài theo hướng khoa học, thực tiễn và có giá trị ứng dụng.

Chúng em xin chân thành cảm ơn quý thầy cô trong Khoa/Viện đã giảng dạy và truyền đạt cho chúng em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Những kiến thức và kinh nghiệm mà quý thầy cô đã chia sẻ không chỉ giúp chúng em hoàn thành đồ án này mà còn là hành trang quan trọng cho quá trình học tập và công tác sau này.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến Khoa và Nhà trường đã tạo điều kiện thuận lợi về cơ sở vật chất, môi trường học tập và nghiên cứu, giúp chúng em có thể triển khai và hoàn thành đề tài đúng tiến độ.

Chúng em cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và hỗ trợ chúng em cả về tinh thần lẫn thời gian trong suốt quá trình học tập và thực hiện đồ án. Sự động viên đó là nguồn khích lệ to lớn giúp chúng em vượt qua những khó khăn và áp lực trong quá trình nghiên cứu.

Mặc dù đã có nhiều cố gắng và nỗ lực trong quá trình thực hiện, song do hạn chế về thời gian, kiến thức và kinh nghiệm nghiên cứu, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý và chỉ dẫn từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn.

Nhóm tác giả

Lê Đăng Khoa

Vũ Trọng Khánh

Lương Văn Khanh

TÓM TẮT

Trong bối cảnh già hóa dân số và nhu cầu chăm sóc người cao tuổi tại nhà ngày càng tăng. Việc giám sát an toàn, duy trì liên lạc và hỗ trợ sinh hoạt hằng ngày cho người cao tuổi đặt ra nhiều khó khăn với các gia đình có người chăm sóc thường xuyên vắng mặt. Đồ án nghiên cứu và xây dựng hệ thống robot trợ lý ảo ứng dụng trí tuệ nhân tạo (Artificial Intelligence – AI), sử dụng thiết bị Android gắn trên robot làm bộ não xử lý. Hệ thống hướng đến việc tăng khả năng hỗ trợ kịp thời khi xảy ra sự cố, tạo phương thức tương tác thuận tiện bằng giọng nói cho người cao tuổi và giúp người chăm sóc theo dõi từ xa hiệu quả hơn.

Giải pháp gồm hai ứng dụng Android. Ứng dụng Robot khai thác camera để xử lý thị giác máy tính tại biên. Chuỗi xử lý kết hợp mô hình ước lượng tư thế You Only Look Once (YOLO), thuật toán Simple Online and Realtime Tracking (SORT) và đặc trưng tái định danh để theo dõi, điều khiển robot tự động di chuyển theo người cần chăm sóc; mô hình Long Short-Term Memory (LSTM) cùng bước hậu kiểm được sử dụng để nhận biết tình huống nghi té ngã. Khi phát hiện dấu hiệu bất thường, robot chủ động hỏi thăm bằng giọng nói để xác nhận tình trạng của người dùng; trong trường hợp người dùng cần trợ giúp hoặc không có phản hồi, hệ thống gửi cảnh báo và thiết lập cuộc gọi đến người chăm sóc. Bên cạnh đó, robot có khả năng trò chuyện, giúp nhắc nhở và điều khiển qua giọng nói, qua đó hỗ trợ một số nhu cầu giao tiếp và sinh hoạt thường ngày. Ứng dụng Caregiver cho phép người chăm sóc quản lý hồ sơ, thiết lập các nhắc nhở, xem nhật ký các nhắc nhở, cảnh báo té ngã và cuộc gọi; đồng thời có thể bật hoặc tắt các chức năng giám sát, gọi video hai chiều và điều khiển robot bằng cần điều khiển ảo.

Kết quả của đồ án là một nguyên mẫu tích hợp hai ứng dụng di động, khối xử lý trí tuệ nhân tạo, hạ tầng kết nối và phần cứng robot. Là sự tận dụng năng lực tính toán và cảm biến của thiết bị Android để giảm yêu cầu về phần cứng chuyên dụng. Là tiền đề cho việc tiếp tục nghiên cứu và phát triển các ứng dụng, sản phẩm công nghệ hỗ trợ chăm sóc người cao tuổi tại nhà. Hệ thống có thể được hoàn thiện thêm về độ tin cậy, bảo mật, khả năng ghép cặp nhiều thiết bị và các chức năng theo dõi sức khỏe, qua đó góp phần mở rộng khả năng ứng dụng trí tuệ nhân tạo và robot trong lĩnh vực chăm sóc người cao tuổi.

Từ khóa: robot trợ lý, chăm sóc người cao tuổi, phát hiện té ngã, thị giác máy tính, xử lý tiếng nói, Android, Internet vạn vật.

MỤC LỤC

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	iv
Danh mục hình ảnh	xi
Danh mục bảng biểu	xii
Danh mục từ viết tắt	xiv
1 Giới thiệu	1
1.1 Bối cảnh và tính cấp thiết của đề tài	1
1.2 Phát biểu bài toán	2
1.3 Câu hỏi nghiên cứu	3
1.4 Mục tiêu đề tài	3
1.4.1 Mục tiêu tổng quát	3
1.4.2 Mục tiêu cụ thể	4
1.5 Đối tượng và phạm vi nghiên cứu	4
1.5.1 Đối tượng nghiên cứu	4
1.5.2 Phạm vi chức năng	5
1.5.3 Giới hạn nghiên cứu	5
1.6 Phương pháp nghiên cứu	5
1.6.1 Khảo cứu và phân tích tài liệu	5
1.6.2 Phân tích yêu cầu và đồng thiết kế hệ thống	6
1.6.3 Phát triển lập và kiểm chứng theo mã nguồn	6
1.6.4 Kiểm thử và đánh giá	6
1.7 Các công trình và giải pháp liên quan	6
1.7.1 Các hướng tiếp cận cho bài toán phát hiện té ngã	7
1.7.2 Các giải pháp theo dõi người và bám mục tiêu	13
1.7.3 Chatbot AI và hội thoại hỗ trợ người cao tuổi	16
1.7.4 Các nền tảng robot tích hợp và bài học áp dụng	21

1.7.5	Khoảng trống nghiên cứu và định vị của đề tài	21
1.8	Đóng góp của đề tài	22
1.9	Cấu trúc báo cáo	23
1.10	Kết luận chương	23
2	Cơ sở lý thuyết	24
2.1	Nền tảng Intel OpenBot (OpenBot)	24
2.1.1	Giới thiệu và ý tưởng cốt lõi	24
2.1.2	Kiến trúc và các thành phần liên quan	24
2.1.3	Giải pháp kết nối và điều khiển	25
2.1.4	AI và cơ chế theo dõi đối tượng	26
2.1.5	Khả năng áp dụng cho đồ án	27
2.2	Nền tảng Thị giác máy tính (Computer Vision)	27
2.2.1	Bài toán Object Detection	28
2.2.2	Lịch sử phát triển Object Detection	28
2.2.3	Khái niệm Bounding Box	29
2.2.4	Intersection over Union (IoU)	30
2.2.5	True Positive, False Positive, False Negative	30
2.2.6	Đánh giá hiệu suất Mean Average Precision (mAP)	31
2.2.7	Các kiến trúc phát hiện đối tượng phù hợp thiết bị di động	32
2.2.8	Theo dõi đối tượng và tạo chuỗi thời gian	33
2.3	Họ mô hình YOLO và YOLO26-Pose	34
2.3.1	Quá trình phát triển của họ YOLO	34
2.3.2	Cấu trúc backbone–neck–head	35
2.3.3	Các thay đổi chính của YOLO26	36
2.3.4	Quy trình suy luận và hậu xử lý YOLO-Pose	36
2.3.5	Input và Output của YOLO-Pose	37
2.3.6	Các mức kích thước của YOLO26-Pose	38
2.4	Mô hình chuỗi thời gian (Sequence Modeling)	39
2.4.1	Mạng nơ-ron hồi quy (RNN)	39
2.4.2	Mạng Bộ nhớ Ngắn-Dài hạn (LSTM)	39
2.5	Xử lý Ngôn ngữ Tự nhiên (NLP) và Hệ thống Giọng nói	41
2.5.1	Mô hình ngôn ngữ lớn (LLM) và Kiến trúc Transformer	41
2.5.2	Khái niệm AI Agent	43

2.5.3	Mô hình Ngôn ngữ Lớn (Large Language Models)	43
2.5.4	Hệ thống chuyển đổi Giọng nói (STT & TTS)	50
2.5.5	Một số giải pháp Nhận dạng Giọng nói:	51
2.5.6	Phương pháp tiếp cận Xử lý Ngôn ngữ Tự nhiên (NLP) trong dự án	51
2.5.7	Phương thức triển khai: Điện toán Biên (Edge Computing) trên thiết bị Android	51
2.6	Giới thiệu Dataset Fall Detection – LE2I	52
2.6.1	Mục tiêu và đặc điểm của dataset LE2I	52
2.6.2	Dữ liệu video và Annotation	53
2.6.3	Ưu điểm và hạn chế của dataset LE2I	53
2.6.4	So sánh với các Dataset quy mô lớn (DiverseFALL10500)	54
2.7	Suy luận AI trên thiết bị và TensorFlow Lite	54
2.8	Firebase Authentication và Realtime Database	54
2.9	WebRTC, signaling và ICE	55
2.10	Bluetooth Low Energy và GATT	55
2.11	Động cơ BLDC và điều khiển FOC	56
2.12	An toàn và quản trị rủi ro AI	56
2.13	Kết luận chương	57
3	Phân tích và thiết kế hệ thống	58
3.1	Phân tích yêu cầu hệ thống	58
3.1.1	Bài toán, mục tiêu và phạm vi	58
3.1.2	Tác nhân, điều kiện vận hành và ranh giới hệ thống	60
3.1.3	Yêu cầu chức năng	61
3.1.4	Yêu cầu phi chức năng	63
3.2	Kiến trúc tổng quan hệ thống	64
3.2.1	Dẫn xuất kiến trúc từ yêu cầu	64
3.2.2	Kiến trúc logic tổng quan	65
3.2.3	Các module và trách nhiệm	66
3.3	Các luồng xử lý chính	69
3.3.1	Biểu đồ ca sử dụng	69
3.3.2	Luồng giám sát và xử lý té ngã	71
3.3.3	Luồng cuộc gọi khẩn cấp và điều khiển từ xa	72

3.3.4	Luồng hội thoại và nhắc lịch	74
3.4	Thiết kế và lựa chọn công nghệ cho các module	74
3.4.1	Module ứng dụng người chăm sóc và điều phối tại robot	77
3.4.2	Module cảm nhận, phát hiện pose và theo dõi ID0	77
3.4.3	Module tự bám theo và điều khiển chuyển động	79
3.4.4	Module phát hiện, hậu kiểm và xác nhận té ngã	79
3.4.5	Module hội thoại và lời nhắc	80
3.4.6	Module dữ liệu dùng chung và liên lạc thời gian thực	80
3.4.7	Module cầu nối BLE và chấp hành thân robot	81
3.4.8	Module chuẩn bị và triển khai mô hình AI	81
3.5	Thiết kế dữ liệu và hợp đồng giao tiếp	82
3.5.1	Cây dữ liệu Firebase	82
3.5.2	Hợp đồng điều khiển trên RTDB	83
3.5.3	Hợp đồng BLE và vòng điều khiển firmware	84
3.5.4	Cơ chế dừng an toàn nhiều tầng	85
3.6	An toàn, bảo mật và quyền riêng tư	85
3.7	Truy vết và quyết định kiến trúc	86
3.7.1	Ma trận truy vết yêu cầu	86
3.7.2	Đánh đổi kiến trúc	87
3.8	Kết luận chương	88
4	Hiện thực hệ thống	89
4.1	Kiến trúc hiện thực và ánh xạ công nghệ	89
4.2	Môi trường phát triển và tổ chức các thành phần	90
4.3	Hiện thực RobotApp	91
4.3.1	Khởi động, điều hướng và vòng đời dịch vụ	91
4.3.2	TrackingFragment và quyền sở hữu camera	91
4.3.3	Nạp và suy luận mô hình YOLO26n-Pose	92
4.3.4	SORT, Kalman, Hungarian và Re-ID	95
4.3.5	Khóa mục tiêu ID0 và sinh lệnh auto-follow	95
4.3.6	LSTM phát hiện té ngã	96
4.3.7	Hậu kiểm té ngã	96
4.3.8	Luồng xác nhận và cảnh báo	96
4.3.9	Chatbot giọng nói	97

4.3.10	Lịch nhắc	98
4.3.11	WebRTC trong RobotApp	98
4.3.12	RobotControlManager và BLE	98
4.4	Hiện thực CaregiverApp	99
4.4.1	Khởi động, xác thực và hồ sơ	99
4.4.2	Bật/tắt chức năng theo dõi	99
4.4.3	Lịch và nhật ký	99
4.4.4	Cuộc gọi đến nền	100
4.4.5	Video call và joystick	100
4.5	Hiện thực dữ liệu dùng chung	100
4.6	Huấn luyện và chuyển đổi mô hình AI	101
4.6.1	Xử lý dữ liệu	101
4.6.2	Trích xuất đặc trưng	101
4.6.3	Huấn luyện mô hình	102
4.6.4	Chuyển đổi và tối ưu mô hình	102
4.6.5	Kiểm tra tương đương sau chuyển đổi	103
4.7	Hiện thực firmware phần thân robot	103
4.7.1	Khởi tạo phần cứng	103
4.7.2	BLE GATT server	104
4.7.3	Deadzone và watchdog	104
4.8	Cấu hình và triển khai	104
4.9	Giới hạn của phiên bản hiện thực	105
4.10	Kết luận chương	105
5	Kết quả và thảo luận	106
5.1	Mục tiêu đánh giá	106
5.2	Cấu hình và tiêu chí đánh giá	106
5.3	Kết quả hiện thực và artifact	107
5.3.1	Mức độ hoàn thành thành phần	107
5.3.2	Kết quả đóng gói Android	108
5.3.3	Kết quả artifact mô hình	108
5.4	Kiểm thử tự động ở mức logic	109
5.4.1	Ánh xạ điều khiển	109
5.4.2	Hậu kiểm té ngã	109

5.5	Đánh giá hợp đồng tích hợp bằng kiểm tra tĩnh	110
5.5.1	Nhất quán Firebase	110
5.5.2	Nhất quán BLE	110
5.5.3	Chuỗi timeout	110
5.5.4	Quyền sở hữu tài nguyên	111
5.6	Kịch bản kiểm thử chức năng đầu cuối	111
5.7	Thiết kế phép đo hiệu năng	113
5.7.1	Độ trễ pipeline AI	113
5.7.2	Độ trễ điều khiển	113
5.7.3	WebRTC	113
5.7.4	Độ chính xác té ngã	113
5.8	Kết quả Benchmark Mô hình AI	114
5.8.1	Benchmark Độ chính xác (Accuracy & Metrics)	114
5.8.2	Benchmark Tốc độ suy luận (Inference Speed / FPS)	114
5.9	Thảo luận kết quả	115
5.9.1	Điểm mạnh	115
5.9.2	Hạn chế	115
5.9.3	Mối đe dọa tới tính hợp lệ	116
5.10	Mức đáp ứng mục tiêu	116
5.11	Khuyến nghị trước nghiệm thu và trình diễn	116
5.12	Kết luận chương	117
6	Kết luận	118
6.1	Tổng kết	118
6.2	Kết quả đạt được	119
6.3	Hạn chế	119
6.4	Hướng phát triển	120
6.4.1	Đánh giá AI có thể tái lập	120
6.4.2	Tăng độ bền theo dõi và phát hiện	120
6.4.3	An toàn và điều khiển robot	121
6.4.4	Backend và bảo mật nhiều người dùng	121
6.4.5	Khả năng hoạt động khi mất mạng	121
6.4.6	Cải thiện trải nghiệm người cao tuổi	121
6.4.7	Tối ưu triển khai	121

6.5	Kết luận cuối cùng	122
	Tài liệu tham khảo	123
	Phụ lục	133
	Phụ lục A – Đặc tả use case	133
	Phụ lục B – Hợp đồng dữ liệu và giao thức	137
	Phụ lục C – Danh mục module mã nguồn	138
	Phụ lục D – Manifest artifact AI	140
	Phụ lục E – Quy trình build và triển khai	141
	Phụ lục F – Phiếu ghi kết quả kiểm thử	142
	Phụ lục G – Checklist an toàn trình diễn	142

DANH SÁCH HÌNH VẼ

2.1	Minh họa hộp giới hạn của đối tượng trong ảnh.	29
2.2	Cấu trúc khái quát của một detector YOLO gồm backbone, neck và head.	35
2.3	Cấu trúc chi tiết của một tế bào LSTM (LSTM cell) với hệ thống các cổng kiểm soát thông tin.	40
2.4	Sơ đồ kiến trúc của mô hình Transformer với cơ chế Multi-Head Attention.	42
2.5	Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ	44
2.6	Kiến trúc Transformer với các khối Encoder và Decoder	45
2.7	Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention	46
2.8	Một số khung hình minh họa trong môi trường của dataset LE2I.	53
3.1	Kiến trúc tổng quan của hệ thống robot hỗ trợ chăm sóc người cao tuổi	66
3.2	Biểu đồ ca sử dụng tổng thể của hệ thống	70
3.3	Biểu đồ hoạt động của quy trình phát hiện, hậu kiểm và xử lý té ngã	71
3.4	Biểu đồ tuần tự của quá trình phát hiện té ngã và thiết lập cuộc gọi khẩn cấp	73
4.1	Kiến trúc hiện thực của hệ thống	90
4.2	Ví dụ khung hình té ngã trong dữ liệu tham khảo	97
4.3	Minh họa kiến trúc mạng LSTM phân tích chuỗi thời gian của các đặc trưng (Nguồn: Tham khảo)	102

DANH SÁCH BẢNG

1.1	Tỷ lệ các chiến lược phát hiện té ngã trong khảo sát của Newaz và Hanada [48]	7
1.2	So sánh bằng chứng và giới hạn của các hướng phát hiện té ngã tiêu biểu	10
1.3	So sánh các hướng tracking-by-detection liên quan đến đề tài	16
1.4	So sánh các phương án hiện thực chatbot trên Android	19
1.5	Đối chiếu các nhóm giải pháp với phạm vi tích hợp của đề tài	22
2.1	Các ý tưởng OpenBot được tham khảo trong đồ án	27
2.2	So sánh định tính các họ detector liên quan đến đồ án	33
2.3	Thứ tự keypoint người theo cấu hình COCO-Pose của YOLO26	38
2.4	So sánh các mức YOLO26-Pose ở chế độ end-to-end trên COCO	39
2.5	So sánh Transformer và CNN	47
2.6	So sánh các công nghệ Speech-to-Text.	51
3.1	Cụ thể hóa yêu cầu luận văn theo phạm vi đề tài	59
3.2	Tác nhân và trách nhiệm trong hệ thống	60
3.3	Danh sách yêu cầu chức năng	61
3.4	Yêu cầu phi chức năng	63
3.5	Từ nhóm yêu cầu đến quyết định phân rã kiến trúc	65
3.6	Các module trong kiến trúc logic của hệ thống	67
3.7	Tóm tắt các ca sử dụng chính	69
3.8	So sánh các lựa chọn công nghệ xuyên suốt hệ thống	74
3.9	Cấu trúc dữ liệu của Firebase Realtime Database	82
3.10	Các điểm chuẩn ánh xạ lệnh điều khiển	84
3.11	Mối đe dọa, bảo vệ hiện có và phân cần hoàn thiện	85
3.12	Truy vết từ yêu cầu đến module và kiểm thử	87
4.1	Các thành phần hiện thực của hệ thống	90
4.2	Hai mô hình YOLO26n-Pose được đóng gói trong RobotApp	92
4.3	Các hệ tọa độ trong pipeline YOLO26n-Pose của RobotApp	94
4.4	Các mô hình chính sau khi huấn luyện và chuyển đổi	103
5.1	Cấu hình phần mềm được đánh giá	106
5.2	Kết quả hiện thực theo thành phần	107
5.3	Artifact đóng gói hiện có	108
5.4	Đánh giá định tính các biến thể mô hình	109

5.5	Bộ unit test ánh xạ điều khiển	109
5.6	Bộ unit test hậu kiểm té ngã	110
5.7	Danh mục kiểm thử đầu cuối	111
5.8	Kết quả Benchmark độ chính xác của mô hình LSTM trên tập Test	114
5.9	Thời gian suy luận (Inference Time) trung bình trên Android	114
5.10	Đối chiếu mục tiêu và kết quả	116
6.11	Thông số BLE triển khai	139
6.12	Module chính của RobotApp	139
6.13	Module chính của CaregiverApp	140
6.15	Mẫu tổng hợp benchmark AI	143
6.16	Mẫu tổng hợp phát hiện té ngã theo sự cố	143
6.17	Mẫu tổng hợp độ trễ và an toàn điều khiển	143

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
AP	Average Precision	Độ chính xác trung bình
API	Application Programming Interface	Giao diện lập trình ứng dụng
BERT	Bidirectional Encoder Representations from Transformers	Biểu diễn mã hóa hai chiều từ Transformer
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu đối tượng phổ biến trong ngữ cảnh
CPU	Central Processing Unit	Bộ xử lý trung tâm
FFN	Feed-Forward Network	Mạng nơ-ron truyền thẳng
FOC	Field-Oriented Control	Điều khiển định hướng từ trường
FPS	Frames Per Second	Số khung hình trên giây
GATT	Generic Attribute Profile	Hồ sơ thuộc tính của Bluetooth Low Energy
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
GPT	Generative Pre-trained Transformer	Mô hình Transformer sinh được tiền huấn luyện
IoT	Internet of Things	Internet vạn vật
IoU	Intersection over Union	Tỷ lệ phần giao trên phần hợp
ICE	Interactive Connectivity Establishment	Cơ chế thiết lập đường kết nối qua NAT
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Mạng bộ nhớ dài-ngắn hạn
MLP	Multilayer Perceptron	Mạng perceptron đa lớp
MOTA	Multiple Object Tracking Accuracy	Độ chính xác theo dõi đa đối tượng
NAT	Network Address Translation	Biên dịch địa chỉ mạng

Từ viết tắt	Thuật ngữ tiếng Anh	Ý nghĩa tiếng Việt
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
NLU	Natural Language Understanding	Hiểu ngôn ngữ tự nhiên
NMS	Non-Maximum Suppression	Úc chế không cực đại
P2P	Peer-to-Peer	Kết nối ngang hàng
R-CNN	Region-based Convolutional Neural Network	Mạng nơ-ron tích chập dựa trên vùng
RAG	Retrieval-Augmented Generation	Sinh nội dung có tăng cường truy xuất
Re-ID	Person Re-identification	Tái định danh người
RGB	Red, Green, Blue	Hệ màu đỏ, xanh lá và xanh dương
RTDB	Realtime Database	Cơ sở dữ liệu thời gian thực Firebase
SDP	Session Description Protocol	Giao thức mô tả phiên truyền thông
SORT	Simple Online and Realtime Tracking	Thuật toán theo dõi trực tuyến thời gian thực
SSD	Single Shot MultiBox Detector	Mô hình phát hiện đối tượng một giai đoạn
SOTA	State of the Art	Trình độ tiên tiến nhất tại thời điểm xem xét
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
UID	User Identifier	Định danh người dùng
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên nền web
WHO	World Health Organization	Tổ chức Y tế Thế giới
YOLO	You Only Look Once	Họ mô hình thị giác máy tính thời gian thực

Chương 1 – Giới thiệu

1.1 Bối cảnh và tính cấp thiết của đề tài

Già hóa dân số đã trở thành bài toán hiện hữu đối với hệ thống y tế, an sinh xã hội và từng gia đình. Tổ chức Y tế Thế giới (WHO) dự báo số người từ 60 tuổi trở lên trên toàn cầu tăng từ khoảng 1 tỷ người năm 2020 lên 1,4 tỷ người vào năm 2030 và 2,1 tỷ người vào năm 2050; khoảng hai phần ba số người cao tuổi sẽ sống tại các quốc gia thu nhập thấp và trung bình [1]. Quá trình chuyển đổi này diễn ra nhanh hơn trước, trong khi nguồn nhân lực chăm sóc dài hạn, cơ sở hạ tầng và khả năng chi trả không tăng với cùng tốc độ.

Tại Việt Nam, áp lực còn rõ rệt hơn vì thời gian chuyển từ cơ cấu dân số “già hóa” sang “dân số già” tương đối ngắn. Dự báo dựa trên Tổng điều tra dân số và nhà ở năm 2019 cho thấy Việt Nam có thể trở thành quốc gia có dân số già vào khoảng năm 2036, khi nhóm từ 65 tuổi trở lên đạt xấp xỉ 14% dân số; đến năm 2069, người cao tuổi có thể chiếm 27,1% dân số [2]. Nghiên cứu về kinh tế chăm sóc người cao tuổi tại Việt Nam còn dự báo số người cần hỗ trợ thực hiện các hoạt động sống hằng ngày tăng từ khoảng 4,7 triệu năm 2025 lên 6,5 triệu năm 2035 [3]. Mô hình chỉ dựa vào chăm sóc trực tiếp bởi thành viên gia đình vì vậy sẽ ngày càng khó duy trì, đặc biệt tại đô thị khi người thân thường làm việc xa nhà.

Té ngã là một rủi ro tiêu biểu cần được phát hiện sớm. WHO định nghĩa té ngã là sự kiện khiến một người ngoài ý muốn nằm lại trên mặt đất, sàn hoặc một mức thấp hơn. Mỗi năm thế giới ghi nhận khoảng 684.000 ca tử vong do té ngã và 37,3 triệu trường hợp đủ nghiêm trọng để cần chăm sóc y tế; nhóm trên 60 tuổi chịu số ca tử vong cao nhất [4]. Trong nhà, sự cố có thể xảy ra tại thời điểm không có người chăm sóc. Khoảng thời gian từ khi xảy ra sự cố đến khi người thân biết và can thiệp vì vậy có ý nghĩa trực tiếp đối với mức độ hậu quả.

Trong bối cảnh đó, hệ thống phát hiện té ngã (Fall Detection System–FDS) có mục tiêu nhận biết sự kiện và tạo cảnh báo để rút ngắn thời gian chờ trợ giúp; nó không đồng nghĩa với hệ thống dự báo hay ngăn ngừa mọi cú ngã. Tổng quan của Newaz và Hanada cho thấy FDS đã phát triển theo nhiều nhánh cảm biến và kiến trúc trong hơn ba thập niên, đồng thời nhấn mạnh rằng hiệu quả kỹ thuật phải được xem xét cùng dữ liệu đánh giá, khả năng triển khai, bảo mật và riêng tư [48]. Phân biệt này giúp định vị đúng đề tài: nguyên mẫu hỗ trợ phát hiện và phản ứng sau tình huống nghi té ngã, không đưa

ra chẩn đoán y khoa hay cam kết phòng ngừa tai nạn.

Bên cạnh rủi ro té ngã, người cao tuổi thường phải quản lý nhiều thuốc và lịch sinh hoạt. Một nghiên cứu tổng quan có hệ thống về việc tuân thủ điều trị bằng thuốc ở người cao tuổi cho thấy tình trạng mắc đồng thời nhiều bệnh, sử dụng nhiều loại thuốc, suy giảm chức năng và phác đồ điều trị phức tạp đều có thể làm giảm mức độ tuân thủ [5]. Cô đơn và cô lập xã hội cũng là các yếu tố nguy cơ quan trọng đối với sức khỏe tinh thần ở tuổi già [6]. Do đó, một giải pháp hỗ trợ tại nhà không nên chỉ là camera giám sát thụ động; hệ thống cần kết hợp nhắc việc, tương tác tự nhiên, liên lạc trực tiếp và cơ chế cảnh báo có thể kiểm chứng.

Sự phổ biến của điện thoại thông minh tạo điều kiện xây dựng robot chi phí thấp. Điện thoại cung cấp camera, microphone, loa, kết nối mạng và năng lực tính toán đủ cho nhiều tác vụ thị giác máy tính. Công trình OpenBot chứng minh một điện thoại Android có thể đóng vai trò bộ não và hệ cảm biến cho thân xe robot giá thấp, đồng thời hỗ trợ các tác vụ như bám theo người và điều hướng [10]. Cách tiếp cận này phù hợp với bối cảnh của đề tài: tái sử dụng phần cứng phổ biến, đưa suy luận AI đến gần nguồn dữ liệu và chỉ dùng vi điều khiển cho vòng điều khiển động cơ.

Từ các nhu cầu trên, đề tài **“Nghiên cứu và phát triển robot trợ lý ảo ứng dụng trí tuệ nhân tạo hỗ trợ chăm sóc và giám sát người cao tuổi tại nhà”** xây dựng một nguyên mẫu tích hợp thay vì các chức năng rời rạc. Hệ thống gồm ứng dụng RobotApp trên thiết bị Android gắn với phần thân robot hỗ trợ di chuyển, ứng dụng CaregiverApp cho người chăm sóc, firmware ESP32 điều khiển hai động cơ BLDC ở thân robot và chuỗi mô hình AI được chuyển đổi để suy luận trên thiết bị. Giá trị cốt lõi nằm ở việc kết nối các giải pháp công nghệ thành một hệ thống tương đối hoàn chỉnh hỗ trợ chăm sóc người cao tuổi tại nhà.

1.2 Phát biểu bài toán

Bài toán đặt ra là thiết kế một robot di động trong nhà có thể hỗ trợ người cao tuổi mà vẫn cho phép người chăm sóc theo dõi và can thiệp từ xa. Hệ thống phải giải quyết đồng thời bốn nhóm vấn đề:

1. **Nhận biết an toàn:** khai thác camera để phát hiện và theo dõi đúng người mục tiêu, nhận biết chuỗi tư thế nghi té ngã trong điều kiện ảnh thay đổi và hạn chế cảnh báo giả từ hành vi nằm/ngồi bình thường.

2. **Tương tác và chăm sóc:** hỗ trợ hội thoại giọng nói tiếng Việt, phát lời nhắc theo lịch và hỏi xác nhận khi phát hiện sự cố.
3. **Liên lạc và điều khiển:** thiết lập cuộc gọi âm thanh/hình ảnh thời gian thực; truyền lệnh joystick từ CaregiverApp qua hạ tầng đám mây tới RobotApp rồi qua BLE tới ESP32.
4. **An toàn hệ thống:** loại bỏ lệnh cũ hoặc không hợp lệ, dừng robot khi mất kết nối, tránh tranh chấp nguồn điều khiển và bảo vệ dữ liệu người dùng.

Các yêu cầu trên có quan hệ chặt chẽ. Chẳng hạn, camera vừa phục vụ cuộc gọi WebRTC vừa cung cấp ảnh cho AI; nếu hai thành phần đồng thời mở camera sẽ gây xung đột tài nguyên. Tương tự, joystick từ xa và chế độ tự bám đều có thể phát sinh lệnh chuyển động; nếu không xác định quyền sở hữu, robot có thể nhận hai chỉ thị trái ngược. Vì vậy, đề tài được tiếp cận như một bài toán đồng thiết kế phần mềm di động, AI biên, giao tiếp thời gian thực và điều khiển nhúng.

1.3 Câu hỏi nghiên cứu

- Có thể tổ chức kiến trúc nào để một thiết bị Android đồng thời đảm nhiệm camera, suy luận AI, hội thoại, liên lạc và cầu nối điều khiển phần thân robot?
- Làm thế nào duy trì danh tính người cần theo dõi khi vùng bao thay đổi mạnh, bị che khuất ngắn hạn hoặc xuất hiện thêm người trong khung hình?
- Làm thế nào kết hợp thông tin tư thế theo thời gian với luật hậu kiểm để giảm cảnh báo giả và vẫn giữ nguyên tắc an toàn khi người dùng không phản hồi?
- Hợp đồng dữ liệu nào bảo đảm lệnh điều khiển từ xa có thể được phát hiện là mới, hợp lệ và được dừng tự động khi đường truyền gián đoạn?
- Những giới hạn nào cần được công bố để phân biệt một hệ thống hỗ trợ nghiên cứu với thiết bị y tế hoặc sản phẩm thương mại hoàn chỉnh?

1.4 Mục tiêu đề tài

1.4.1 Mục tiêu tổng quát

Nghiên cứu, thiết kế và hiện thực nguyên mẫu robot trợ lý ảo có khả năng tương tác bằng giọng nói, nhắc lịch, theo dõi người, phát hiện tình huống nghi ngờ, gọi video

và cho phép người chăm sóc điều khiển từ xa; trong đó điện thoại thông minh là nền tảng tính toán chính và ESP32 đảm nhiệm điều khiển hai động cơ.

1.4.2 Mục tiêu cụ thể

- Xây dựng RobotApp bằng Android/Kotlin để quản lý camera, suy luận mô hình TFLite, hội thoại, nhắc lịch, WebRTC và kết nối BLE.
- Xây dựng CaregiverApp để quản lý hồ sơ, lịch nhắc, trạng thái theo dõi, cảnh báo, lịch sử, cuộc gọi và joystick điều khiển robot.
- Xây dựng firmware ESP32 dùng BLE GATT, SimpleFOC và phản hồi encoder AS5600 để điều khiển vận tốc hai động cơ BLDC.
- Xây dựng chuỗi xử lý thị giác gồm mô hình pose, giải mã/NMS, SORT, Kalman, Hungarian, Re-ID, khóa người mục tiêu ID0 và LSTM phát hiện té ngã.
- Chuyển đổi mô hình từ định dạng huấn luyện sang ONNX/TensorFlow Lite, tạo biến thể đầu vào 320×320 và FP16/GPU để triển khai trên Android.
- Thiết kế cơ chế xác nhận sự cố bằng giọng nói, ghi nhận cảnh báo và tự khởi tạo cuộc gọi khi phản hồi cho thấy nguy hiểm hoặc không có phản hồi.
- Xác định các yêu cầu an toàn, bảo mật, khả năng bảo trì và giới hạn của nguyên mẫu thông qua kiểm tra mã nguồn và kiểm thử có thể tái lập.

1.5 Đối tượng và phạm vi nghiên cứu

1.5.1 Đối tượng nghiên cứu

Đối tượng kỹ thuật gồm nền tảng robot dựa trên smartphone; hai ứng dụng Android; thị giác máy tính và học sâu trên thiết bị; theo dõi đa đối tượng; xử lý chuỗi tư thế; nhận dạng và tổng hợp tiếng nói; mô hình ngôn ngữ lớn; Firebase Realtime Database; WebRTC; BLE GATT; điều khiển vận tốc BLDC bằng SimpleFOC; và quy trình chuyển đổi mô hình sang TFLite.

Đối tượng sử dụng gồm người cao tuổi sinh hoạt tại nhà và người thân/người chăm sóc từ xa. Trong nguyên mẫu, RobotApp và CaregiverApp của một cặp thiết bị đăng nhập cùng tài khoản Firebase; toàn bộ kênh điều khiển, trạng thái AI, signaling và nhật ký thời gian thực được cách ly dưới `pair_sessions/{uid}`. Cách tổ chức này tách dữ liệu

giữa các tài khoản, nhưng mỗi tài khoản hiện mới đại diện cho một cặp robot–caregiver và chưa mô hình hóa nhiều robot hay nhiều vai trò trong cùng gia đình.

1.5.2 Phạm vi chức năng

- Theo dõi một người mục tiêu, tự bám theo ở mức điều khiển chuyển động cơ bản và phát hiện tình huống nghi ngờ từ camera RGB.
- Hội thoại bằng giọng nói thông qua SpeechRecognizer, Groq API và Android Text-ToSpeech; không huấn luyện một LLM riêng.
- Tạo và thực hiện lịch nhắc; ghi nhật ký phản hồi; hiển thị lịch sử cảnh báo và cuộc gọi.
- Gọi video hai chiều bằng WebRTC, sử dụng Firebase RTDB làm kênh trao đổi offer, answer, ICE candidate và trạng thái; không xây dựng signaling server riêng.
- Điều khiển robot qua chuỗi CaregiverApp–Firebase–RobotApp–BLE–ESP32; CaregiverApp không kết nối trực tiếp tới ESP32.

1.5.3 Giới hạn nghiên cứu

Nguyên mẫu không được định vị là thiết bị chẩn đoán hay thiết bị y tế. Dữ liệu té ngã chủ yếu là tình huống dừng lại; khả năng tổng quát với người cao tuổi thật, ánh sáng yếu, vật cản, camera rung và nhiều người cần nghiên cứu thực địa có phê duyệt đạo đức. Hệ thống phụ thuộc Internet cho Firebase, Groq và thiết lập cuộc gọi; khi mất Internet, vòng điều khiển cục bộ BLE có thể dừng an toàn nhưng các chức năng từ xa không hoạt động. Việc đánh giá lâm sàng, chứng nhận an toàn điện/cơ khí, khả năng chống nước, độ bền dài hạn và tuân thủ quy định thiết bị y tế nằm ngoài phạm vi đồ án.

1.6 Phương pháp nghiên cứu

1.6.1 Khảo cứu và phân tích tài liệu

Nhóm tổng hợp nguồn từ tổ chức quốc tế, tiêu chuẩn kỹ thuật, bài báo bình duyệt và tài liệu chính thức. Các nhóm tài liệu bao gồm già hóa và té ngã, robot hỗ trợ, phát hiện đối tượng/tư thế, theo dõi, LSTM, giao tiếp thời gian thực, cơ sở dữ liệu thời gian thực, BLE và điều khiển FOC. Kết quả khảo cứu được dùng để xác lập yêu cầu và lựa chọn kiến trúc, không dùng thay thế đánh giá trên nguyên mẫu.

Riêng đối với phát hiện té ngã, tài liệu được phân loại theo taxonomy của các tổng quan gần đây: IoT/cloud, thiết bị đeo, smartphone, cảm biến môi trường, thị giác và các tín hiệu sinh học/động học [48]. Sau lớp phân loại, nhóm đối chiếu lại các bài nghiên cứu gốc để lấy đúng flow thuật toán, cấu hình cảm biến, bộ dữ liệu và điều kiện thử nghiệm. Cách làm này hạn chế việc dùng một bài tổng quan làm bằng chứng thay cho kết quả thực nghiệm ban đầu.

1.6.2 Phân tích yêu cầu và đồng thiết kế hệ thống

Từ kịch bản chăm sóc tại nhà, nhóm xác định tác nhân, use case, yêu cầu chức năng/phi chức năng và tình huống lỗi. Thiết kế được chia theo ranh giới triển khai: Care-giverApp, dịch vụ Internet, RobotApp và firmware. Các hợp đồng giao tiếp được mô tả bằng cấu trúc dữ liệu, đơn vị, miền giá trị, timestamp, timeout và hành vi khi lỗi.

1.6.3 Phát triển lập và kiểm chứng theo mã nguồn

Các chức năng được phát triển theo lát cắt dọc: từ giao diện người chăm sóc, bản ghi RTDB, xử lý tại RobotApp đến payload BLE và chuyển động động cơ. Đối với AI, quy trình đi từ mô hình huấn luyện, xuất ONNX/TFLite, kiểm tra tensor, tích hợp Android, theo dõi mục tiêu, xử lý chuỗi và hậu kiểm. Mã nguồn đang triển khai được xem là nguồn sự thật khi có khác biệt với tài liệu cũ.

1.6.4 Kiểm thử và đánh giá

Đánh giá gồm bốn mức: kiểm tra tĩnh hợp đồng và cấu hình; unit test cho ánh xạ lệnh và luật hậu kiểm; build ứng dụng; thử nghiệm tích hợp trên thiết bị với camera, Firebase, WebRTC, BLE và robot. Chỉ số gồm khả năng build, tỷ lệ test đạt, thời gian phản hồi từng chặng, số khung hình xử lý mỗi giây, precision/recall/F1 của cảnh báo té ngã, độ ổn định ID và tỷ lệ dừng an toàn. Khi chưa có log đo đáng tin cậy, báo cáo ghi rõ trạng thái “chưa đo” thay vì suy diễn số liệu.

1.7 Các công trình và giải pháp liên quan

Phạm vi liên quan trực tiếp đến đề tài không chỉ là robot chăm sóc nói chung mà gồm ba bài toán kỹ thuật có quan hệ chặt chẽ: (i) nhận biết một sự kiện té ngã từ chuỗi ảnh; (ii) duy trì đúng người mục tiêu khi robot và người cùng di chuyển; và (iii) tạo kênh hội thoại tự nhiên nhưng không làm suy yếu quy trình phản ứng an toàn. Newaz và Hanada phân loại nghiên cứu FDS thành các nhóm IoT, thiết bị đeo, cloud, thị giác,

smartphone, âm thanh, tín hiệu sinh học và tín hiệu động học; đồng thời khảo sát cảm biến, thuật toán, bộ dữ liệu, kiểm định, bảo mật và riêng tư [48]. Dựa trên khung rộng đó, phần này đi sâu vào các nhánh sát với nguyên mẫu và phân tích theo luồng xử lý, thay vì chỉ liệt kê sản phẩm.

1.7.1 Các hướng tiếp cận cho bài toán phát hiện té ngã

Tổng quan sự phát triển của các phương pháp phát hiện té ngã. Trong hơn ba thập niên, bài toán phát hiện té ngã đã được nghiên cứu trên nhiều loại tín hiệu và nền tảng tính toán khác nhau. Tổng quan của Newaz và Hanada khảo sát các công trình xuất hiện trên Google Scholar trong giai đoạn 1990–2022, từ đó chọn 75 nghiên cứu để phân tích về cảm biến, thuật toán, bộ dữ liệu, phương pháp kiểm định, bảo mật và quyền riêng tư [48]. Kết quả cho thấy tín hiệu sinh học chiếm 22,69% số công trình được thống kê, tiếp theo là thị giác với 20,87%, smartphone 18,56%, tín hiệu động học 13,27%, điện toán đám mây 10,38%, IoT 6,68%, thiết bị đeo 4,48% và âm thanh 3,07%. Phân bố này được trình bày trong Bảng 1.1.

Bảng 1.1: Tỷ lệ các chiến lược phát hiện té ngã trong khảo sát của Newaz và Hanada [48]

Thứ tự	Chiến lược	Tỷ lệ	Dữ liệu hoặc vai trò điển hình
1	Tín hiệu sinh học	22,69%	ECG, EEG, EMG hoặc các biến đổi sinh lý liên quan tới sự kiện
2	Dựa trên thị giác	20,87%	Ảnh RGB/depth, silhouette, optical flow, hộp người và keypoint
3	Dựa trên smart-phone	18,56%	IMU tích hợp, xử lý cục bộ, định vị và truyền cảnh báo
4	Tín hiệu động học	13,27%	Gia tốc, vận tốc góc, tư thế và quỹ đạo chuyển động
5	Dựa trên cloud	10,38%	Lưu trữ, suy luận từ xa, tổng hợp sự kiện và thông báo
6	Dựa trên IoT	6,68%	Kết nối cảm biến, gateway, dịch vụ giám sát và người chăm sóc
7	Thiết bị đeo	4,48%	Cảm biến gắn trên cơ thể, đồng hồ, vòng tay hoặc thắt lưng
8	Phân tích âm thanh	3,07%	Va chạm, tiếng kêu cứu, định vị nguồn âm và đặc trưng phổ

Các tỷ lệ trên phản ánh mức độ xuất hiện của từng chiến lược trong tập tài liệu được khảo sát, không biểu thị thị phần thương mại hay thứ hạng về độ chính xác. Ranh giới giữa các nhóm cũng không hoàn toàn tách biệt: smartphone có thể đồng thời đảm nhiệm vai trò cảm biến, thiết bị xử lý biên và nút kết nối IoT; điện toán đám mây chủ yếu cung cấp năng lực lưu trữ, phân tích và truyền thông báo chứ không trực tiếp tạo ra

bằng chứng về cú ngã. Do dữ liệu của bài tổng quan kết thúc ở năm 2022, các tỷ lệ này được sử dụng để mô tả xu hướng lịch sử; những nghiên cứu công bố từ năm 2023 trở đi được xem xét bổ sung trong các phần tiếp theo.

Xét theo bản chất dữ liệu đầu vào, các phương pháp có thể được hệ thống thành ba nhóm lớn: tín hiệu thu trực tiếp trên cơ thể, tín hiệu ghi nhận từ môi trường và thông tin thị giác trong video. IoT, smartphone và điện toán đám mây tạo thành lớp hạ tầng hỗ trợ thu nhận, xử lý và chuyển tiếp cảnh báo cho các nhóm phương pháp này. Sự phân chia đó vừa làm rõ ưu thế của từng nguồn dữ liệu, vừa cho phép đánh giá mức độ phù hợp với một robot chăm sóc hoạt động tự chủ trong không gian sống.

Phương pháp dựa trên thiết bị đeo và tín hiệu chuyển động. Các hệ thống thuộc nhóm này thường sử dụng gia tốc kế, con quay hồi chuyển hoặc IMU gắn tại thắt lưng, ngực, cổ tay hay tích hợp sẵn trong điện thoại. Tín hiệu được lấy mẫu, lọc nhiễu và chia thành các cửa sổ thời gian trước khi đưa qua luật ngưỡng hoặc mô hình phân loại để xác nhận sự kiện. Một đại lượng thường được sử dụng là độ lớn vector gia tốc

$$a = \sqrt{a_x^2 + a_y^2 + a_z^2}, \quad (1.1)$$

sau đó đổi chiều diễn biến giảm tải, va chạm, thay đổi góc thân và trạng thái bất động. Bourke, O'Brien và Lyons đã đánh giá một thuật toán ngưỡng sử dụng gia tốc kế ba trục gắn trên cơ thể, qua đó cho thấy khả năng hiện thực bộ phát hiện có chi phí tính toán thấp [49]. Trên nền tảng smartphone, Abbate và cộng sự bổ sung bước nhận biết những hoạt động thường ngày dễ bị nhầm với té ngã trước khi tự động gửi yêu cầu trợ giúp [50]. Việc xem xét cả trạng thái trước và sau va chạm giúp giảm sự phụ thuộc vào một đỉnh gia tốc đơn lẻ.

Các phương pháp dựa trên chuyển động có chi phí tính toán thấp, không ghi lại hình ảnh trong không gian sống, ít chịu ảnh hưởng của điều kiện chiếu sáng và vẫn hoạt động khi người dùng ở ngoài tầm quan sát của camera. Khi luật ngưỡng được thay bằng SVM, random forest, CNN một chiều hoặc LSTM/GRU, mô hình có khả năng học quan hệ giữa nhiều pha chuyển động. Bộ dữ liệu SisFall, được thu từ hai gia tốc kế và một con quay đặt tại thắt lưng, bao gồm nhiều hoạt động thường ngày và kiểu ngã khác nhau [51]. Kết quả đánh giá thay đổi đáng kể giữa các nhóm tuổi cho thấy mô hình huấn luyện chủ yếu bằng những cú ngã dàn dựng của người trẻ có thể suy giảm khả năng khái quát khi áp dụng cho người cao tuổi.

Hạn chế lớn nhất nằm ở sự tuân thủ của người dùng: thiết bị phải được đeo đúng vị trí và hướng, duy trì đủ pin và không bị bỏ quên. Những hoạt động như chạy, nhảy, ngồi mạnh hoặc đặt điện thoại xuống mặt bàn có thể tạo xung gia tốc tương tự va chạm, trong khi một cú trượt chậm lại có khả năng không vượt ngưỡng phát hiện. Tín hiệu sinh học cung cấp thêm ngữ cảnh về trạng thái cơ thể nhưng đòi hỏi cảm biến chuyên dụng, dễ chịu ảnh hưởng của tiếp xúc điện cực và làm tăng mức độ nhạy cảm của dữ liệu. Đối với robot chăm sóc, yêu cầu người cao tuổi luôn mang thêm một thiết bị sẽ làm giảm tính tự nhiên của quá trình sử dụng; vì vậy IMU phù hợp hơn với vai trò cảm biến hỗ trợ.

Phương pháp dựa trên cảm biến môi trường. Thay vì gắn cảm biến lên cơ thể, nhóm phương pháp này bố trí thiết bị thu nhận trong không gian cần giám sát. Acoustic-FADE sử dụng mảng microphone và SRP-PHAT để định vị nguồn âm, beamforming để tăng cường tín hiệu, MFCC để biểu diễn đặc trưng phổ, đồng thời khai thác độ cao nguồn âm nhằm hỗ trợ phân biệt tiếng ngã với các âm thanh khác [52]. Trên tập thử nghiệm gồm 120 âm thanh ngã và 120 âm thanh không ngã được mô phỏng, hệ thống đạt độ nhạy 100% tại độ đặc hiệu 97%. Mặc dù cho kết quả cao trong điều kiện thí nghiệm, tín hiệu âm thanh dễ thay đổi theo tiếng nền, độ vang, vật liệu sàn và âm thanh do đồ vật rơi; dữ liệu dàn dựng cũng chưa phản ánh đầy đủ sự đa dạng của tai nạn tự nhiên.

Camera chiều sâu và hệ stereo bổ sung quan hệ hình học ba chiều giữa cơ thể với mặt sàn. Stone và Skubic xây dựng một quy trình hai giai đoạn trên Kinect: trước hết xác định các đoạn chuỗi trong đó người ở gần sàn, sau đó dùng tập hợp cây quyết định để ước lượng khả năng đã xảy ra cú ngã [53]. Công trình được đánh giá trong thời gian dài tại 13 căn hộ, ghi nhận 454 lần ngã nhưng chỉ có chín trường hợp diễn ra tự nhiên. Sự chênh lệch này phản ánh khó khăn trong việc thu thập dữ liệu té ngã thực tế có nhãn đáng tin cậy. Solbach và Tsotsos kết hợp CNN ước lượng tư thế với camera stereo để khôi phục pose 3D và mặt phẳng sàn [54]. Thông tin hình học giúp giảm ảnh hưởng của màu sắc và nền cảnh, nhưng yêu cầu phần cứng, phạm vi đo và quy trình hiệu chuẩn riêng.

Radar mmWave khai thác khoảng cách, vận tốc Doppler hoặc point cloud mà không tạo ra hình ảnh có thể nhận dạng khuôn mặt, đồng thời vẫn hoạt động trong điều kiện thiếu sáng. Hệ thống mmFall sử dụng point cloud 4D và hybrid variational RNN autoencoder để học phân bố của các hoạt động bình thường, sau đó kết hợp điểm bất thường với sự hạ thấp của tâm cơ thể [56]. Trong 50 cú ngã thử nghiệm, phương pháp phát hiện được 98% và tạo hai báo động giả. Tuy nhiên, độ thưa của point cloud, phản xạ đa đường và cách bố trí phòng có thể làm thay đổi đáng kể tín hiệu. Để khắc phục giới

hạn của từng cảm biến riêng lẻ, Kwolek và Kepski sử dụng IMU để kích hoạt một sự kiện tiềm năng, rồi xác minh bằng ảnh chiều sâu, đặc trưng hình dạng, khoảng cách tới sàn và SVM [55]. Hợp nhất cảm biến giúp các nguồn dữ liệu bù trừ điểm mù cho nhau, nhưng kéo theo chi phí phần cứng, đồng bộ thời gian, hiệu chuẩn và bảo trì cao hơn.

Bảng 1.2: So sánh bằng chứng và giới hạn của các hướng phát hiện té ngã tiêu biểu

Hướng	Bằng chứng chính	Ưu điểm	Giới hạn khi triển khai
IMU/thiết bị đeo	Gia tốc, vận tốc góc, góc thân, bất động	Nhẹ, riêng tư, không phụ thuộc camera	Phải đeo/sạc đúng cách; nhạy vị trí và người dùng
Âm thanh	Va chạm, MFCC, vị trí và độ cao nguồn âm	Không đeo, không lưu ảnh	Nhạy tiếng nền, độ vang, đồ vật rơi và vật liệu sàn
Depth/stereo	Silhouette/pose 3D, tâm người, khoảng cách tới sàn	Hình học ba chiều rõ, giảm phụ thuộc màu sắc	Cần phần cứng và hiệu chuẩn; vùng quan sát cố định
Radar mmWave	Doppler, point cloud, quỹ đạo tâm cơ thể	Hoạt động thiếu sáng, bảo vệ riêng tư tốt hơn RGB	Dữ liệu thừa, đa đường, chi phí cảm biến
RGB một khung hình	Hộp người/ngã và đặc trưng hình ảnh	Nhanh, định vị trực tiếp, tận dụng camera sẵn có	Dễ nhầm lẫn/cúi; nhạy ánh sáng, góc nhìn và che khuất
RGB pose–thời gian	Keypoint, quỹ đạo khớp, thay đổi tư thế theo chuỗi	Mô tả được diễn biến cú ngã, phù hợp xử lý biên	Lỗi pose/tracking lan sang LSTM; cần dữ liệu chuỗi đa dạng
Đa cảm biến	Quyết định từ hai hoặc nhiều nguồn	Bù điểm mù và tăng khả năng xác minh	Tăng chi phí, đồng bộ, hiệu chuẩn và bảo trì

Phương pháp dựa trên thị giác. Với tỷ lệ 20,87%, thị giác là nhóm lớn thứ hai trong khảo sát của Newaz và Hanada [48]. Sự phát triển của camera tích hợp và bộ xử lý di động khiến hướng tiếp cận này ngày càng phù hợp với các hệ thống giám sát tại biên. Từ video, hệ thống có thể đồng thời ước lượng vị trí người, hình dạng hộp bao, tư thế khớp, tốc độ hạ thấp cơ thể và trạng thái sau va chạm. Khung hình gắn với sự kiện còn hỗ trợ người chăm sóc xác minh tình huống từ xa. Những lợi thế đó đi kèm các vấn đề về quyền riêng tư, ánh sáng yếu, che khuất, góc quan sát và chuyển động nền khi camera được đặt trên robot di động.

Các phương pháp thị giác ban đầu thường dùng silhouette, tỷ lệ chiều cao–chiều rộng, hướng trục chính hoặc lịch sử chuyển động. Hướng optical flow biểu diễn chuyển động giữa các khung hình; Núñez-Marcos, Azkune và Arganda-Carreras xếp chồng op-

tical flow rồi dùng CNN dựa trên VGG-16 để phân lớp đoạn video [59]. Cách này nắm được động học tốt hơn ảnh tĩnh, nhưng bước tính dòng quang học tốn tài nguyên và đặc biệt nhạy với chuyển động camera của robot.

Với detector một giai đoạn, ảnh RGB đi qua mạng YOLO, ngưỡng độ tin cậy và NMS để trả về hộp người ngã. Công trình “Visionary Vigilance” xây dựng Diverse-FALL10500 với khoảng 10.500 ảnh đa dạng và tối ưu YOLOv8s bằng Focus cùng CBAM [58]. Detector ảnh có tốc độ tốt và định vị trực tiếp sự cố, nhưng một tư thế nằm trong ảnh đơn không chứng minh đã có chuyển tiếp té ngã: người dùng có thể nằm nghỉ, cúi nhặt đồ, tập thể dục hoặc bị quan sát ở góc bất lợi. Vì vậy, accuracy theo khung hình cao vẫn có thể dẫn đến nhiều cảnh báo giả khi camera hoạt động hàng giờ.

Ước lượng pose chuyển người thành tập keypoint (x, y, c) , trong đó c là độ tin cậy. Sau khi chuẩn hóa theo hộp người hoặc kích thước cơ thể, hệ thống có thể tạo các đặc trưng như góc thân, tỷ lệ chiều cao–chiều rộng, độ cao tương đối và vận tốc của hông, vai, gối. So với pixel RGB, skeleton giảm phụ thuộc vào màu quần áo và nền, đồng thời làm rõ diễn biến động học. Tuy nhiên, che khuất và góc nhìn xấu có thể làm mất keypoint; bởi vậy pose phải được gắn với đúng track trước khi ghép thành chuỗi. Juraev và cộng sự cũng chỉ ra các khó khăn về thiếu dữ liệu ngã thực, thay đổi góc camera/ánh sáng và che khuất; dữ liệu tổng hợp có thể hỗ trợ, nhưng lựa chọn mô hình thời gian vẫn phải được kiểm chứng trên miền triển khai [60].

Kết hợp ước lượng tư thế với mô hình thời gian. Hạn chế của quyết định dựa trên từng khung hình tạo động lực kết hợp bộ ước lượng tư thế với mô hình chuỗi. Trong kiến trúc này, YOLO-pose xác định vị trí người và các khớp ở mỗi thời điểm, còn LSTM khai thác sự phụ thuộc giữa các khung liên tiếp để phân biệt một quá trình té ngã với tư thế nằm hoặc cúi người thông thường. Luồng xử lý tổng quát gồm

video RGB $\rightarrow$ YOLO-pose $\rightarrow$ hộp người và keypoint $\rightarrow$ gán ID/track $\rightarrow$ chuẩn hóa và tạo đặc trưng $\rightarrow$ cửa sổ chuỗi $\rightarrow$ LSTM $\rightarrow$ làm mượt/hậu kiểm $\rightarrow$ sự kiện cảnh báo.

Nhờ duy trì trạng thái theo thời gian, LSTM có thể học sự nối tiếp giữa mất thăng bằng, hạ thấp nhanh, va chạm và trạng thái nằm bất thường thay vì chỉ nhận dạng một dáng người đơn lẻ [38]. YOLO-pose và LSTM vì thế đảm nhiệm hai vai trò bổ sung: mô hình đầu trích xuất biểu diễn không gian, còn mô hình sau diễn giải sự biến đổi của biểu diễn đó theo thời gian.

FallNet là một hiện thực tiêu biểu của kiến trúc này. Mô hình được phát triển từ

YOLOv8n-pose, trong đó khối C2F được thay bằng DyC2F nhằm cải thiện khả năng trích xuất đặc trưng động và giảm chi phí tính toán; quỹ đạo keypoint sau đó được đưa vào LSTM để nhận biết sớm xu hướng ngã [61]. Nghiên cứu báo cáo độ chính xác 97%, precision 92%, recall 97% và F1-score 94% với 3,4 triệu tham số cùng 6,4 GFLOPs. Khi triển khai trên Raspberry Pi 3B+, nguyên mẫu đạt 13 FPS với độ trễ xử lý tại thiết bị khoảng 100 ms. Việc báo cáo đồng thời chất lượng phân loại, quy mô mô hình và tốc độ trên phần cứng thực làm cho công trình có giá trị tham khảo trực tiếp đối với các hệ thống AI biên.

Vai trò của bước theo dõi được thể hiện rõ hơn trong YOSAP-LSTM. Hệ thống dùng YOLOv8 để phát hiện người, SORT để duy trì định danh, AlphaPose để trích xuất skeleton và mạng 1D CNN-LSTM để phân loại chuỗi [62]. Trên bộ dữ liệu công trường của nghiên cứu, mô hình đạt độ chính xác 98,66%, độ nhạy 97,32%, độ đặc hiệu 99,10% và tốc độ 6,44 FPS trên NVIDIA Jetson Xavier NX. Khả năng xử lý nhiều người và các mức che khuất khác nhau cho thấy tracking là mất xích cần thiết trước khi hình thành chuỗi pose. Tuy nhiên, miền công trường, cấu hình Jetson và dữ liệu chuyên biệt khác đáng kể so với môi trường nhà ở trên smartphone; do đó kết quả 98,66% không thể được xem là hiệu năng kỳ vọng của đồ án.

Ở một biến thể khác, Liu và cộng sự đề xuất HFDMA-Pose, cải tiến YOLOv8s bằng SPD-Conv, bổ sung đầu phát hiện vật thể nhỏ và sử dụng BCIoU; kết quả của detector được chuyển sang AlphaPose và kết hợp với các đặc trưng tức thời cũng như trạng thái để xác định cú ngã [63]. Trên Le2i và bộ MPFDD nhiều người, phương pháp cải thiện tương đối so với AlphaPose 4,30% về độ chính xác, 4,57% về F1 và 37,50% về FPS. Mặc dù không sử dụng LSTM, nghiên cứu cho thấy chất lượng phát hiện người và keypoint quyết định trực tiếp chất lượng của tầng nhận dạng hành động phía sau; đồng thời, bằng chứng tức thời chỉ trở nên đáng tin cậy khi được đặt trong một trạng thái kéo dài theo thời gian.

Các con số trên không nên đặt cạnh nhau như một bảng xếp hạng. FallNet, YOSAP-LSTM và HFDMA-Pose dùng miền dữ liệu, định nghĩa nhãn, cách chia tập, phần cứng và đơn vị đo khác nhau. Accuracy còn có thể che giấu mất cân bằng lớp; trong chăm sóc dài hạn, recall của cú ngã, precision, F1, số báo giả mỗi giờ, độ trễ phát hiện và khả năng tổng quát sang người/căn phòng chưa thấy đều quan trọng. Đánh giá đáng tin cậy cần tách train/test theo người hoặc video, không chia ngẫu nhiên các khung gần nhau của cùng một cảnh, và phải thử bổ sung trên camera thực của robot. CAUCAFall là một nguồn dữ liệu hữu ích do có thay đổi khoảng cách, hướng ngã và

ánh sáng trong môi trường ít kiểm soát, nhưng các cú ngã vẫn chủ yếu phải dàn dựng vì lý do đạo đức và an toàn [57].

Định hướng áp dụng trong đề án. Từ các kết quả đã phân tích, kiến trúc pose-LSTM phù hợp với nguyên mẫu robot chăm sóc trên cả phương diện chức năng lẫn khả năng triển khai. Camera và bộ xử lý sẵn có trên smartphone cho phép quan sát không tiếp xúc mà không yêu cầu người cao tuổi mang thêm cảm biến. Hộp người và keypoint có thể được dùng chung cho chức năng bám mục tiêu, trong khi chuỗi pose giúp phân biệt quá trình té ngã với một tư thế nằm chủ động. Xử lý cục bộ còn hạn chế việc truyền liên tục video lên đám mây và chỉ phát sinh trao đổi mạng khi cần đồng bộ sự kiện hoặc cảnh báo.

Trong hiện thực của đề án, keypoint thuộc người đang được theo dõi được chuẩn hóa và ghép thành cửa sổ 15 khung hình gồm 69 đặc trưng trước khi đưa vào LSTM. Đầu ra nhị phân chỉ được xem là tín hiệu thô: hệ thống cần năm khung hình FALL liên tiếp mới mở một sự cố và chỉ hậu kiểm khi chuỗi trở lại NO FALL. Ba khung hình đầu/vai ổn định ở phía trên mốc 70% chiều cao ảnh làm loại ứng viên; ba khung hình đầu/vai nằm dưới mốc này, hoặc 12 khung hình liên tiếp không còn đủ quan sát thân trên, dẫn tới yêu cầu xác nhận bằng giọng nói. Trong lúc xử lý sự cố, robot chuyển sang chế độ tạo khung an toàn: dừng ngay, chỉ được lùi chậm hoặc quay tại chỗ để đưa người nằm vào trọn khung hình, tuyệt đối không tiến gần.

Kiến trúc nhiều tầng cũng làm xuất hiện chuỗi lan truyền sai số: detector bỏ sót có thể khiến tracker mất định danh; chuyển đổi định danh làm gián đoạn hoặc trộn chuỗi pose; keypoint thiếu chính xác tiếp tục ảnh hưởng tới dự đoán của LSTM. Cửa sổ 15 khung hình đồng thời tạo ra một khoảng khởi động và độ trễ không thể loại bỏ hoàn toàn. Vì vậy, hệ thống duy trì ngưỡng tin cậy cho keypoint, không đồng nhất việc mất quan sát với trạng thái an toàn, sử dụng khoảng khóa để tránh lặp cảnh báo và xác minh kết quả ở cấp sự kiện. Trong phạm vi đề án, YOLO-LSTM được lựa chọn như một phương án cân bằng giữa lượng thông tin thu được, chi phí phần cứng, khả năng xử lý tại biên và sự tích hợp với các chức năng hiện có của robot.

1.7.2 Các giải pháp theo dõi người và bám mục tiêu

Phát hiện người trả lời câu hỏi “có người ở đâu trong khung hình hiện tại”, còn tracking phải trả lời “người này có phải người ở khung hình trước hay không”. Đây là mất xích bắt buộc đối với robot bám người: nếu ID thay đổi khi hai người giao nhau hoặc

khi mục tiêu bị che khuất, robot có thể đi theo nhằm người; đồng thời chuỗi pose dùng cho phát hiện té ngã sẽ bị trộn giữa nhiều đối tượng.

SORT. SORT là kiến trúc tracking-by-detection gọn nhẹ. Ở mỗi khung hình, detector cung cấp các hộp người; bộ lọc Kalman dự đoán trạng thái hộp của từng track; ma trận chi phí được lập từ mức chồng lấp IoU giữa dự đoán và quan sát; thuật toán Hungarian tìm phép gán có tổng chi phí nhỏ; cuối cùng hệ thống cập nhật track khớp, tạo track mới và loại track mất quá lâu [39][44][45]. Luồng này không cần huấn luyện riêng cho tracker và phù hợp suy luận thời gian thực. Điểm yếu là chỉ dựa nhiều vào chuyển động và hình học hộp, nên dễ đổi ID khi che khuất lâu, chuyển động camera nhanh hoặc các đối tượng đứng gần nhau.

Deep SORT. Deep SORT bổ sung vector đặc trưng ngoại hình được học sâu và kết hợp khoảng cách ngoại hình với ràng buộc chuyển động. Khi mục tiêu xuất hiện trở lại sau che khuất ngắn, đặc trưng này giúp nối phát hiện mới với track cũ và giảm số lần đổi ID [40]. Đổi lại, mỗi hộp người cần thêm một lượt suy luận mạng Re-ID, làm tăng thời gian xử lý, bộ nhớ và độ phức tạp triển khai trên Android. Hiệu quả cũng giảm khi người dùng mặc trang phục gần giống nhau, ánh sáng thay đổi mạnh hoặc vùng cắt quá nhỏ.

ByteTrack. Nhiều tracker bỏ ngay các hộp có độ tin cậy thấp, trong khi người bị che một phần thường chính là trường hợp detector cho điểm thấp. ByteTrack tách phát hiện thành nhóm điểm cao và điểm thấp: ghép track với nhóm điểm cao trước, sau đó dùng các hộp điểm thấp còn lại để khôi phục đối tượng thật bị che khuất; các hộp không phù hợp bị xem là nền [41]. Ý tưởng “không vứt bỏ quá sớm bằng chứng yếu” hữu ích cho robot di động, nhưng vẫn đòi hỏi điều chỉnh ngưỡng và quản lý vòng đời track để tránh nối nhầm.

OC-SORT và các tracker hợp nhất. OC-SORT phân tích lại hạn chế của giả thiết vận tốc tuyến tính trong SORT. Khi một người bị che khuất, Kalman không nhận quan sát mới và sai số dự đoán tích lũy; nếu người đó đổi hướng, track dễ bị nối sang đối tượng khác. OC-SORT dùng các quan sát detector để dựng quỹ đạo ảo trong đoạn che khuất và sửa trạng thái chuyển động khi mục tiêu xuất hiện trở lại, nhờ đó vẫn giữ thiết kế online, thời gian thực nhưng bền vững hơn với chuyển động phi tuyến [42]. Đây là gợi ý phù

hợp cho phiên bản sau của đồ án vì robot và người mục tiêu đều có thể thay đổi hướng, làm chuyển động tương đối trong ảnh không còn tuyến tính.

Một nhánh khác là học chung detection và Re-ID trong một mạng. FairMOT dùng biểu diễn anchor-free và cân bằng hai nhiệm vụ phát hiện–tái định danh, tránh việc nhánh detection lấn át nhánh embedding [43]. Kiến trúc hợp nhất có thể tiết kiệm việc chạy hai backbone riêng, nhưng yêu cầu huấn luyện dữ liệu có nhãn identity và quá trình chuyển đổi mô hình phức tạp hơn. Với nguyên mẫu chỉ ưu tiên một người mục tiêu trên smartphone, tracker mô-đun vẫn dễ kiểm tra và thay thế hơn.

Việc chọn tracker không nên chỉ dựa trên FPS. MOTA phản ánh tổng hợp false positive, false negative và ID switch; IDF1 đo mức nhất quán nhận dạng; HOTA cân bằng chất lượng detection với association. Đối với robot bám người, số lần đổi ID, thời gian phục hồi sau che khuất và xác suất bám nhầm người còn quan trọng hơn MOTA tổng quát. Thử nghiệm cũng phải có chuyển động camera, vì benchmark camera cố định không biểu diễn đầy đủ việc thân robot quay hoặc rung.

Từ tracking đến lệnh bám người. Sau khi có track mục tiêu, bộ điều khiển thường dùng sai lệch ngang giữa tâm hộp và tâm ảnh để tạo lệnh quay, đồng thời dùng diện tích/chiều cao hộp làm đại lượng gần đúng cho khoảng cách để tạo lệnh tiến–lùi. Có thể mô tả luồng là *camera* → *detector* → *tracker* → *chọn mục tiêu* → *sai lệch vị trí/khoảng cách* → *vận tốc trái–phải* → *BLE* → *firmware*. OpenBot chứng minh cách tận dụng camera, cảm biến và năng lực tính toán của smartphone để điều khiển một thân xe chi phí thấp [10]. Đồ án kế thừa tư tưởng phân chia smartphone–thân robot, nhưng thay bài toán lái xe tham chiếu bằng bám người và giám sát chăm sóc.

Trong hiện thực của đề tài, tracker theo hướng SORT được tăng cường bằng khoảng cách pose và đặc trưng màu HSV nhẹ để hỗ trợ liên kết lại; mục tiêu chính được bảo vệ bằng ID0 và có cơ chế phục hồi sau khi mất dấu. Thiết kế này nằm giữa SORT thuần hình học và Deep SORT dùng mạng Re-ID: chi phí tính toán thấp hơn nhưng vẫn khai thác thêm bằng chứng đáng người/màu sắc. Quan trọng hơn, độ tin cậy của track được truyền tới điều khiển: khi mất mục tiêu hoặc quan sát không đủ tin cậy, hành vi đúng là giảm/dừng chuyển động thay vì tiếp tục dùng một lệnh cũ.

Bảng 1.3: So sánh các hướng tracking-by-detection liên quan đến đề tài

Giải pháp	Bảng chứng ghép track	Ưu điểm và giới hạn	Khả năng áp dụng
SORT	Kalman, IoU, Hungarian	Gọn, nhanh, không cần mạng Re-ID; dễ đổi ID khi che khuất hoặc giao nhau	Phù hợp làm lõi tracker trên smartphone
Deep SORT	Chuyển động và embedding ngoại hình	Giảm đổi ID; cần suy luận đặc trưng và tài nguyên lớn hơn	Hữu ích nếu thiết bị đủ mạnh và nhiều người xuất hiện
ByteTrack	Ghép hai lượt với hộp điểm cao và thấp	Khôi phục quan sát bị che khuất; nhạy với ngưỡng và chất lượng detector	Gợi ý giữ bằng chứng yếu có kiểm soát
OC-SORT	Quan sát, Kalman và quỹ đạo ảo qua che khuất	Bền vững hơn với chuyển động phi tuyến; vẫn phụ thuộc detector	Hướng nâng cấp cho tình huống người và robot cùng đổi hướng
FairMOT	Học chung detection và embedding Re-ID	Một backbone nhưng cần dữ liệu identity và huấn luyện riêng	Chưa cần thiết cho nguyên mẫu một mục tiêu, phù hợp mở rộng đa người
Đề tài	Hình học, pose, HSV và chính sách ID0	Nhẹ hơn Re-ID sâu; vẫn phụ thuộc góc nhìn, ánh sáng và trang phục	Cân bằng giữa tốc độ, ổn định ID và khả năng chạy Android

1.7.3 Chatbot AI và hội thoại hỗ trợ người cao tuổi

Trong đồ án, chatbot được định hướng như một người bạn trò chuyện bằng giọng nói trên robot. Người dùng có thể gọi robot, đặt câu hỏi, kể chuyện hoặc trao đổi về những chủ đề hằng ngày mà không phải thao tác với bàn phím và màn hình nhỏ. Mục tiêu chính là tạo một kênh giao tiếp tự nhiên, phản hồi ngắn gọn và dễ sử dụng; chatbot không được xem là công cụ chẩn đoán, trị liệu tâm lý hay tư vấn y khoa.

Kiến trúc của một chatbot giọng nói. Một lượt hội thoại hoàn chỉnh thường đi qua chuỗi kích hoạt → thu âm → nhận dạng tiếng nói (STT) → mô hình ngôn ngữ lớn (LLM) → tổng hợp tiếng nói (TTS) → phát âm thanh. Bên cạnh ba khối xử lý chính, ứng dụng còn cần phát hiện khoảng lặng, quản lý lịch sử hội thoại, ngắt câu trả lời đang phát, xử lý mất mạng và biểu diễn trạng thái để người dùng biết robot đang nghe, suy nghĩ hay nói. Chất lượng trải nghiệm vì thế phụ thuộc vào độ trễ đầu cuối và cách phối hợp các thành

phần, không chỉ vào chất lượng văn bản do LLM sinh ra.

Xiaozhi-ESP32 là một hiện thực tham khảo tiêu biểu cho cách tổ chức này trên thiết bị có tài nguyên hạn chế. Nền tảng thực hiện đánh thức bằng từ khóa ngay trên thiết bị, sau đó truyền âm thanh nén Opus qua WebSocket hoặc MQTT/UDP để sử dụng chuỗi ASR–LLM–TTS; giao thức còn quy định các trạng thái bắt đầu nghe, bắt đầu nói, kết thúc nói và hủy lượt hội thoại [82]. Điểm phù hợp với đồ án không nằm ở phần cứng ESP32 mà ở nguyên tắc kiến trúc: thiết bị đầu cuối chỉ duy trì vòng đời phiên, thu/phát âm thanh và giao diện trạng thái, còn tác vụ ngôn ngữ nặng có thể được chuyển tới dịch vụ bên ngoài. Cách phân vai này cho phép một thiết bị nhỏ tạo cảm giác hội thoại liên tục mà không phải chứa toàn bộ mô hình AI.

Trên Android, cùng một luồng có thể được hiện thực bằng các API nền tảng. `SpeechRecognizer` cung cấp kết quả tạm thời và kết quả cuối, nhờ đó giao diện có thể hiển thị lời nói theo thời gian thực và gửi câu hoàn chỉnh tới LLM. Tuy nhiên, tài liệu Android lưu ý rằng dịch vụ nhận dạng mặc định có thể truyền âm thanh tới máy chủ và không được thiết kế cho việc nghe liên tục vì tiêu thụ pin và băng thông; đối tượng nhận dạng cũng phải được giải phóng bằng `destroy()` khi không còn sử dụng [80]. Từ Android 12, ứng dụng có thể kiểm tra khả năng nhận dạng cục bộ và tạo `createOnDeviceSpeechRecognizer()` trên thiết bị hỗ trợ. Vì mức hỗ trợ phụ thuộc nhà sản xuất, phiên bản hệ điều hành và gói ngôn ngữ, ứng dụng vẫn cần cơ chế kiểm tra khả dụng và phương án quay về dịch vụ mặc định.

Ở chiều đầu ra, `TextToSpeech` của Android có thể đọc văn bản ngay lập tức hoặc tổng hợp thành tệp âm thanh. Ứng dụng phải chờ quá trình khởi tạo hoàn tất, kiểm tra ngôn ngữ/giọng đọc và gọi `shutdown()` khi kết thúc [81]. API cho phép phân biệt giọng nhúng với giọng yêu cầu kết nối mạng thông qua thuộc tính của `Voice`; vì vậy khả năng hoạt động ngoại tuyến không đồng nhất trên mọi điện thoại. Dù chất lượng giọng phụ thuộc engine và gói tiếng Việt được cài đặt, Android TTS vẫn là lựa chọn nhẹ nhất cho nguyên mẫu vì không phải tích hợp mô hình tổng hợp giọng nói hay trả phí theo số ký tự.

Các lựa chọn cho khối STT và TTS. Giải pháp sử dụng dịch vụ hệ thống Android có ưu thế về dung lượng ứng dụng, công sức tích hợp và khả năng tận dụng engine đã được tối ưu theo thiết bị. Hạn chế là hành vi giữa các phiên bản Android và nhà sản xuất không hoàn toàn đồng nhất; nhận dạng mặc định có thể cần mạng, trong khi giọng tiếng

Việt ngoại tuyến có thể chưa được cài sẵn. Đây là lựa chọn phù hợp khi mục tiêu là xây dựng nhanh một ứng dụng nhẹ và miễn phí ở mức API nền tảng.

Nếu yêu cầu bắt buộc hoạt động không mạng, STT có thể được đóng gói trực tiếp bằng Vosk. Dự án cung cấp thư viện Android, hỗ trợ nhận dạng luồng, tiếng Việt và các mô hình nhỏ dành cho smartphone hoặc Raspberry Pi; mô hình nhỏ thường có dung lượng khoảng 50 MB và cần khoảng 300 MB bộ nhớ khi chạy [86]. Một lựa chọn rộng hơn là sherpa-onnx, cung cấp các APK và thư viện chạy cục bộ cho nhận dạng luồng, VAD, phát hiện từ khóa và TTS trên Android [87]. Hai hướng này loại bỏ phí API và giữ âm thanh tại thiết bị, nhưng làm tăng dung lượng cài đặt, mức dùng CPU/RAM và công việc lựa chọn mô hình tiếng Việt. Độ chính xác trong môi trường có tiếng động cơ robot cũng cần được đo trên thiết bị thật thay vì suy ra từ bộ dữ liệu đọc tiếng nói sạch.

Đối với TTS, engine Android nên được ưu tiên trước; thư viện ngoại tuyến chỉ cần thiết khi giọng hệ thống không hỗ trợ tiếng Việt hoặc chất lượng không đáp ứng yêu cầu. Mô hình TTS đóng gói mang lại hành vi nhất quán và không phụ thuộc dịch vụ của nhà sản xuất, song có thể làm APK lớn hơn đáng kể và tạo thêm tải khi RobotApp đồng thời chạy camera, detector, tracking và LSTM. Như vậy, cấu hình nhẹ hợp lý là STT/TTS của Android với kiểm tra khả năng ngoại tuyến; Vosk hoặc sherpa-onnx là phương án thay thế khi tính độc lập mạng quan trọng hơn dung lượng và tài nguyên.

LLM qua API và LLM chạy trên thiết bị. Với phương án API, Android chỉ gửi văn bản cùng một phần lịch sử hội thoại và nhận lại các đoạn phản hồi. Ưu điểm nổi bật là ứng dụng nhẹ, có thể sử dụng mô hình lớn hơn khả năng phần cứng của điện thoại và dễ thay đổi mô hình mà không phát hành lại APK. Kết quả dạng streaming có thể được tách theo câu và chuyển sớm sang TTS, nhờ đó người dùng nghe phần đầu câu trả lời trước khi toàn bộ nội dung được tạo xong. Đổi lại, hội thoại phụ thuộc đường truyền, hạn mức nhà cung cấp và thời gian phản hồi máy chủ; nội dung gửi đi cũng phải được giới hạn để tránh lộ thông tin cá nhân không cần thiết.

Groq cung cấp endpoint Chat Completions tương thích với cấu trúc message và hỗ trợ phản hồi streaming [77]. Gói miễn phí phù hợp cho phát triển nguyên mẫu nhưng áp dụng giới hạn theo số yêu cầu và token; khi vượt giới hạn, dịch vụ trả mã HTTP 429 [78]. Gemini Developer API cũng có tầng miễn phí cho một số mô hình và hạn mức nhất định, trong khi tầng trả phí cung cấp quota cao hơn [79]. Vì mô hình, quota và chính sách giá có thể thay đổi, “miễn phí” trong trường hợp này chỉ nên được hiểu là không phát

sinh chi phí trong phạm vi thử nghiệm, không phải cam kết vận hành miễn phí lâu dài. Ngoài ra, khóa truy cập không nên được xem là bí mật nếu đóng trực tiếp trong APK; một sản phẩm triển khai thực tế cần proxy phía máy chủ, xác thực người dùng và giới hạn quota để bảo vệ khóa.

Chạy LLM trực tiếp trên điện thoại loại bỏ phí theo lượt gọi, vẫn hoạt động khi mất mạng và giữ nội dung hội thoại cục bộ. Google AI Edge Gallery chứng minh khả năng chạy các mô hình mở bằng LiteRT trên Android và cung cấp mã nguồn tham khảo cho hội thoại, quản lý mô hình và benchmark [83]. Tuy nhiên, “chạy tại thiết bị” không đồng nghĩa với “nhẹ”: cấu hình tham khảo của dự án cho Gemma 3 1B lượng tử hóa 4-bit có tệp mô hình khoảng 555 MB và bộ nhớ đỉnh ước tính 2 GB; Qwen2.5 1.5B 8-bit có tệp khoảng 1,63 GB và bộ nhớ đỉnh khoảng 2,5 GB [84]. MLC LLM cũng hỗ trợ đóng gói model weights và sử dụng GPU Android, nhưng quy trình tích hợp cần NDK, runtime native, tokenizer và model library; ví dụ Gemma 2B lượng tử hóa trong tài liệu ước tính khoảng 3 GB VRAM [85].

Các mô hình nhỏ và lượng tử hóa có thể giảm dung lượng, song thường đánh đổi khả năng hiểu tiếng Việt, độ ổn định của hội thoại dài và tốc độ sinh token trên điện thoại tầm trung. Chúng cũng cạnh tranh RAM, CPU/GPU và nhiệt với pipeline thị giác đang chạy trên RobotApp. Vì vậy, LLM cục bộ phù hợp khi quyền riêng tư và khả năng ngoại tuyến là ưu tiên tuyệt đối, thiết bị có đủ bộ nhớ và chất lượng mô hình đã được kiểm chứng. Đối với nguyên mẫu cần hội thoại tiếng Việt tự nhiên trên nhiều điện thoại, sử dụng API vẫn là phương án nhẹ và thực tế hơn.

Bảng 1.4: So sánh các phương án hiện thực chatbot trên Android

Phương án	Ưu điểm	Hạn chế	Mức phù hợp
Luật/kho câu trả lời	Rất nhẹ, ngoại tuyến, kết quả xác định	Hội thoại cứng và khó bao phủ cách diễn đạt tự nhiên	Lệnh, xác nhận và trạng thái an toàn
LLM qua API	APK nhẹ, phản hồi tốt, hỗ trợ streaming, dễ đổi mô hình	Cần mạng, phụ thuộc quota; phải bảo vệ khóa và dữ liệu	Hội thoại hằng ngày trong nguyên mẫu
LLM cục bộ	Ngoại tuyến, dữ liệu không rời thiết bị, không tính phí theo lượt	Model lớn, dùng nhiều RAM và năng lượng; phụ thuộc phần cứng	Điện thoại mạnh hoặc yêu cầu riêng tư cao
Kiến trúc lai	Cân bằng tính tự nhiên với khả năng kiểm soát	Cần bộ định tuyến ý định và quản lý trạng thái rõ ràng	Phù hợp nhất với robot chăm sóc

Tổ chức hội thoại phù hợp với người cao tuổi. Một chatbot trò chuyện hằng ngày không cần câu trả lời dài hoặc lập luận phức tạp. Giá trị sử dụng đến từ việc kích hoạt dễ nhớ, nghe được nhiều cách diễn đạt, trả lời đúng ngôn ngữ, duy trì một lượng ngữ cảnh vừa đủ và phản hồi sớm. Giao diện nên thể hiện rõ các trạng thái đang nghe, đang xử lý và đang nói; cho phép lặp lại khi STT nhận sai; tránh đọc ký hiệu, đường dẫn hoặc đoạn văn quá dài. Hồ sơ như tên gọi, cách xưng hô và sở thích có thể giúp cuộc trò chuyện tự nhiên hơn, nhưng chỉ nên đưa vào prompt những trường cần thiết cho lượt hội thoại hiện tại.

Kiến trúc lai tách hội thoại mở khỏi tác vụ có tính quyết định. Những câu hỏi thông thường được chuyển tới LLM để sinh phản hồi linh hoạt. Ngược lại, lệnh gọi điện, xác nhận lịch nhắc và phản hồi sau cảnh báo té ngã đi qua bộ nhận diện ý định, từ khóa, timeout và máy trạng thái của ứng dụng. Cách tổ chức này tương đồng với kết luận của tổng quan về conversational agent: khả năng tiếp nhận ngôn ngữ tự nhiên không đồng nghĩa với việc mọi quyết định đều phải do mô hình sinh đảm nhiệm [88]. Nó cũng giới hạn ảnh hưởng của hiện tượng LLM tạo thông tin không chính xác tới các chức năng liên quan an toàn.

Giải pháp được hiện thực trong RobotApp. RobotApp sử dụng SpeechRecognizer với ngôn ngữ vi-VN, nhận cả kết quả tạm thời và kết quả cuối. Khi ở trạng thái ngủ, kết quả nhận dạng được kiểm tra để phát hiện tên gọi của robot; sau khi đánh thức, câu hoàn chỉnh được chuyển tới dịch vụ LLM. Ứng dụng quản lý các trạng thái khởi tạo, ngủ, đánh thức, lắng nghe, suy nghĩ, phát lời, nhắc lịch, xác nhận té ngã và cuộc gọi. Việc biểu diễn rõ trạng thái tránh cho STT tiếp tục thu chính âm thanh do TTS phát ra và cho phép hủy hoặc khởi động lại một lượt khi xảy ra lỗi.

Phần sinh câu trả lời được hiện thực bằng Groq Chat Completions qua OkHttp. System prompt được tạo từ hồ sơ người dùng, lịch sử chỉ giữ một số message gần nhất để giới hạn token, còn phản hồi SSE được nhận dưới dạng luồng. Các đoạn văn bản được tích lũy đến ranh giới câu rồi đưa vào hàng đợi Android TTS; nhờ đó robot có thể bắt đầu nói trước khi LLM hoàn tất toàn bộ câu trả lời. Android TTS được chọn thay cho một dịch vụ giọng nói bên ngoài để giảm phụ thuộc, không phát sinh phí tổng hợp theo ký tự và tận dụng giọng tiếng Việt đã cài trên điện thoại.

Cấu hình này đáp ứng mục tiêu nhẹ và chi phí thấp của nguyên mẫu: mô hình ngôn ngữ không chiếm RAM hay dung lượng APK, STT/TTS tận dụng hệ sinh thái Android,

còn API miễn phí được sử dụng trong giới hạn thử nghiệm. Hạn chế còn lại là phụ thuộc mạng, quota, chất lượng engine giọng nói của từng thiết bị và rủi ro bảo mật khi gọi API trực tiếp từ ứng dụng. Hướng phát triển phù hợp là chuyển khóa API sang một proxy được xác thực, ưu tiên on-device SpeechRecognizer khi thiết bị hỗ trợ, bổ sung VAD hoặc wake-word engine cục bộ và đánh giá một LLM lượng tử hóa nhỏ như chế độ dự phòng ngoại tuyến. Các thay đổi này có thể được thực hiện độc lập nhờ ba khối STT, LLM và TTS đã được tách thành các dịch vụ riêng.

1.7.4 Các nền tảng robot tích hợp và bài học áp dụng

SocialRobot là một ví dụ về kiến trúc tích hợp robot di động với chăm sóc xã hội ảo theo hướng lấy người dùng làm trung tâm. Nền tảng kết hợp khả năng di chuyển, cảm biến RGB-D/laser, định vị–lập bản đồ trên ROS với hạ tầng mạng quản lý hồ sơ và nhóm chăm sóc để cung cấp dịch vụ cá nhân hóa [93]. Giá trị tham khảo đối với đề tài nằm ở việc tách năng lực robot tại chỗ khỏi dịch vụ và dữ liệu chăm sóc, đồng thời xem người thân/người chăm sóc là một tác nhân của hệ thống, không chỉ là người nhận cảnh báo.

OpenBot cung cấp bài học bổ sung về tối giản phần cứng: smartphone đảm nhiệm camera, suy luận, giao diện và kết nối; vi điều khiển trên thân xe tập trung vào chấp hành [10][12]. Đây là ý tưởng nền tảng được đề án vận dụng mạnh. Tuy nhiên, nguyên mẫu của đề tài thay đổi mục tiêu ứng dụng: hai ứng dụng RobotApp–CaregiverApp được phân vai; Firebase đồng bộ lịch và cảnh báo; WebRTC cung cấp hiện diện từ xa; BLE nối quyết định mức ứng dụng với firmware ESP32; pipeline pose–tracking–LSTM cung cấp khả năng giám sát cục bộ. Như vậy, đóng góp không nằm ở một thuật toán hoàn toàn mới mà ở cách ghép các khối thành luồng chăm sóc có phản hồi và điều kiện an toàn.

1.7.5 Khoảng trống nghiên cứu và định vị của đề tài

Các nghiên cứu phát hiện té ngã thường dừng ở kết quả trên bộ dữ liệu; nghiên cứu tracking tập trung chỉ số MOT; chatbot tập trung chất lượng phản hồi; còn nền tảng robot giá thấp tập trung điều khiển và khả năng mở rộng. Tổng quan FDS cũng nhận diện nhu cầu tiếp tục hoàn thiện dữ liệu, khả năng phát hiện nhiều kiểu ngã, bảo mật và riêng tư khi hệ thống kết nối IoT/cloud [48]. Khi triển khai chung, lỗi của một khối lan sang khối sau: detector bỏ sót làm tracker mất ID, mất ID làm trộn chuỗi pose, dự đoán sai tạo cảnh báo, và diễn giải hội thoại sai có thể đóng sự cố không an toàn. Khoảng trống mà đề án xử lý là luồng liên thành phần từ quan sát đến hành động, có ứng dụng dành riêng cho người chăm sóc và có đường dừng an toàn tới phần cứng thật. Bảng 1.5 tổng

hợp sự khác biệt.

Bảng 1.5: Đối chiếu các nhóm giải pháp với phạm vi tích hợp của đề tài

Nhóm giải pháp	Ý tưởng/công nghệ chính	Giới hạn khi dùng độc lập	Cách đề tài kế thừa
Fall detector ảnh	YOLO, hộp người ngã, suy luận một khung hình	Thiếu diễn biến; dễ nhầm nằm/ngồi thấp	Pose, cửa sổ LSTM và hậu kiểm sự kiện
Fall detection theo pose	Keypoint, đặc trưng hình học, LSTM/Transformer	Nhảy với che khuất và chất lượng keypoint	Chuẩn hóa pose, tracking mục tiêu, xác nhận nhiều khung hình
Person tracking	SORT, Deep SORT, ByteTrack	Tối ưu ID nhưng không tự bảo đảm chuyển động robot an toàn	ID0, phục hồi bằng pose/HSV và dừng khi mất mục tiêu
Chatbot chăm sóc	NLU/NLG, tri thức, LLM, STT/TTS	Có thể hiểu sai hoặc sinh nội dung không kiểm chứng	LLM cho trò chuyện; luật trạng thái cho phản ứng té ngã
OpenBot và SocialRobot	Smartphone hoặc robot di động kết hợp dịch vụ	Không trùng đầy đủ bài toán và phần cứng của đề tài	Kế thừa phân lớp tính toán–chấp hành và mô hình nhóm chăm sóc
Nguyên mẫu đề tài	Hai ứng dụng Android, AI biên, Firebase, WebRTC, BLE và firmware	Chưa phải thiết bị y tế; cần đánh giá thực địa và bảo mật đa người dùng	Đóng góp tích hợp, truy vết sự kiện và hợp đồng an toàn xuyên suốt

1.8 Đóng góp của đề tài

- Kiến trúc nguyên mẫu gồm hai ứng dụng Android, Firebase, WebRTC, dịch vụ LLM, BLE và firmware điều khiển động cơ.
- Pipeline AI biên kết hợp pose, SORT/Kalman/Hungarian, Re-ID, người mục tiêu ID0, LSTM 15 khung hình với 69 đặc trưng và bộ hậu kiểm theo bằng chứng.
- Cơ chế chuyển quyền camera: CameraX sở hữu camera khi hoạt động bình thường; trong cuộc gọi, WebRTC sở hữu camera và chia sẻ khung hình cục bộ cho pipeline AI.
- Chuỗi an toàn chuyển động gồm giới hạn miền lệnh, timestamp chống lệnh cũ, watchdog ở Android và watchdog 800 ms trong firmware.
- Quy trình phản ứng sự cố: dừng robot, tạo đúng một cảnh báo, hỏi xác nhận bằng giọng nói, phân loại phản hồi và gọi người chăm sóc theo nguyên tắc mặc định an toàn.

- Tài liệu hóa các điểm còn hạn chế: cơ chế ghép cặp hiện dùng chung một UID, chưa phân vai trò/nhiều robot, phụ thuộc dịch vụ đám mây, quản lý khóa API và nhu cầu kiểm thử thực địa.

1.9 Cấu trúc báo cáo

Báo cáo gồm sáu chương. Chương 1 trình bày bối cảnh, bài toán, mục tiêu, phạm vi, phương pháp và nghiên cứu liên quan. Chương 2 hệ thống hóa cơ sở lý thuyết về robot smartphone, AI, theo dõi, phát hiện té ngã, LLM, giao tiếp và điều khiển. Chương 3 phân tích yêu cầu và thiết kế kiến trúc, dữ liệu, luồng xử lý và cơ chế an toàn. Chương 4 mô tả hiện thực hai ứng dụng, pipeline AI, chuyển đổi mô hình và firmware. Chương 5 trình bày phương pháp kiểm thử, kết quả, thảo luận giới hạn và mối đe dọa tới tính hợp lệ. Chương 6 tổng kết đóng góp và đề xuất hướng phát triển. Phụ lục cung cấp đặc tả use case, hợp đồng dữ liệu, quy trình build và danh mục kiểm thử.

1.10 Kết luận chương

Chương này đã xác lập tính cấp thiết từ xu hướng già hóa, rủi ro té ngã, nhu cầu nhắc việc và liên lạc tại nhà; đồng thời định vị nguyên mẫu trong bối cảnh robot hỗ trợ và phát hiện té ngã hiện có. Bài toán của đề tài không chỉ là huấn luyện một mô hình AI mà là bảo đảm nhiều thành phần hoạt động như một hệ thống có hành vi dự đoán được. Các kiến thức nền cho những quyết định đó được trình bày ở chương tiếp theo.

Chương 2 – Cơ sở lý thuyết

Phần này trình bày nền tảng lý thuyết cần thiết cho hệ thống robot thông minh tích hợp trí tuệ nhân tạo phục vụ giám sát và phát hiện té ngã. Nội dung được tổ chức theo thứ tự: tìm hiểu nền tảng Intel OpenBot; hệ thống nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên; công nghệ Internet vạn vật (IoT) cho kết nối và điều khiển; các kiến trúc mạng nơ-ron nhân tạo (MLP, CNN); mô hình YOLO và lý do lựa chọn YOLOv11; cùng phương pháp xây dựng tập dữ liệu đa nhiệm phục vụ huấn luyện mô hình phát hiện ngã.

2.1 Nền tảng Intel OpenBot (OpenBot)

2.1.1 Giới thiệu và ý tưởng cốt lõi

OpenBot là nền tảng robot mã nguồn mở khởi nguồn từ công trình của Matthias Müller và Vladlen Koltun, được công bố tại IEEE ICRA 2021. Ý tưởng chính của OpenBot là sử dụng điện thoại thông minh làm “bộ não” cho một thân xe robot chi phí thấp. Thay vì trang bị riêng camera, máy tính nhúng mạnh, màn hình và mô-đun mạng, nền tảng tận dụng camera, cảm biến, CPU/GPU, pin và khả năng kết nối đã có trên smartphone [10].

Trong kiến trúc này, điện thoại được gắn trực tiếp lên robot và tham gia vào vòng xử lý: thu hình ảnh, chạy mô hình AI, quyết định hướng di chuyển và gửi lệnh xuống vi điều khiển. Thân xe đảm nhiệm cấp nguồn, điều khiển động cơ và đọc các cảm biến thấp tầng. Công trình gốc chứng minh cách tiếp cận này có thể thực hiện các tác vụ như bám theo người và điều hướng tự động thời gian thực trên điện thoại Android phổ thông [10]. Website và kho mã chính thức tiếp tục phát triển OpenBot theo hướng một nền tảng mở phục vụ nghiên cứu, giáo dục robot và AI [11][12].

Ý tưởng này có liên hệ trực tiếp với đồ án. RobotApp cũng sử dụng smartphone để thu hình ảnh, chạy mô hình AI, giao tiếp mạng và điều phối hoạt động; ESP32 tập trung vào giao tiếp BLE và điều khiển động cơ. Nhờ đó, các thuật toán thị giác có thể thay đổi mà không phải thiết kế lại toàn bộ phần thân robot.

2.1.2 Kiến trúc và các thành phần liên quan

OpenBot có thể được khái quát thành ba lớp chính:

- **Lớp smartphone:** Robot App truy cập camera và cảm biến, chạy mô hình Tensor-

Flow Lite, quản lý chế độ điều khiển và truyền lệnh cho thân xe.

- **Lớp vi điều khiển và thân robot:** Arduino Nano hoặc ESP32 nhận lệnh, tạo PWM cho driver động cơ, đọc encoder, sonar, bumper và điện áp pin rồi gửi trạng thái trở lại ứng dụng [15].
- **Lớp điều khiển bên ngoài:** gamepad, điện thoại thứ hai, trình duyệt hoặc chương trình Python có thể gửi lệnh và nhận hình ảnh từ robot [14].

Kho mã OpenBot cũng phản ánh cách phân chia trên: thư mục `android` chứa Robot App và Controller App; `firmware` chứa chương trình cho MCU; `body` chứa thiết kế thân xe; `controller` chứa các phương thức điều khiển; còn `policy` hỗ trợ thu thập dữ liệu và huấn luyện chính sách lái [12]. Cấu trúc mô-đun này là một tham khảo hữu ích cho việc tách RobotApp, CaregiverApp, firmware và phần AI trong đồ án.

Điểm quan trọng của kiến trúc OpenBot là phân công theo năng lực của thiết bị. Smartphone phù hợp với camera, AI, giao diện và kết nối Internet; MCU phù hợp với vòng điều khiển động cơ có chu kỳ ngắn. Việc không đặt toàn bộ xử lý trên MCU giúp triển khai được các mô hình thị giác phức tạp, trong khi vẫn giữ lớp chấp hành độc lập và có khả năng dừng khi mất lệnh.

2.1.3 Giải pháp kết nối và điều khiển

OpenBot hỗ trợ USB serial và BLE giữa smartphone với vi điều khiển. Với USB, Robot App thường sử dụng baud rate 115200 để gửi lệnh và nhận telemetry. Với các cấu hình ESP32, BLE cho phép bố trí điện thoại không dây và thuận tiện hơn, nhưng ứng dụng phải xử lý quét thiết bị, kết nối lại và mất liên lạc [13][15].

Firmware OpenBot sử dụng giao thức văn bản ngắn gọn. Lệnh điều khiển chứa giá trị cho phía trái và phải; các lệnh khác cấu hình đèn, chu kỳ đo sonar, tốc độ bánh hoặc điện áp. Firmware còn hỗ trợ heartbeat: nếu không nhận được gói mới trong thời gian quy định, robot phải dừng. Đây là giải pháp quan trọng vì smartphone và kết nối không dây không bảo đảm hoạt động liên tục [15].

Ở mức điều khiển từ xa, OpenBot cung cấp nhiều lựa chọn: gamepad Bluetooth, Controller App trên điện thoại thứ hai, Node.js controller qua Wi-Fi, Python controller với RTSP và web controller qua Internet. Robot App có thể truyền hình ảnh bằng WebRTC hoặc RTSP tùy client [14].

Trong đồ án, ý tưởng kết nối được điều chỉnh theo mục tiêu chăm sóc người cao

tuổi:

- RobotApp kết nối ESP32 bằng BLE GATT thay cho giao thức serial nguyên bản của OpenBot;
- CaregiverApp gửi lệnh từ xa qua Firebase RTDB, do người chăm sóc có thể không ở cùng mạng với robot;
- WebRTC được dùng cho cuộc gọi audio/video giữa caregiver và robot;
- lệnh có timestamp, RobotApp có watchdog 500 ms và firmware có watchdog 800 ms để robot dừng khi mất luồng điều khiển.

Như vậy, đồ án kế thừa nguyên tắc phân tầng và dừng an toàn của OpenBot, nhưng xây dựng lại kênh truyền để phù hợp mô hình điều khiển từ xa qua Internet.

2.1.4 AI và cơ chế theo dõi đối tượng

Robot App của OpenBot hỗ trợ suy luận TensorFlow Lite trực tiếp trên điện thoại. Người dùng có thể lựa chọn model, ngưỡng confidence, số luồng và backend CPU, GPU hoặc NNAPI. Tài liệu OpenBot cung cấp các detector như MobileNet SSD, YOLO và EfficientDet, đồng thời benchmark chúng trên nhiều mẫu smartphone. Cách thiết kế này cho thấy model triển khai trên robot phải cân bằng độ chính xác, tốc độ, nhiệt độ và khả năng tương thích của thiết bị [13].

Trong chức năng Object Tracking, detector nhận dạng các đối tượng thuộc tập lớp COCO và trả về vùng bao. Sai lệch giữa tâm vùng bao với tâm ảnh có thể dùng để điều chỉnh hướng quay; kích thước vùng bao được dùng như một dấu hiệu gần đúng về khoảng cách để giảm tốc khi robot tiến gần mục tiêu. Luồng xử lý cơ bản gồm:

Camera → phát hiện đối tượng → chọn mục tiêu → tính sai lệch vị trí → lệnh bánh xe.

Đối với bám theo người, OpenBot cung cấp ý tưởng nền tảng là đặt nhận thức và điều khiển trong cùng một vòng kín trên smartphone. Tuy nhiên, chỉ chọn detection có nhãn “person” ở từng frame có thể làm đổi mục tiêu khi nhiều người xuất hiện hoặc khi người đang bám bị che khuất. Vì vậy, đồ án phát triển thêm cơ chế tracking theo thời gian.

Pipeline của đồ án sử dụng mô hình pose để phát hiện người và keypoint, SORT/Kalman/Hungarian để liên kết detection giữa các frame, cùng Re-ID nhẹ để hỗ

trợ duy trì người mục tiêu. Người được chọn được bảo vệ bằng ID0; vị trí và kích thước vùng bao được chuyển thành lệnh bám. Chuỗi pose cũng được đưa vào LSTM và bộ hậu kiểm để phát hiện té ngã. Đây là phần mở rộng từ ý tưởng object/person tracking của OpenBot sang yêu cầu nhận diện đúng người và giám sát an toàn.

2.1.5 Khả năng áp dụng cho đồ án

Bảng 2.1 tổng hợp những nội dung OpenBot có liên quan trực tiếp đến hệ thống hiện tại.

Bảng 2.1: Các ý tưởng OpenBot được tham khảo trong đồ án

Nội dung	OpenBot	Áp dụng trong đồ án
Smartphone làm bộ não	Camera, AI, giao diện và logic điều phối đặt trên điện thoại	RobotApp chạy pose, tracking, LSTM, hội thoại và cuộc gọi.
Tách xử lý và chấp hành	Smartphone xử lý bậc cao; MCU điều khiển PWM	RobotApp sinh lệnh; ESP32 điều khiển hai BLDC bằng SimpleFOC.
Kết nối thân robot	USB serial hoặc BLE	BLE GATT với giao thức hai set-point và watchdog.
Điều khiển từ thiết bị khác	Gamepad, Controller App, web/Python controller	CaregiverApp gửi lệnh qua Firebase RTDB và nhận trạng thái từ xa.
AI tracking	Detector TFLite, vùng bao và luật bám mục tiêu	Pose, SORT, Kalman, Hungarian, Re-ID và bảo vệ ID0.
Truyền hình ảnh	WebRTC hoặc RTSP tùy bộ điều khiển	WebRTC cho cuộc gọi audio/video caregiver-robot.

OpenBot vì vậy là nền tảng tham khảo quan trọng về ý tưởng và các giải pháp kỹ thuật nền, đặc biệt là smartphone-as-a-robot-brain, xử lý AI tại biên, kết nối smartphone-MCU và vòng kín theo dõi mục tiêu. Đồ án kế thừa các nguyên tắc này nhưng bổ sung kiến trúc hai ứng dụng, nghiệp vụ chăm sóc, phát hiện té ngã, tương tác giọng nói, Firebase và phần thân BLDC/FOC riêng.

2.2 Nền tảng Thị giác máy tính (Computer Vision)

Thị giác máy tính cung cấp dữ liệu cảm nhận cho hai chức năng chính của robot: giám sát tư thế nhằm phát hiện té ngã và xác định vị trí người để tự động bám theo. Một pipeline điển hình gồm thu nhận khung hình, trích xuất đặc trưng bằng mạng nơ-ron tích chập (Convolutional Neural Network – CNN), phát hiện người và các điểm khớp, liên kết kết quả qua thời gian, sau đó chuyển chuỗi đặc trưng sang bộ phân loại hành vi. CNN học trực tiếp các bộ lọc không gian từ dữ liệu, nhờ đó thay thế phần lớn quá trình thiết

kế đặc trưng thủ công của các phương pháp thị giác cổ điển [35]. Các khái niệm từ phát hiện đối tượng đến đánh giá mô hình dưới đây là nền tảng chung để phân tích MobileNet SSD, YOLO và EfficientDet, không phụ thuộc vào một họ kiến trúc cụ thể.

2.2.1 Bài toán Object Detection

Phát hiện đối tượng (Object Detection) trả lời đồng thời hai câu hỏi: trong ảnh có những đối tượng thuộc lớp nào và chúng nằm ở đâu. Vì vậy, khác với phân loại ảnh chỉ sinh một hoặc một số nhãn cho toàn ảnh, đầu ra của bộ phát hiện là một tập

$$\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^M, \quad (2.1)$$

trong đó b_i là hộp giới hạn, c_i là nhãn lớp và $s_i \in [0, 1]$ là độ tin cậy của dự đoán thứ i . Số lượng đối tượng M thay đổi theo từng khung hình. Tổng quan của Zou và cộng sự phân Object Detection thành hai nhóm lớn: detector hai giai đoạn, trước hết sinh vùng ứng viên rồi phân loại/tinh chỉnh vùng; và detector một giai đoạn, dự đoán trực tiếp lớp cùng vị trí trên bản đồ đặc trưng [24].

Trong đồ án, đầu ra cần thiết không chỉ là hộp bao quanh người. Bộ ước lượng tư thế còn trả về các điểm khớp của cơ thể; hộp người hỗ trợ theo dõi và điều khiển bám mục tiêu, trong khi tọa độ khớp tạo chuỗi đặc trưng cho mô hình LSTM. Do đó, phát hiện một người trong từng khung hình mới là tầng cảm nhận ban đầu, chưa đồng nghĩa với kết luận người đó đã té ngã.

2.2.2 Lịch sử phát triển Object Detection

Quá trình phát triển của Object Detection có thể khái quát qua ba giai đoạn [24]:

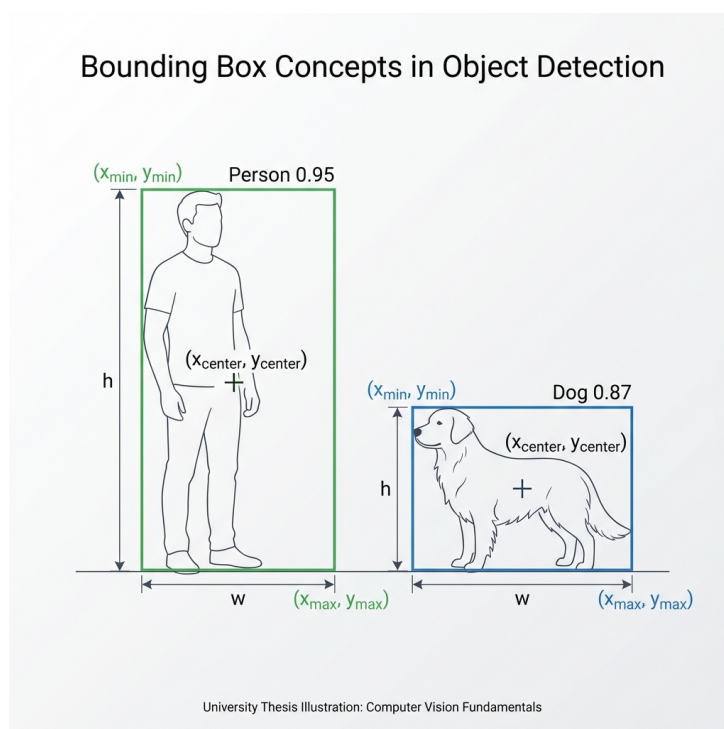
- **Đặc trưng thủ công và cửa sổ trượt:** các bộ phát hiện như Viola–Jones, HOG và DPM sử dụng đặc trưng được thiết kế trước kết hợp bộ phân loại. Chúng đặt nền móng cho phát hiện khuôn mặt và người đi bộ, nhưng việc quét nhiều vị trí, tỉ lệ và thiết kế đặc trưng riêng cho từng bài toán làm hạn chế khả năng mở rộng.
- **Detector hai giai đoạn dựa trên học sâu:** R-CNN áp dụng CNN lên các vùng ứng viên [25]; Faster R-CNN đưa mạng sinh vùng đề xuất vào cùng kiến trúc, cải thiện đáng kể tính thống nhất và tốc độ [26]. Nhóm này thường có lợi thế về độ chính xác định vị nhưng chi phí suy luận và bộ nhớ có thể lớn đối với thiết bị di động.
- **Detector một giai đoạn và tối ưu cho thiết bị biên:** SSD dự đoán đối tượng trên

nhiều bản đồ đặc trưng chỉ trong một lần suy luận [27]; YOLO hợp nhất bài toán thành ánh xạ trực tiếp từ ảnh đến hộp và xác suất lớp [16]. Sau đó, các thiết kế backbone nhẹ, hợp nhất đặc trưng đa tỉ lệ và tìm kiếm kiến trúc tạo ra MobileNet SSD, EfficientDet và nhiều biến thể YOLO hướng đến cân bằng độ chính xác–độ trễ.

Xu hướng hiện nay không chỉ tối ưu độ chính xác trên tập kiểm thử mà còn quan tâm độ trễ, bộ nhớ, mức tiêu thụ năng lượng và khả năng chuyển đổi mô hình. Đây là các tiêu chí đặc biệt quan trọng khi camera và suy luận AI cùng hoạt động liên tục trên smartphone gắn trên robot.

2.2.3 Khái niệm Bounding Box

Bounding box là hình chữ nhật song song với các trục ảnh dùng để mô tả vị trí và kích thước đối tượng. Hai cách biểu diễn thường gặp là $(x_{min}, y_{min}, x_{max}, y_{max})$, với hai góc đối diện của hộp, và (x_c, y_c, w, h) , với tâm, chiều rộng và chiều cao. Tọa độ có thể ở đơn vị pixel hoặc được chuẩn hóa về $[0, 1]$ theo kích thước ảnh.



Hình 2.1: Minh họa hộp giới hạn của đối tượng trong ảnh.

Với ảnh có kích thước $W \times H$, phép chuyển từ tọa độ hai góc sang dạng tâm đã

chuẩn hóa được xác định bởi

$$x_c = \frac{x_{min} + x_{max}}{2W}, \quad y_c = \frac{y_{min} + y_{max}}{2H}, \quad (2.2)$$

$$w = \frac{x_{max} - x_{min}}{W}, \quad h = \frac{y_{max} - y_{min}}{H}. \quad (2.3)$$

Khi triển khai, cần phân biệt hệ tọa độ của ảnh gốc, tensor đầu vào sau khi co giãn hoặc thêm biên và lớp hiển thị trên màn hình. Hộp dự đoán phải được ánh xạ ngược đúng tỉ lệ; nếu không, sai số tiền xử lý có thể làm lệch cả kết quả theo dõi lẫn lệnh quay của robot dù mô hình vẫn dự đoán chính xác trong không gian tensor.

2.2.4 Intersection over Union (IoU)

Intersection over Union đo mức chồng lấp giữa hộp dự đoán B_p và hộp nhãn chuẩn B_g :

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (2.4)$$

IoU bằng 0 khi hai hộp không giao nhau và bằng 1 khi trùng khít. Trong đánh giá, một dự đoán chỉ có thể ghép với nhãn chuẩn khi đúng lớp và IoU đạt ngưỡng quy định. Trong hậu xử lý, IoU còn được dùng bởi Non-Maximum Suppression (NMS) để loại các dự đoán trùng lặp; trong tracking-by-detection, nó có thể đóng vai trò độ tương đồng giữa hộp dự báo của track và detection mới.

Không có một ngưỡng IoU phù hợp cho mọi mục đích. PASCAL VOC thường báo cáo AP tại IoU 0,5 [30], trong khi giao thức COCO đánh giá trên dãy ngưỡng từ 0,50 đến 0,95, bước 0,05, nhằm phản ánh chặt chẽ hơn chất lượng định vị [31]. Vì vậy, mọi kết quả mAP phải đi kèm giao thức và ngưỡng IoU; không nên so sánh trực tiếp các con số được tính theo chuẩn khác nhau.

2.2.5 True Positive, False Positive, False Negative

Tại một lớp và một ngưỡng IoU xác định, các dự đoán được sắp theo độ tin cậy giảm dần rồi ghép một-một với nhãn chuẩn:

- **True Positive (TP)** là dự đoán đúng lớp, đạt ngưỡng IoU và được ghép với một nhãn chuẩn chưa có dự đoán nào nhận trước đó.
- **False Positive (FP)** là dự đoán sai lớp, không đạt ngưỡng IoU, không có nhãn tương ứng hoặc là dự đoán trùng lặp cho một nhãn đã được ghép.

- **False Negative (FN)** là đối tượng có trong nhãn chuẩn nhưng không được bất kỳ dự đoán hợp lệ nào phát hiện.

Quy tắc ghép một–một là cần thiết: nếu nhiều hộp cùng bao quanh một người, chỉ hộp hợp lệ có độ tin cậy cao nhất được tính là TP, các hộp dư được tính là FP. Ba đại lượng này là cơ sở để xây dựng Precision, Recall, đường cong Precision–Recall, AP và cuối cùng là mAP.

2.2.6 Đánh giá hiệu suất Mean Average Precision (mAP)

Precision và Recall. Precision biểu thị mức đáng tin của các cảnh báo dương, còn Recall biểu thị khả năng tìm đủ các đối tượng thực sự tồn tại:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (2.5)$$

Tăng ngưỡng confidence thường loại bớt dự đoán yếu, có thể làm Precision tăng nhưng Recall giảm. Với hệ thống chăm sóc người cao tuổi, Recall thấp có nghĩa là nguy cơ bỏ sót người hoặc sự kiện cần giám sát; ngược lại, Precision thấp làm tăng nhiễu cho tracker và các bước xử lý sau. Vì thế không thể đánh giá detector chỉ bằng một trong hai chỉ số.

Đường cong Precision–Recall. Với từng lớp, các dự đoán được sắp theo confidence giảm dần. Khi lần lượt chấp nhận thêm dự đoán, cặp $(\text{Recall}, \text{Precision})$ thay đổi và tạo thành đường cong Precision–Recall (PR). Đường cong mô tả hiệu năng trên toàn bộ miền ngưỡng confidence, thay vì tại một ngưỡng vận hành duy nhất.

Average Precision. AP tóm tắt đường cong PR của một lớp tại một ngưỡng khớp xác định. Với bao chính xác nội suy $p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$, AP có thể được diễn đạt như diện tích dưới đường cong:

$$AP = \int_0^1 p_{\text{interp}}(r) dr. \quad (2.6)$$

Trong tính toán rời rạc, giao thức đánh giá quy định cụ thể các mức recall lấy mẫu và cách nội suy. Do đó AP không chỉ phụ thuộc mô hình mà còn phụ thuộc tập dữ liệu và quy ước đánh giá.

Mean Average Precision. Với tập C lớp và một ngưỡng IoU τ , mAP là trung

bình AP trên các lớp:

$$mAP_{\tau} = \frac{1}{|C|} \sum_{c \in C} AP_{c,\tau}. \quad (2.7)$$

Theo COCO, chỉ số chính tiếp tục lấy trung bình trên mười ngưỡng $T = \{0,50, 0,55, \dots, 0,95\}$ [31]:

$$mAP_{[.50:.95]} = \frac{1}{|C||T|} \sum_{c \in C} \sum_{\tau \in T} AP_{c,\tau}. \quad (2.8)$$

AP_{50} dễ đạt hơn AP_{75} vì chấp nhận hộp ít khít hơn; $mAP_{[.50:.95]}$ phản ánh đồng thời khả năng nhận biết và chất lượng định vị. Ngoài ra, COCO còn tách kết quả theo đối tượng nhỏ, vừa và lớn. Đối với mô hình pose, chuỗi Precision–Recall–AP–mAP vẫn được giữ nguyên nhưng điều kiện ghép keypoint thường dựa trên Object Keypoint Similarity (OKS), vì vậy cần phân biệt rõ box mAP và pose mAP khi trình bày benchmark.

2.2.7 Các kiến trúc phát hiện đối tượng phù hợp thiết bị di động

Sau khi thống nhất đầu ra, quy tắc ghép và cách đánh giá, có thể so sánh các kiến trúc ứng viên trên cùng hệ tiêu chí. Ba hướng phổ biến đối với ứng dụng Android là MobileNet SSD, YOLO và EfficientDet.

MobileNet SSD. SSD thực hiện dự đoán hộp và lớp trên nhiều mức bản đồ đặc trưng để xử lý các kích thước đối tượng khác nhau [27]. Khi ghép với MobileNet—backbone sử dụng tích chập tách theo chiều sâu nhằm giảm lượng phép tính—mô hình có kích thước nhỏ, dễ lượng tử hóa và phù hợp với TensorFlow Lite [28]. Hạn chế là cấu hình chuẩn chủ yếu trả về bounding box; nếu cần 17 điểm khớp cơ thể, hệ thống phải bổ sung một mô hình pose hoặc thay đổi đầu dự đoán.

YOLO. YOLO mô hình hóa detection như một mạng hợp nhất, dự đoán trực tiếp vị trí và xác suất lớp trong một lần forward [16]. Các biến thể nhỏ phù hợp với xử lý thời gian thực, đồng thời họ mô hình có phiên bản pose trả về cả hộp người và keypoint. Lợi thế này giúp tránh một tầng ước lượng tư thế riêng; đổi lại, việc chuyển đổi sang TensorFlow Lite và giải mã đầu ra phải khớp đúng phiên bản xuất mô hình, còn tốc độ thực tế phụ thuộc thiết bị, kích thước đầu vào và delegate được sử dụng.

EfficientDet. EfficientDet kết hợp backbone EfficientNet, mạng hợp nhất đặc trưng hai chiều có trọng số (BiFPN) và compound scaling để đồng thời thay đổi độ sâu, độ rộng và độ phân giải [29]. Họ mô hình tạo nhiều mức quy mô để lựa chọn theo ngân

sách tính toán. Tuy nhiên, tương tự SSD chuẩn, EfficientDet là detector hộp; pipeline cần thêm thành phần pose nếu muốn tạo trực tiếp chuỗi khớp cho LSTM.

Bảng 2.2: So sánh định tính các họ detector liên quan đến đồ án

Họ mô hình	Đặc điểm chính	Ưu điểm khi triển khai Android	Hạn chế đối với bài toán của đồ án
MobileNet SSD	SSD một giai đoạn với backbone nhẹ	Mô hình gọn; hệ sinh thái TensorFlow Lite phổ biến; dễ tạo baseline phát hiện người	Đầu ra chuẩn chỉ có hộp và lớp; cần thêm tầng pose để cung cấp keypoint cho LSTM
YOLO	Detector hộp nhất; có các biến thể detection và pose	Cân bằng tốc độ–độ chính xác; biến thể pose sinh đồng thời hộp người và keypoint	Hậu xử lý và tensor đầu ra phụ thuộc phiên bản export; cần kiểm chứng trên đúng thiết bị đích
EfficientDet	EfficientNet, BiFPN và compound scaling	Có nhiều mức mô hình; khai thác đặc trưng đa tỉ lệ hiệu quả	Phiên bản chuẩn không sinh keypoint; cấu hình lớn có thể vượt ngân sách độ trễ của smart-phone

Các giá trị FPS hoặc mAP công bố trong những bài báo khác nhau không được đặt cạnh nhau như một phép so sánh trực tiếp nếu khác tập dữ liệu, độ phân giải, phần cứng và giao thức đo. Với yêu cầu của đồ án, tiêu chí quyết định là khả năng sinh đồng thời hộp người và 17 keypoint trong một mô hình đủ nhẹ để chạy trên Android. Vì vậy hướng YOLO-Pose được chọn để huấn luyện và chuyển đổi sang TensorFlow Lite; phiên bản mô hình, delegate và số đo thực nghiệm được trình bày tại Chương 4 và Chương 5.

2.2.8 Theo dõi đối tượng và tạo chuỗi thời gian

Detector xử lý từng khung hình độc lập nên chưa bảo đảm cùng một người giữ nguyên định danh theo thời gian. Tracking-by-detection bổ sung bước dự báo trạng thái và liên kết detection để tạo tracklet. SORT sử dụng bộ lọc Kalman cho dự báo chuyển động, IoU cho độ tương đồng và thuật toán Hungarian cho bài toán gán tối ưu [39]. Deep SORT bổ sung embedding ngoại hình nhằm giảm đổi ID khi che khuất [40]; ByteTrack khai thác cả detection có confidence thấp trong bước liên kết thứ hai để phục hồi đối tượng khó quan sát [41].

Với trạng thái x_t , ma trận chuyển trạng thái F , điều khiển u_t và quan sát z_t , hai

pha cơ bản của bộ lọc Kalman là

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} + Bu_t, \quad (2.9)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^T + Q, \quad (2.10)$$

$$K_t = P_{t|t-1}H^T(H P_{t|t-1}H^T + R)^{-1}, \quad (2.11)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(z_t - H\hat{x}_{t|t-1}), \quad (2.12)$$

$$P_{t|t} = (I - K_tH)P_{t|t-1}. \quad (2.13)$$

Ma trận chi phí giữa các track và detection được giải bằng thuật toán Hungarian [45]. Tuy nhiên, IoU đơn lẻ có thể suy giảm mạnh khi người chuyển nhanh từ tư thế đứng sang nằm. Theo mã nguồn hiện tại, đồ án dùng SORT được mở rộng bằng khoảng cách tâm, tỉ lệ kích thước, khoảng cách pose và histogram vùng thân; mục tiêu chính được gán ID0, có tuổi track dài hơn và cơ chế phục hồi có điều kiện. Cách lựa chọn này giữ pipeline nhẹ trên Android nhưng vẫn cung cấp chuỗi keypoint nhất quán cho LSTM và tín hiệu vị trí ổn định cho bộ điều khiển bám người.

2.3 Họ mô hình YOLO và YOLO26-Pose

2.3.1 Quá trình phát triển của họ YOLO

YOLOv1 đặt nền móng cho detector một giai đoạn bằng cách dự đoán đồng thời hộp và xác suất lớp từ toàn ảnh [16]. Từ ý tưởng này, nhiều nhánh nghiên cứu và hiện thực cùng sử dụng tên YOLO nhưng không phải tất cả đều do một nhóm tác giả phát triển liên tục. Vì vậy, số phiên bản chỉ nên được xem là một phần của định danh; khi so sánh còn phải xác định đơn vị phát triển, nhiệm vụ, kích thước mô hình, tập dữ liệu và chế độ xuất.

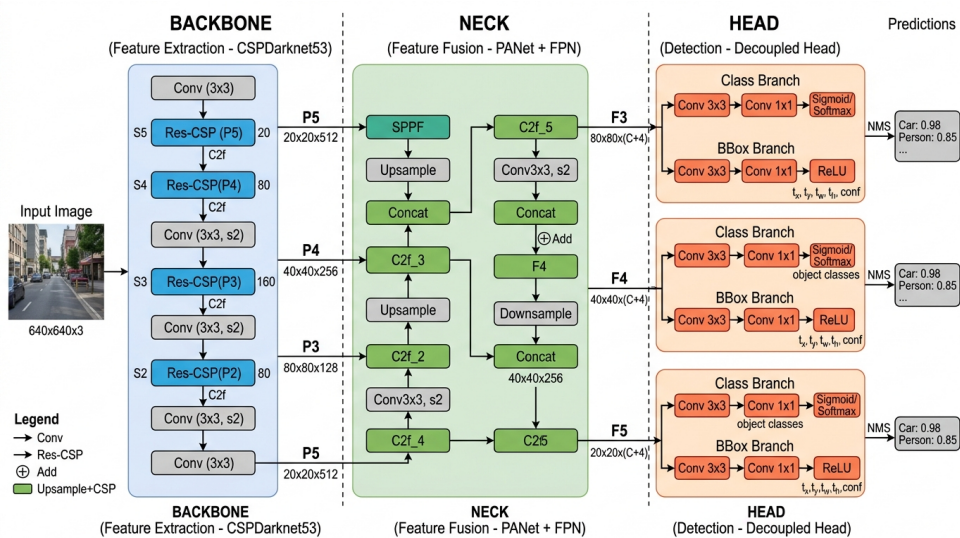
Những mốc có ảnh hưởng trực tiếp đến lựa chọn mô hình trong đồ án được tóm tắt như sau:

- **YOLOv1** hợp nhất định vị và phân loại trong một mạng, ưu tiên tốc độ thời gian thực nhưng còn hạn chế với vật thể nhỏ và các đối tượng nằm gần nhau [16].
- **YOLOv3** tăng cường backbone và dự đoán ở nhiều tỉ lệ, giúp xử lý tốt hơn sự thay đổi kích thước đối tượng [17].
- **YOLOv5** nhấn mạnh tính thực dụng của pipeline huấn luyện–suy luận–xuất mô hình, tạo tiền đề cho việc triển khai YOLO trên nhiều runtime [18].

- **YOLOv8** chuyển sang đầu dự đoán anchor-free và tách nhánh hồi quy hộp với phân loại, giảm phụ thuộc vào thiết kế anchor thủ công [21].
- **YOLOv10** sử dụng hai chiến lược gán nhãn one-to-many và one-to-one để huấn luyện detector end-to-end không cần NMS ở đường suy luận [19].
- **YOLO11** tiếp tục tinh chỉnh các khối trích xuất và hợp nhất đặc trưng, đồng thời duy trì nhiều tác vụ như detection, segmentation và pose [64].
- **YOLO26** kế thừa nền tảng YOLO11 nhưng tập trung vào suy luận end-to-end, đầu hồi quy nhẹ và các đầu dự đoán chuyên biệt cho từng tác vụ [20].

2.3.2 Cấu trúc backbone–neck–head

Dù chi tiết từng phiên bản khác nhau, các mô hình YOLO hiện đại có thể được đọc theo ba khối. Backbone biến ảnh thành các bản đồ đặc trưng phân cấp; các lớp nông giữ nhiều chi tiết vị trí, còn lớp sâu biểu diễn ngữ nghĩa mạnh hơn. Neck hợp nhất đặc trưng theo cả hướng từ sâu đến nông và từ nông đến sâu để tạo các mức P3, P4, P5. Với stride lần lượt là 8, 16 và 32, mỗi phần tử trên các mức này quan sát một vùng ảnh có kích thước và ngữ cảnh khác nhau. Head chuyển đặc trưng đa tỉ lệ thành dự đoán hộp, độ tin cậy, lớp và—đối với biến thể pose—tọa độ keypoint. Cách phân tách này hữu ích khi đọc tệp YAML của mô hình, kiểm tra lớp không được runtime hỗ trợ hoặc xác định phần đầu ra cần giải mã [21].



Hình 2.2: Cấu trúc khái quát của một detector YOLO gồm backbone, neck và head.

2.3.3 Các thay đổi chính của YOLO26

YOLO26 sử dụng kiến trúc hai đầu trong quá trình huấn luyện. Đầu one-to-many tạo nhiều ứng viên dương cho mỗi đối tượng, cung cấp tín hiệu học dày; đầu one-to-one học ánh xạ một dự đoán chính cho mỗi nhãn chuẩn. Ở chế độ end-to-end mặc định, chỉ đầu one-to-one được giữ lại khi suy luận, trả về tối đa 300 detection và không cần NMS. Khi ưu tiên độ chính xác hoặc runtime không hỗ trợ toán tử top- K của đường end-to-end, mô hình có thể dùng đầu one-to-many và NMS như detector dense thông thường [20].

YOLO26 đồng thời loại Distribution Focal Loss (DFL) khỏi đầu hồi quy hộp, thể hiện trong cấu hình bởi `reg_max=1`. Thay vì dự đoán phân phối trên nhiều bin cho mỗi cạnh hộp, đầu mới hồi quy trực tiếp, nhờ đó giảm số kênh và chi phí giải mã. Trong huấn luyện, MuSGD được dùng để cải thiện hội tụ; Progressive Loss dịch dần trọng số giám sát từ đầu one-to-many sang one-to-one; STAL bảo đảm các đối tượng rất nhỏ vẫn nhận được ứng viên dương. Ba kỹ thuật sau tác động đến quá trình tối ưu trọng số, không làm tăng giao diện đầu vào của ứng dụng Android [20].

2.3.4 Quy trình suy luận và hậu xử lý YOLO-Pose

Quy trình suy luận YOLO-Pose bắt đầu bằng việc sửa hướng ảnh, chuyển về thứ tự kênh màu mà mô hình yêu cầu, resize hoặc letterbox và chuẩn hóa miền giá trị. Tensor ảnh sau đó đi qua interpreter để tạo tensor dự đoán. Với các mức đặc trưng có stride $s \in \mathcal{S}$, số vị trí ứng viên của đầu dense có dạng tổng quát

$$N = \sum_{s \in \mathcal{S}} \left(\frac{W_i}{s} \right) \left(\frac{H_i}{s} \right). \quad (2.14)$$

Mỗi ứng viên chứa thông tin hộp, score/lớp và $3K$ giá trị keypoint. Tensor có thể sắp theo $[1, N, D]$ hoặc $[1, D, N]$, vì vậy kích thước và thứ tự trục phải được đọc trước khi duyệt các detection.

Ở đầu raw, hậu xử lý đổi bounding box sang dạng $xyxy$, lọc score và áp dụng NMS để loại các hộp trùng. Ở đầu end-to-end, mô hình đã chọn một tập dự đoán cuối nên thường không cần NMS bên ngoài. Sau đó, bộ giải mã tách các bộ ba keypoint, lọc theo confidence và ánh xạ đồng nhất cả hộp lẫn keypoint về không gian ảnh nguồn. Kết quả có cấu trúc mới được chuyển cho tracker, mô hình chuỗi thời gian hoặc tầng hiển thị. Trình tự này giúp cô lập sự khác nhau giữa các exporter ở bộ giải mã, thay vì để định dạng tensor lan sang toàn bộ ứng dụng.

2.3.5 Input và Output của YOLO-Pose

Input. Về mặt logic, đầu vào của YOLO-Pose là tensor ảnh $\mathbf{X}$ có batch B , chiều cao H_i , chiều rộng W_i và ba kênh màu. Khi triển khai, cần xác định chính xác thứ tự chiều mà runtime yêu cầu, chẳng hạn channels-last $[B, H_i, W_i, 3]$ hoặc channels-first $[B, 3, H_i, W_i]$. Ảnh phải được sửa đúng hướng, chuyển sang RGB, resize hoặc letterbox và biến đổi về kiểu dữ liệu cũng như miền giá trị khớp với lúc export. Mô hình float thường nhận float32 đã chuẩn hóa về $[0, 1]$; mô hình lượng tử có thể nhận uint8 hoặc int8 cùng scale và zero-point riêng. Sai thứ tự RGB/BGR hoặc chuẩn hóa lặp hai lần không làm tensor sai shape nhưng có thể làm giảm mạnh chất lượng dự đoán.

Kích thước $W_i \times H_i$ quyết định mức chi tiết mà backbone quan sát và số vị trí dự đoán ở các đầu đa tỉ lệ. Độ phân giải lớn có lợi cho người ở xa hoặc keypoint nhỏ nhưng làm tăng số phép tính, bộ nhớ trung gian và độ trễ gần theo diện tích ảnh. Với các mức đặc trưng có stride 8, 16 và 32, kích thước đầu vào thường được chọn chia hết cho 32 để các bản đồ đặc trưng có kích thước nguyên. Resize trực tiếp đơn giản nhưng có thể làm biến dạng tỉ lệ cơ thể; letterbox giữ tỉ lệ ảnh nguồn nhưng bổ sung vùng đệm cần được loại khỏi tọa độ khi hậu xử lý.

Output. YOLO26-Pose dự đoán đồng thời người và tư thế trong một lần suy luận, thay vì phát hiện người rồi chạy một mạng pose riêng trên từng vùng. Với cấu hình COCO, mỗi detection có $K = 17$ keypoint và mỗi điểm được biểu diễn bởi

$$k_j = (x_j, y_j, p_j), \quad j = 1, \dots, 17, \quad (2.15)$$

trong đó (x_j, y_j) là tọa độ và $p_j \in [0, 1]$ là confidence. Tập cấu hình chính thức mô tả giao diện này bằng kpt_shape=[17, 3] [23]. Trong huấn luyện, YOLO26 kết hợp loss dựa trên OKS với Residual Log-Likelihood Estimation (RLE): OKS chuẩn hóa sai số theo kích thước người và độ khó của từng loại khớp, còn RLE mô hình hóa phần dư và độ bất định của tọa độ [20]. Các thành phần loss thuộc giai đoạn huấn luyện; đầu ra suy luận vẫn là hộp, score/lớp và các bộ ba keypoint.

Bounding box có thể ở dạng tâm-kích thước $B_{cxywh} = (c_x, c_y, w_b, h_b)$ hoặc dạng hai góc $B_{xyxy} = (x_{\min}, y_{\min}, x_{\max}, y_{\max})$. Phép chuyển đổi là

$$\begin{aligned} x_{\min} &= c_x - \frac{w_b}{2}, & y_{\min} &= c_y - \frac{h_b}{2}, \\ x_{\max} &= c_x + \frac{w_b}{2}, & y_{\max} &= c_y + \frac{h_b}{2}. \end{aligned} \quad (2.16)$$

Dạng $xyxy$ phù hợp để tính IoU, cắt vùng ảnh và kiểm tra biên; dạng $cxcywh$ thuận tiện cho đầu hồi quy. Mỗi bounding box liên kết với đúng bộ keypoint của detection đó. Keypoint không phải tọa độ cục bộ tính từ góc hộp mà nằm trong cùng hệ tọa độ ảnh; hộp mô tả phạm vi tổng thể, còn keypoint mô tả cấu trúc hình học bên trong.

Tọa độ output có thể ở đơn vị pixel hoặc được chuẩn hóa về $[0, 1]$ theo $x_n = x/W$, $y_n = y/H$. Nếu ảnh nguồn (W_s, H_s) được resize trực tiếp thành ảnh mô hình (W_i, H_i) , phép ánh xạ ngược là

$$x_s = x_i \frac{W_s}{W_i}, \quad y_s = y_i \frac{H_s}{H_i}. \quad (2.17)$$

Với letterbox, cần trừ phần đệm (d_x, d_y) rồi chia hệ số $r = \min(W_i/W_s, H_i/H_s)$. Với ảnh phản chiếu ngang, điểm đổi thành $x' = W - x$ và bounding box phải hoán đổi hai cạnh theo $x'_{\min} = W - x_{\max}$, $x'_{\max} = W - x_{\min}$. Mọi phép biến đổi phải được áp dụng đồng nhất cho hộp và toàn bộ keypoint. Confidence của từng điểm cũng cần được giữ lại để các tầng sau loại hoặc giảm ảnh hưởng của khớp bị che khuất.

Thứ tự keypoint phải được giữ nhất quán từ dữ liệu huấn luyện, mô hình đến bộ giải mã:

Bảng 2.3: Thứ tự keypoint người theo cấu hình COCO-Pose của YOLO26

Chỉ số	Điểm cơ thể	Chỉ số	Điểm cơ thể
0	Mũi	9	Cổ tay trái
1	Mắt trái	10	Cổ tay phải
2	Mắt phải	11	Hông trái
3	Tai trái	12	Hông phải
4	Tai phải	13	Gối trái
5	Vai trái	14	Gối phải
6	Vai phải	15	Cổ chân trái
7	Khuỷu tay trái	16	Cổ chân phải
8	Khuỷu tay phải	—	—

2.3.6 Các mức kích thước của YOLO26-Pose

Họ mô hình cung cấp năm mức n, s, m, l và x. Tăng mức mô hình thường làm tăng khả năng biểu diễn và pose mAP, đồng thời tăng số tham số, phép tính, bộ nhớ và độ trễ. Bảng 2.4 sử dụng kết quả E2E chính thức tại đầu vào 640×640 trên COCO Keypoints val2017 [20]; số liệu CPU ONNX và T4 TensorRT chỉ cho thấy xu hướng giữa các mức,

không phải tốc độ dự kiến trên Android/TensorFlow Lite.

Bảng 2.4: So sánh các mức YOLO26-Pose ở chế độ end-to-end trên COCO

Mô hình	Tham số (M)	GFLOPs	Pose mAP	CPU ONNX (ms)	T4 TRT (ms)
YOLO26n-Pose	2,9	7,5	57,2	40,3	1,8
YOLO26s-Pose	10,4	23,9	63,0	85,3	2,7
YOLO26m-Pose	21,5	73,1	68,8	218,0	5,0
YOLO26l-Pose	25,9	91,3	70,4	275,4	6,5
YOLO26x-Pose	57,6	201,7	71,6	565,4	12,2

Mức nano có chi phí thấp nhất trong khi vẫn giữ đầy đủ đầu pose; các mức lớn hơn đòi hỏi thêm tham số và độ trễ để đạt mAP cao hơn. Vì vậy, lựa chọn cho một hệ thống cụ thể không chỉ dựa vào mAP lớn nhất mà phải xét đồng thời độ phân giải, tần số xử lý, bộ nhớ, nhiệt độ thiết bị và chất lượng chuỗi keypoint cung cấp cho tác vụ phía sau.

2.4 Mô hình chuỗi thời gian (Sequence Modeling)

2.4.1 Mạng nơ-ron hồi quy (RNN)

Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) là một lớp mạng nơ-ron nhân tạo chuyên biệt để xử lý dữ liệu dạng chuỗi (sequence data) như văn bản, âm thanh, hoặc chuỗi thời gian (time-series). Trong ngữ cảnh phát hiện té ngã, luồng video chứa các khung hình liên tiếp tạo thành một chuỗi dữ liệu thời gian ghi nhận chuyển động của con người [36].

Khác với mạng truyền thẳng (Feedforward Neural Network) nơi dữ liệu chỉ đi một chiều từ đầu vào đến đầu ra, RNN sở hữu các "vòng lặp" (loops) cho phép thông tin có thể lưu lại và truyền qua các bước thời gian. Nhờ cơ chế trạng thái ẩn (hidden state), mạng RNN có thể nhớ được thông tin từ quá khứ để ảnh hưởng đến quyết định ở hiện tại. Tuy nhiên, RNN truyền thống gặp phải vấn đề **triệt tiêu đạo hàm (vanishing gradient)** và **bùng nổ đạo hàm (exploding gradient)** khi phải học các phụ thuộc xa (long-term dependencies) trong những chuỗi dữ liệu dài.

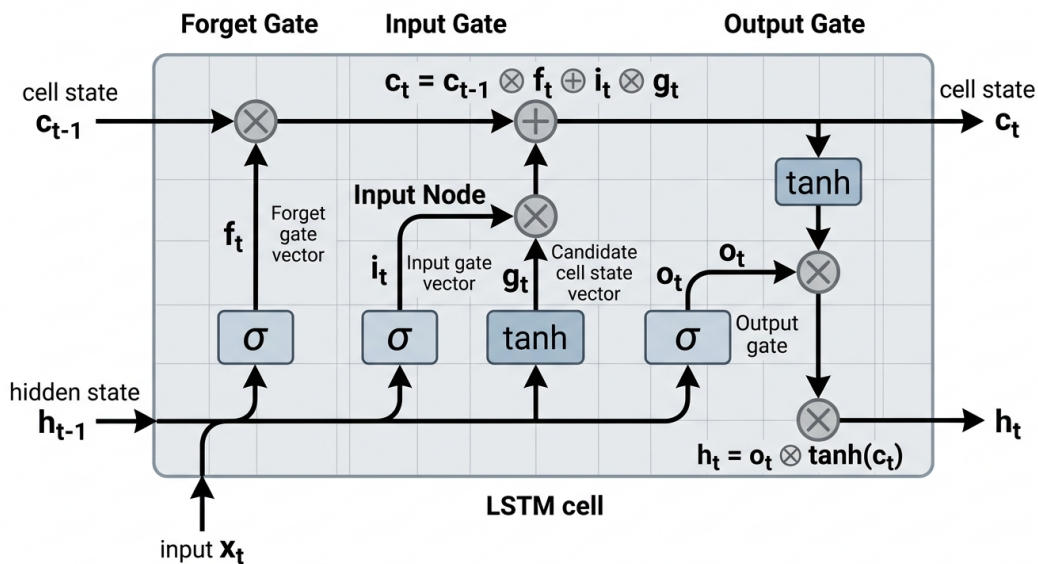
2.4.2 Mạng Bộ nhớ Ngắn-Dài hạn (LSTM)

Để khắc phục nhược điểm của RNN, mạng **Long Short-Term Memory (LSTM)** được đề xuất bởi Hochreiter và Schmidhuber (1997) [38]. LSTM có khả năng học các phụ thuộc dài hạn cực kỳ xuất sắc nhờ cấu trúc "Cell State" (trạng thái tế bào) hoạt động như một băng chuyền thông tin xuyên suốt các bước thời gian, giúp đạo hàm không bị

suy giảm quá nhanh.

Cốt lõi của kiến trúc LSTM nằm ở ba cổng (gates) đóng vai trò kiểm soát luồng thông tin:

- **Cổng quên (Forget Gate):** Quyết định lượng thông tin từ quá khứ (trạng thái tế bào trước đó) cần bị loại bỏ hoặc lãng quên.
- **Cổng đầu vào (Input Gate):** Quyết định lượng thông tin mới (từ dữ liệu hiện tại và trạng thái ẩn trước đó) nào sẽ được cập nhật vào trạng thái tế bào.
- **Cổng đầu ra (Output Gate):** Quyết định thông tin nào từ trạng thái tế bào sẽ được xuất ra dưới dạng trạng thái ẩn mới cho bước thời gian tiếp theo.



Hình 2.3: Cấu trúc chi tiết của một tế bào LSTM (LSTM cell) với hệ thống các cổng kiểm soát thông tin.

Trong kiến trúc phát hiện té ngã, YOLO-Pose trích xuất các điểm khớp ở từng khung hình và bộ theo dõi SORT mở rộng liên kết chúng theo đúng người. LSTM tiếp nhận chuỗi tọa độ đã chuẩn hóa để học sự biến đổi tư thế theo thời gian. Nhờ vậy, quyết định không chỉ dựa trên một ảnh tức thời mà còn xét quá trình chuyển động, hỗ trợ phân biệt té ngã với các hoạt động gần giống như ngồi xuống, cúi người hoặc chủ động nằm.

Một detector trên từng ảnh chỉ quan sát tư thế tại một thời điểm. Trong bài toán té ngã, quá trình chuyển từ đứng/ngồi sang hạ thấp nhanh rồi nằm mới là tín hiệu quan trọng. Recurrent Neural Network (RNN) đưa trạng thái ẩn từ bước $t - 1$ sang bước t ,

nhưng có thể gặp mất hoặc bùng nổ gradient khi chuỗi dài. Long Short-Term Memory (LSTM) bổ sung cell state và ba cổng vào, quên, ra để điều tiết thông tin [38].

Với input x_t , hidden state h_{t-1} và cell state c_{t-1} :

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad (2.18)$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \quad (2.19)$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c), \quad (2.20)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (2.21)$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad (2.22)$$

$$h_t = o_t \odot \tanh(c_t). \quad (2.23)$$

Trong hệ thống, một mẫu là tensor $X \in \mathbb{R}^{15 \times 69}$. Số 15 là chiều thời gian; 69 là vector pose/hình học mỗi frame. Đầu ra softmax hai lớp được diễn giải là normal/fall. Threshold xác suất chỉ là tầng đầu; hysteresis theo số frame liên tiếp và hậu kiểm hình học giúp ngăn trạng thái dao động. Cần lưu ý nếu FPS thay đổi, 15 frame tương ứng thời gian thực khác nhau; một triển khai tốt nên log timestamp hoặc resample chuỗi theo thời gian.

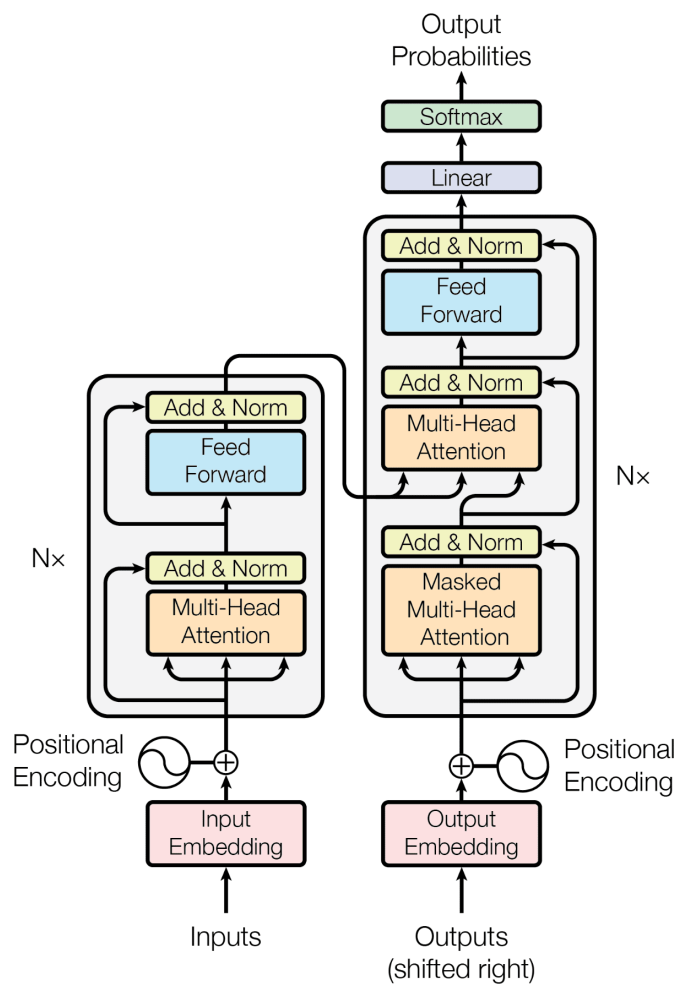
2.5 Xử lý Ngôn ngữ Tự nhiên (NLP) và Hệ thống Giọng nói

2.5.1 Mô hình ngôn ngữ lớn (LLM) và Kiến trúc Transformer

Mô hình ngôn ngữ lớn (Large Language Model - LLM) là một bước đột phá trong lĩnh vực Trí tuệ Nhân tạo, được huấn luyện trên lượng dữ liệu văn bản khổng lồ để có khả năng hiểu, suy luận và tạo ra ngôn ngữ tự nhiên giống con người. Trái tim của các mô hình LLM hiện đại chính là kiến trúc **Transformer** [37].

Kiến trúc Transformer loại bỏ hoàn toàn cơ chế xử lý tuần tự của RNN/LSTM, thay vào đó sử dụng cơ chế **Tự chú ý (Self-Attention)**. Cơ chế này cho phép mô hình đánh giá mức độ liên quan (trọng số) của từng từ trong câu với tất cả các từ khác cùng một lúc, bất kể khoảng cách giữa chúng. Nhờ đó, Transformer không chỉ giải quyết triệt để vấn đề quên thông tin tầm xa mà còn hỗ trợ tính toán song song, giúp tăng tốc độ huấn luyện lên nhiều lần.

Trong phạm vi hệ thống robot hỗ trợ người cao tuổi, việc triển khai một mô hình



Hình 2.4: Sơ đồ kiến trúc của mô hình Transformer với cơ chế Multi-Head Attention.

LLM với hàng tỷ tham số trực tiếp trên phần cứng cục bộ (edge devices) là không khả thi do giới hạn về bộ nhớ và năng lượng. Do đó, hệ thống sử dụng **Groq API** để gọi các mô hình LLM mã nguồn mở (như LLaMA 3 hoặc Mixtral) thông qua điện toán đám mây [77]. Điểm khác biệt của Groq là việc sử dụng chip LPU (Language Processing Unit) chuyên biệt cho inference, mang lại tốc độ phản hồi cực kỳ nhanh (có thể lên tới hàng trăm token/giây). Điều này rất quan trọng để đảm bảo robot có thể giao tiếp, nhắc nhở hoặc trò chuyện với người cao tuổi theo thời gian thực mà không có độ trễ gây khó chịu.

2.5.2 Khái niệm AI Agent

AI Agent (Tác tử trí tuệ nhân tạo) là một hệ thống tự động có khả năng nhận thức môi trường, đưa ra quyết định, và thực hiện hành động để đạt được mục tiêu cụ thể. Khác với các chương trình truyền thống hoạt động theo luật cố định, AI Agent có khả năng:

- **Tự chủ (Autonomy):** Hoạt động độc lập mà không cần sự can thiệp liên tục của con người
- **Phản ứng (Reactivity):** Nhận thức và phản hồi kịp thời với thay đổi của môi trường
- **Chủ động (Proactivity):** Khởi xướng hành động để đạt mục tiêu, không chỉ phản ứng thụ động
- **Xã hội (Social ability):** Tương tác và giao tiếp với con người hoặc các agent khác

Trong hệ thống robot giám sát người cao tuổi, AI Agent đóng vai trò là “bộ não” thông minh, điều phối các thành phần khác nhau (camera, cảm biến, IoT) và giao tiếp tự nhiên với người dùng.

2.5.3 Mô hình Ngôn ngữ Lớn (Large Language Models)

Large Language Models (LLM) là các mô hình học sâu được huấn luyện trên lượng dữ liệu văn bản khổng lồ, có khả năng hiểu và sinh ngôn ngữ tự nhiên ở mức độ cao. LLM đã tạo ra bước đột phá trong AI conversational.

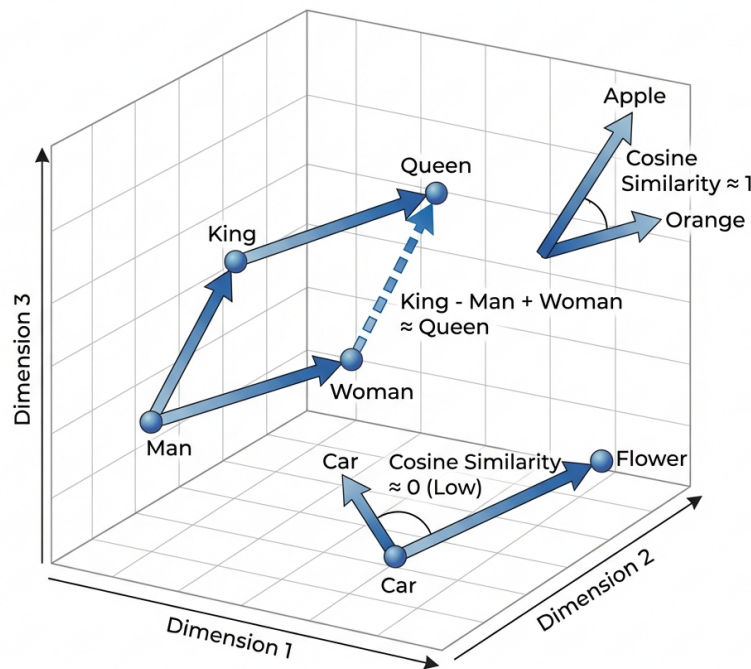
a) Kiến trúc Transformer

LLM hiện đại dựa trên kiến trúc Transformer được giới thiệu trong bài báo “Attention Is All You Need” (Vaswani et al., 2017). Transformer đã tạo ra một cuộc cách mạng trong xử lý ngôn ngữ tự nhiên bằng việc loại bỏ hoàn toàn các cấu trúc hồi quy (RNN, LSTM) và thay thế bằng cơ chế self-attention.

Lớp Embedding và Ánh xạ Không gian Đa chiều

Trước khi dữ liệu được đưa vào Transformer để huấn luyện, mỗi token (từ, ký tự, hoặc subword) cần phải đi qua lớp Embedding. Lớp này có chức năng ánh xạ dữ liệu từ không gian rời rạc (discrete space) sang không gian vector liên tục đa chiều (continuous high-dimensional space).

WORD EMBEDDING CONCEPT IN NLP



Hình 2.5: Minh họa không gian vector embedding và độ tương đồng cosine giữa các từ

Trong không gian embedding đa chiều (thường từ 512 đến 4096 chiều), các từ có ngữ nghĩa tương đồng sẽ có vector gần nhau. Độ tương đồng giữa hai vector thường được đo bằng cosine similarity:

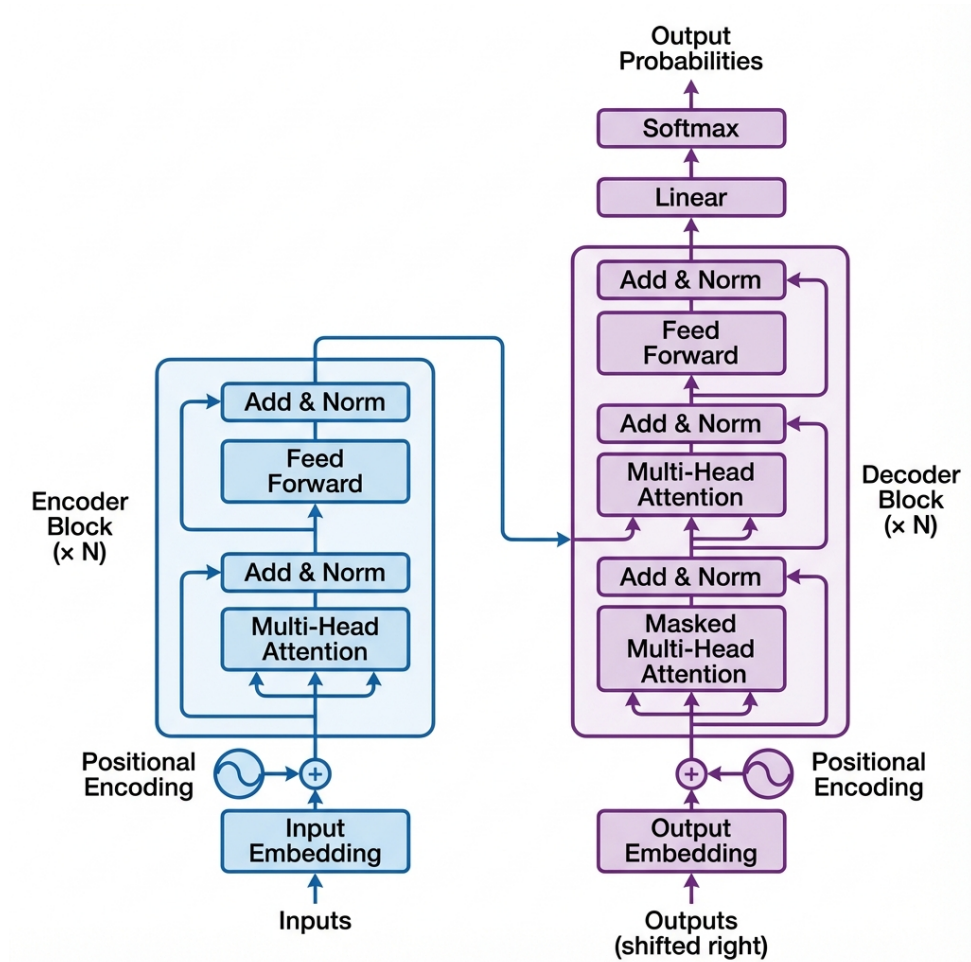
$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}} \quad (2.24)$$

Việc embedding giúp “làm dày” ngữ cảnh của mỗi data point, cho phép mô hình học được các mối quan hệ ngữ nghĩa phức tạp như:

- $\text{vector}(\text{“King”}) - \text{vector}(\text{“Man”}) + \text{vector}(\text{“Woman”}) \approx \text{vector}(\text{“Queen”})$
- Các từ đồng nghĩa có vector gần nhau trong không gian embedding

- Các từ trong cùng ngữ cảnh có cosine similarity cao

Cơ chế Self-Attention



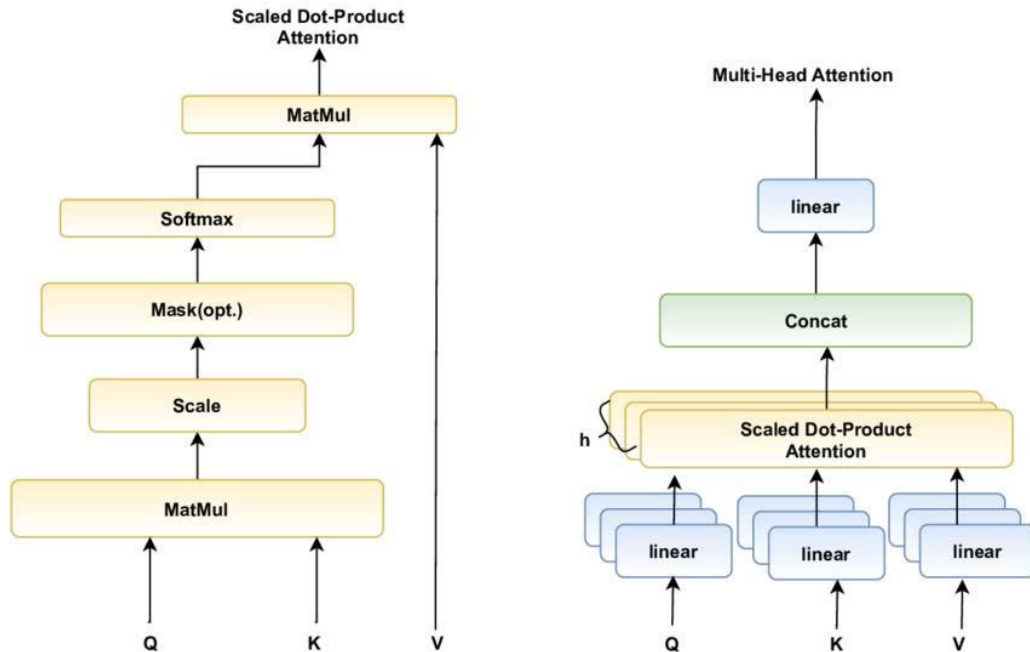
Hình 2.6: Kiến trúc Transformer với các khối Encoder và Decoder

Self-Attention cho phép mỗi vị trí trong chuỗi đầu vào “chú ý” đến tất cả các vị trí khác, học được mối quan hệ giữa các từ bất kể khoảng cách. Cơ chế này hoạt động thông qua ba ma trận: Query (Q), Key (K), và Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.25)$$

Trong đó:

- Q (Query): Biểu diễn “câu hỏi” - token hiện tại muốn tìm thông tin gì
- K (Key): Biểu diễn “chìa khóa” - các token khác chứa thông tin gì



Hình 2.7: Cơ chế Self-Attention: Query, Key, Value và Scaled Dot-Product Attention

- V (Value): Biểu diễn “giá trị” - nội dung thực sự của các token
- d_k : Chiều của vector key, dùng để scale gradient ổn định

Multi-Head Attention mở rộng cơ chế này bằng cách chạy song song nhiều attention head, mỗi head học các loại quan hệ khác nhau:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.26)$$

Trong đó mỗi head được tính toán độc lập: $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Ưu điểm của Multi-Head Attention:

- **Đa dạng hóa biểu diễn:** Mỗi head có thể học các loại quan hệ khác nhau (ví dụ: head 1 học quan hệ cú pháp, head 2 học quan hệ ngữ nghĩa, head 3 học quan hệ vị trí).
- **Tối ưu hóa song song trên GPU:** Việc chia thành nhiều head độc lập cho phép tận dụng tối đa khả năng tính toán song song của GPU. Thay vì thực hiện một phép attention lớn với chiều d_{model} , ta thực hiện h phép attention nhỏ với chiều $d_k = d_{\text{model}}/h$ song song. Điều này phù hợp hoàn hảo với kiến trúc SIMD (Single Instruction Multiple Data) của GPU, giúp:

- Tăng tốc độ training đáng kể nhờ batch matrix multiplication
- Tối ưu hóa throughput trong inference
- Sử dụng hiệu quả băng thông bộ nhớ GPU (memory bandwidth)
- Cho phép scale lên nhiều GPU dễ dàng hơn với tensor parallelism
- **Giảm chi phí tính toán:** Tổng chi phí tính toán tương đương với single-head attention có cùng chiều d_{model} , nhưng hiệu quả hơn nhờ song song hóa.

So sánh Transformer và CNN: Global Context vs Local Context

Một trong những điểm mâu chốt của Transformer so với CNN là cách nắm bắt ngữ cảnh:

Bảng 2.5: So sánh Transformer và CNN

Đặc điểm	Transformer	CNN
Ngữ cảnh	Global Context - Mỗi token có thể attention đến tất cả các token khác trong chuỗi	Local Context - Chỉ nhìn thấy các phần tử trong receptive field (kernel window)
Inductive Bias	Ít inductive bias → Cần lượng dữ liệu rất lớn để học	Có inductive bias mạnh về locality và translation equivariance
Dữ liệu cần	Yêu cầu dataset khổng lồ (hàng trăm GB văn bản)	Có thể hoạt động tốt với dataset nhỏ hơn
Phức tạp	$O(n^2)$ với độ dài chuỗi n	$O(n \cdot k^2)$ với kernel size k
Ưu điểm	Học được long-range dependencies tốt	Hiệu quả với dữ liệu có cấu trúc không gian (ảnh)

Inductive Bias: Transformer có rất ít giả định tiên nghiệm về cấu trúc dữ liệu (minimal inductive bias). Điều này có nghĩa là mô hình phải học toàn bộ ngữ cảnh và các pattern từ dữ liệu, thay vì có sẵn các “bias” như locality trong CNN. Hệ quả là Transformer cần lượng dữ liệu huấn luyện cực lớn để đạt được hiệu suất tốt.

Kiến trúc Encoder-Decoder và Decoder-Only

Transformer gốc bao gồm cả khối Encoder và Decoder:

- **Encoder:** Xử lý chuỗi đầu vào, tạo biểu diễn ngữ cảnh. Sử dụng bidirectional attention (nhìn cả trái và phải).
- **Decoder:** Sinh chuỗi đầu ra từng token một, sử dụng cross-attention để tham chiếu đến encoder output.

Tuy nhiên, các mô hình sinh văn bản như GPT chỉ sử dụng **Decoder-Only** architecture với cơ chế **Masked Self-Attention** (hay Causal Attention):

$$\text{Masked-Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V \quad (2.27)$$

Trong đó M là ma trận mask với $M_{ij} = -\infty$ nếu $j > i$ (các vị trí tương lai bị che). Điều này đảm bảo:

- Mỗi token chỉ attend được đến các token trước nó (autoregressive)
- Tránh “học vẹt” (data leakage) - mô hình không thể nhìn thấy câu trả lời khi đang dự đoán
- Phù hợp cho các tác vụ sinh văn bản (text generation)

Các Khối Phụ trợ trong Transformer

- **Layer Normalization:** Khác với Batch Normalization trong CNN, Layer Norm chuẩn hóa theo chiều feature thay vì theo batch. Điều này phù hợp hơn cho sequence data có độ dài thay đổi:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (2.28)$$

- **Residual Connection:** Skip connection giúp gradient flow tốt hơn và cho phép đào tạo các mạng rất sâu.
- **Feed-Forward Network (FFN):** Mỗi layer có một FFN với hidden dimension lớn (thường 4x embedding dimension):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2.29)$$

- **Positional Encoding:** Vì Transformer không có khái niệm về thứ tự, positional encoding được thêm vào để mã hóa vị trí của các token trong chuỗi.

b) Các mô hình LLM tiêu biểu và các bước ngoặt

Sự phát triển của LLM đã trải qua nhiều bước ngoặt quan trọng:

Các bước ngoặt trong lịch sử LLM:

1. **GPT-1 (2018)**: OpenAI giới thiệu khái niệm pre-training + fine-tuning, chứng minh unsupervised pre-training có thể cải thiện NLP tasks.
2. **BERT (2018)**: Google đề xuất bidirectional pre-training với masked language modeling, tạo bước đột phá trong NLU (Natural Language Understanding).
3. **GPT-3 (2020)**: Với 175 tỷ tham số, GPT-3 chứng minh khả năng few-shot learning mà không cần fine-tuning, mở ra kỷ nguyên prompt engineering.
4. **ChatGPT (2022)**: Sử dụng RLHF (Reinforcement Learning from Human Feedback) để align với mong muốn người dùng, tạo AI conversational thực sự hữu ích.
5. **GPT-4 (2023)**: Mô hình đa phương thức (multimodal) có thể xử lý cả văn bản và hình ảnh, reasoning ability được cải thiện đáng kể.
6. **Open-source revolution (2023-2024)**: LLaMA (Meta), Mistral, Qwen mở cửa cho cộng đồng nghiên cứu và triển khai local.
7. **DeepSeek (2024)**: Bước ngoặt quan trọng từ Trung Quốc, DeepSeek-V3 đạt hiệu suất cạnh tranh với GPT-4 với chi phí huấn luyện thấp hơn đáng kể (chỉ \$5.6M), chứng minh hiệu quả của kiến trúc Mixture of Experts (MoE) và kỹ thuật Multi-head Latent Attention (MLA).

Bảng so sánh các mô hình LLM tiêu biểu:

Mô hình	Nhà phát triển	Tham số	Open-source	Đa phương thức	Đặc điểm nổi bật
GPT-4o	OpenAI	N/A	Không	Có	SOTA reasoning, API phổ biến
Gemini 2.0	Google	N/A	Không	Có	Tích hợp Google Search, dài context
Claude 3.5	Anthropic	N/A	Không	Có	An toàn, coding mạnh
LLaMA 3.1	Meta	8B-405B	Có	Hạn chế	Cộng đồng lớn, dễ fine-tune
Mistral Large	Mistral AI	7B-123B	Có	Có	Nhẹ, hiệu quả cho edge
Qwen 2.5	Alibaba	0.5B-72B	Có	Có	Đa ngôn ngữ, nhiều size
DeepSeek-V3	DeepSeek	671B MoE	Có	Có	Chi phí thấp, MoE hiệu quả

Xu hướng hiện tại trong LLM:

- **Mixture of Experts (MoE)**: Chỉ kích hoạt một phần nhỏ tham số mỗi lần, giảm chi phí inference (DeepSeek, Mixtral).
- **Long context**: Mở rộng context window lên 128K-2M tokens (Gemini, Claude).
- **Multimodal**: Xử lý văn bản, hình ảnh, âm thanh, video trong cùng một mô hình.

- **Edge deployment:** Các mô hình nhỏ (1B-7B) chạy được trên smartphone và thiết bị nhúng.
- **Reasoning:** Cải thiện khả năng suy luận logic và toán học (o1, DeepSeek-R1).

c) Kỹ thuật Prompt Engineering

Prompt engineering là nghệ thuật thiết kế câu lệnh (prompt) để điều khiển hành vi của LLM. Các kỹ thuật prompting chính được so sánh trong bảng sau:

Kỹ thuật	Mô tả	Ví dụ prompt	Phù hợp với
Zero-shot	Yêu cầu trực tiếp không có ví dụ	“Dịch sang tiếng Anh: Xin chào”	Tác vụ đơn giản
Few-shot	Cung cấp 2-5 ví dụ mẫu trước khi hỏi	“Ví dụ: Mèo → Cat. Chó → Dog. Nhà → ?”	Tác vụ cần định dạng cụ thể
System Prompt	Định nghĩa vai trò và quy tắc cho agent	“Bạn là trợ lý y tế, luôn khuyên bệnh nhân gặp bác sĩ”	Chatbot, AI Agent

2.5.4 Hệ thống chuyển đổi Giọng nói (STT & TTS)

Để tạo ra một vòng lặp giao tiếp tự nhiên hoàn chỉnh giữa người và máy (Human-Computer Interaction), hệ thống không chỉ xử lý văn bản mà còn phải nghe và nói. Điều này được thực hiện qua hai quy trình:

- **Speech-to-Text (STT - Chuyển đổi Giọng nói thành Văn bản):** Thu nhận sóng âm (audio waveform) từ microphone của robot, lọc nhiễu, phân tích phổ âm (spectrogram) và sử dụng các mô hình học sâu âm học để nhận diện và chuyển đổi thành chuỗi văn bản. Trong hệ thống này, văn bản thu được từ STT sẽ đóng vai trò làm câu lệnh (prompt) đầu vào cho LLM thông qua Groq API.
- **Text-to-Speech (TTS - Chuyển đổi Văn bản thành Giọng nói):** Nhận chuỗi văn bản đầu ra từ LLM và tổng hợp thành tín hiệu âm thanh giọng nói con người. Hệ thống TTS hiện đại tạo ra giọng nói rất tự nhiên, có ngữ điệu và cảm xúc, giúp robot trở nên thân thiện hơn khi trò chuyện, nhắc nhở uống thuốc hoặc cảnh báo nguy hiểm.

Sự kết hợp giữa STT, Groq API (LLM siêu tốc) và TTS tạo nên một ”bộ não” giao tiếp bằng giọng nói liên tục, đóng vai trò như một người bạn đồng hành ảo, hỗ trợ tương tác tự nhiên tối đa cho người cao tuổi vốn không quen thao tác trên màn hình cảm ứng.

2.5.5 Một số giải pháp Nhận dạng Giọng nói:

Dưới đây là bảng so sánh các giải pháp chuyển đổi giọng nói thành văn bản (Speech-to-Text) phổ biến hiện nay:

Tiêu chí	Google Speech-to-Text	OpenAI Whisper	Vosk
Hình thức	Cloud API (SaaS)	Open-source (Local/Cloud)	Open-source (Local)
Độ chính xác	Rất cao (đặc biệt tiếng Việt)	Cực cao (State-of-the-art)	Khá (tùy thuộc model)
Internet	Bắt buộc	Không bắt buộc	Không cần thiết
Tốc độ	Phụ thuộc băng thông	Phụ thuộc GPU/CPU	Rất nhanh (Real-time)
Chi phí	Trả phí theo lưu lượng	Miễn phí phần mềm	Miễn phí
Phù hợp	Ứng dụng mobile cần độ chính xác tối đa.	Robot có phần cứng mạnh, cần hiểu ngữ cảnh sâu.	Thiết bị nhúng (Raspberry Pi), phản hồi cực nhanh.

Bảng 2.6: So sánh các công nghệ Speech-to-Text.

2.5.6 Phương pháp tiếp cận Xử lý Ngôn ngữ Tự nhiên (NLP) trong dự án

Thay vì tự phát triển các mô hình NLP cục bộ (như Rule-based hay phân loại Intent bằng Machine Learning) vốn bị hạn chế về khả năng hiểu ngữ cảnh tự nhiên, hoặc triển khai trực tiếp các mô hình Deep Learning lớn gây quá tải cho phần cứng robot (edge devices), đề án tiếp cận theo hướng hoàn toàn dựa vào API.

Cụ thể, hệ thống không huấn luyện hay lưu trữ mô hình ngôn ngữ riêng mà gọi trực tiếp **Groq API** để xử lý văn bản (sau khi âm thanh được STT dịch ra). Để duy trì tính mạch lạc trong giao tiếp (contextual memory), hệ thống không sử dụng các giải pháp lưu trữ phức tạp hay RAG, mà đơn giản sử dụng cơ chế **Session Chat** (Lưu trữ phiên trò chuyện). Trong cơ chế này, lịch sử của một vài vòng thoại gần nhất được lưu tạm thời trên RAM của ứng dụng và đính kèm vào phần *prompt* gửi đi mỗi lần. Nhờ vậy, LLM có thể "nhớ" được ngữ cảnh trò chuyện ngắn hạn, giúp phản hồi tự nhiên hơn với chi phí lập trình và tài nguyên hệ thống thấp nhất.

2.5.7 Phương thức triển khai: Điện toán Biên (Edge Computing) trên thiết bị Android

Để đảm bảo hệ thống phản hồi theo thời gian thực, bảo vệ quyền riêng tư và tối ưu hóa băng thông mạng, đề án đặt trọng tâm vào phương thức triển khai **Edge Computing** (Điện toán biên) trực tiếp trên thiết bị Android (Smartphone).

Các luồng thị giác cốt lõi gồm thu nhận ảnh từ camera, suy luận YOLO-Pose, theo dõi bằng SORT mở rộng và phân tích chuỗi bằng LSTM đều được thực thi trên smartphone qua TensorFlow Lite. Ứng dụng ưu tiên GPU delegate cho mô hình tương thích và có đường dự phòng CPU; cấu hình cụ thể được trình bày ở Chương 4.

Việc triển khai Edge Computing mang lại các ưu điểm vượt trội:

- **Độ trễ thấp (Low Latency):** Xử lý ảnh tại chỗ giúp robot phát hiện người ngã ngay lập tức mà không phải chờ dữ liệu gửi lên và tải về từ server.
- **Đảm bảo quyền riêng tư (Privacy):** Dữ liệu hình ảnh sinh hoạt riêng tư của người cao tuổi không bị truyền ra ngoài môi trường Internet.
- **Độ tin cậy cao:** Hệ thống vẫn hoạt động ổn định các tính năng cốt lõi ngay cả khi mất kết nối mạng.

Riêng với tính năng giao tiếp hội thoại (NLP/Chat), thiết bị mới đóng vai trò làm client gọi Groq API (như đã trình bày) nhằm tận dụng năng lực suy luận ngôn ngữ, trong khi toàn bộ phần nhận thức thị giác máy tính và phân tích hành vi đều vận hành độc lập theo mô hình Edge Computing thuần túy.

2.6 Giới thiệu Dataset Fall Detection – LE2I

2.6.1 Mục tiêu và đặc điểm của dataset LE2I

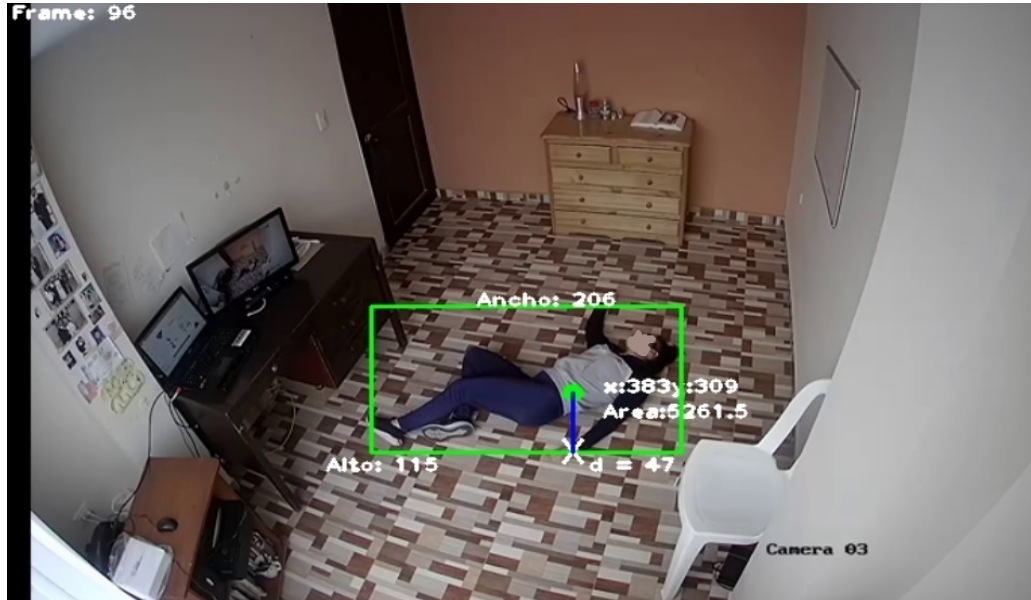
LE2I Fall Detection Dataset là một trong những bộ dữ liệu chuẩn mực và phổ biến nhất dành cho bài toán phát hiện té ngã, được phát triển bởi phòng thí nghiệm Le2i (Đại học Burgundy, Pháp). Bộ dữ liệu cung cấp các video mô phỏng các hoạt động thường ngày (Activities of Daily Living - ADL) và các tình huống té ngã đa dạng [47].

Đặc điểm nổi bật của dataset LE2I:

- **Môi trường thực tế:** Được quay ở 4 bối cảnh khác nhau bao gồm: Nhà ở (Home), Phòng cafe (Coffee room), Văn phòng (Office) và Phòng học (Lecture room).
- **Đa dạng kịch bản ngã:** Bao gồm ngã về phía trước, phía sau, ngã do mất thăng bằng, hoặc ngã khi đang cố gắng đứng lên.
- **Nhiều tự nhiên:** Video chứa các thách thức thực tế như đổ bóng (shadows), thay đổi ánh sáng (illumination variations), và che khuất (occlusions) do đồ vật.

2.6.2 Dữ liệu video và Annotation

Dataset LE2I bao gồm khoảng 191 video với các đặc điểm:



Hình 2.8: Một số khung hình minh họa trong môi trường của dataset LE2I.

- Độ phân giải video là 320x240 pixels với tốc độ khung hình 25 FPS.
- Annotation cung cấp khung hình bắt đầu và kết thúc quá trình ngã (ground truth frame numbers), kết hợp với thông tin Bounding Box của người.
- Các nhãn cơ bản gồm trạng thái ngã (fall) và hoạt động bình thường (no_fall).

2.6.3 Ưu điểm và hạn chế của dataset LE2I

Ưu điểm:

- Bối cảnh phong phú, từ không gian chật hẹp (văn phòng) đến rộng rãi (phòng học).
- Các tình huống ngã được diễn viên thực hiện tự nhiên, bao gồm các hoạt động khó phân biệt như cúi nhặt đồ hoặc ngồi xuống nhanh.
- Là chuẩn đối sánh (benchmark) uy tín được sử dụng rộng rãi trong các nghiên cứu Fall Detection.

Hạn chế:

- Độ phân giải video tương đối thấp (320x240) so với các chuẩn camera HD hiện đại.
- Số lượng chủ thể (diễn viên) còn hạn chế so với các dataset quy mô lớn gần đây.

2.6.4 So sánh với các Dataset quy mô lớn (DiverseFALL10500)

Gần đây, một số nghiên cứu đề xuất các bộ dữ liệu không lồ như DiverseFALL10500 (10,500 ảnh) để khắc phục nhược điểm của các dataset cũ [58].

So sánh LE2I và DiverseFALL10500:

Tiêu chí	LE2I Dataset	DiverseFALL10500
Số lượng video/ảnh	191 videos	10,500 images
Môi trường	4 môi trường trong nhà	Indoor + Outdoor
Độ phân giải	320x240	Đa dạng (HD)
Yếu tố che khuất	Tích hợp sẵn (bàn, ghế)	Đa dạng

Đồ án này lựa chọn sử dụng (hoặc kết hợp) LE2I do đặc thù bộ dữ liệu này phản ánh rất sát các kịch bản trong môi trường nhà ở và có chứa chuỗi chuyển động thời gian (video), vô cùng phù hợp để trích xuất Tracklet cho mô hình LSTM.

2.7 Suy luận AI trên thiết bị và TensorFlow Lite

Edge AI xử lý dữ liệu gần nguồn thay vì gửi toàn bộ lên cloud. Với camera trong nhà, cách này giảm lưu lượng, độ trễ và phạm vi truyền ảnh; đồng thời vẫn cần kiểm soát quyền và lưu trữ cục bộ. TensorFlow Lite cung cấp runtime cho thiết bị di động/biên, hỗ trợ interpreter CPU và delegate phần cứng [65].

Chuỗi chuyển đổi thường gồm model huấn luyện → biểu diễn trung gian → TFLite. Ba vấn đề phải kiểm chứng là: shape/layout tensor; toán tử được runtime/delegate hỗ trợ; và sai số số học sau tối ưu. FP16 giảm kích thước trọng số và băng thông bộ nhớ, nhưng không bảo đảm tăng tốc nếu delegate không sử dụng FP16. INT8 có thể giảm sâu hơn nhưng cần representative dataset và kiểm tra suy giảm keypoint/metric.

NMS có thể đặt trong graph hoặc ngoài graph. NMS trong graph giảm code hậu xử lý nhưng dễ tạo toán tử không tương thích delegate. NMS ngoài graph yêu cầu decoder Kotlin chính xác, đổi lại graph gọn và dễ chạy GPU hơn. Vì vậy kết quả chuyển đổi cần so ở cả tensor và detection cuối cùng.

2.8 Firebase Authentication và Realtime Database

Realtime Database tổ chức dữ liệu thành cây JSON và đồng bộ thay đổi tới client đang lắng nghe [66]. Client Android có cache và hàng đợi write, do đó callback “gần thời gian thực” không đồng nghĩa bản ghi luôn mới. Đối với lệnh robot, timestamp/expiry là

phần của ngữ nghĩa dữ liệu, không chỉ là metadata.

Firebase Authentication cung cấp UID cho người dùng; Security Rules chạy phía máy chủ quyết định read/write và có thể validate kiểu/cấu trúc [67]. Rules mặc định cần từ chối, sau đó cấp quyền tối thiểu theo UID, family và robot. Các node lệnh/signaling không nên dùng chung toàn cục trong hệ thống nhiều robot. Chúng cũng cần validate miền giá trị, người gửi và vòng đời phiên.

Ưu điểm của RTDB là tích hợp Android nhanh, listener đơn giản và phù hợp state/event nhỏ. Hạn chế là schema phi quan hệ, truy vấn có giới hạn và dễ tạo coupling bằng string path. Tách repository/data access, định nghĩa model chung và test rules bằng emulator giúp giảm rủi ro.

2.9 WebRTC, signaling và ICE

WebRTC là tập API/giao thức cho media và dữ liệu thời gian thực giữa các peer [68]. Một phiên cần signaling để trao đổi Session Description Protocol (SDP) và ICE candidate, nhưng chuẩn không bắt buộc một công nghệ signaling cụ thể. Đề tài dùng RTDB cho phần này.

ICE thu thập candidate và kiểm tra cặp đường truyền để vượt Network Address Translator [69]. STUN giúp peer biết địa chỉ public; TURN relay media khi không tạo được đường trực tiếp. Do đó có TURN làm tăng tỷ lệ kết nối nhưng thêm băng thông/độ trễ/chi phí máy chủ. Trạng thái call ở database và trạng thái PeerConnection là hai state machine khác nhau, cần cleanup idempotent để tránh phiên cũ.

WebRTC mã hóa media theo kiến trúc bảo mật giao thức, nhưng ứng dụng vẫn phải bảo vệ signaling, quyền camera/micro và danh tính peer [70]. SDP/ICE có thể lộ thông tin mạng; log không nên lưu vô hạn. Trong hệ thống chăm sóc, phải có chỉ báo rõ khi camera/micro đang truyền.

2.10 Bluetooth Low Energy và GATT

BLE phù hợp liên kết cự ly gần giữa điện thoại và vi điều khiển. GATT xây trên bảng attribute, tổ chức dữ liệu thành service, characteristic và descriptor [71]. Server công bố service/characteristic; client quét, kết nối, khám phá và thực hiện read/write/notify theo property.

Write without response giảm round-trip và phù hợp dòng lệnh cập nhật thường xuyên, nhưng không có ACK ở tầng GATT cho từng write. Ứng dụng phải dùng gửi

lập, sequence/timestamp và watchdog. UUID 128-bit cho custom service tránh xung đột profile chuẩn. Payload văn bản dễ debug nhưng tốn byte và parser cần chặt; protocol nhị phân với version, sequence và checksum là hướng nâng cấp.

BLE disconnect là trạng thái bình thường có thể xảy ra do khoảng cách, radio hoặc vòng đời Android. Motor vì vậy không được giữ setpoint cuối. Callback disconnect và timeout độc lập ở firmware là yêu cầu an toàn, không nên chỉ dựa vào app.

2.11 Động cơ BLDC và điều khiển FOC

Động cơ BLDC/PMSM tạo mô-men bằng tương tác từ trường stator và rotor nam châm. Field-Oriented Control biến đổi dòng ba pha sang hệ trục quay $d-q$, tách thành phần từ thông và mô-men, sau đó dùng PWM để điều khiển điện áp [74]. Với encoder từ AS5600, hệ thống có phản hồi góc/tốc độ cho vòng kín.

Vòng điều khiển vận tốc so sánh tốc độ đặt với tốc độ đo. PID có dạng khái quát:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (2.30)$$

Giới hạn điện áp bảo vệ driver/motor; giới hạn vận tốc bảo vệ cơ khí; lọc thông thấp giảm nhiễu đo. Deadzone giúp thắng ma sát tĩnh nhưng có thể tạo bước nhảy tốc độ, vì vậy bản hoàn thiện nên bổ sung ramp/giới hạn gia tốc. SimpleFOC cung cấp các abstraction motor, driver, sensor và vòng loopFOC/move cho vi điều khiển [73].

2.12 An toàn và quản trị rủi ro AI

Một hệ thống cảnh báo người cao tuổi phải xem cả false positive và false negative. False positive lặp lại làm người dùng mất tin cậy; false negative có thể trì hoãn hỗ trợ. NIST AI RMF đề xuất các chức năng govern, map, measure và manage xuyên suốt vòng đời [94]. Với đồ án, điều này được cụ thể hóa bằng phạm vi sử dụng rõ, metric theo sự cố, log bằng chứng, giám sát lỗi, fallback và công bố giới hạn.

LLM không nên là thành phần duy nhất quyết định an toàn. Từ khóa cục bộ xử lý câu rõ, LLM chỉ phân loại trường hợp mơ hồ và lỗi mặc định nguy hiểm. Tương tự, output LSTM đi qua state machine/hậu kiểm. Các lớp độc lập làm hệ thống dễ kiểm tra hơn, nhưng không thay thế đánh giá thực địa.

2.13 Kết luận chương

Chương đã trình bày nền tảng robot dựa trên smartphone, mạng nơ-ron, CNN, detector họ YOLO, pose, dữ liệu té ngã, tracking, LSTM, LLM, nhận dạng giọng nói, AI biên, Firebase, WebRTC, BLE và FOC. Các nội dung được liên kết với lựa chọn hiện thực thay vì giả định một mô hình khảo sát chính là model cuối. Điểm then chốt là phát hiện té ngã của đề tài không dựa trên một nhãn ảnh duy nhất mà kết hợp pose, duy trì ID0, chuỗi LSTM và hậu kiểm có bằng chứng. Chương tiếp theo chuyển các khái niệm này thành yêu cầu, kiến trúc và hợp đồng cụ thể.

Chương 3 – Phân tích và thiết kế hệ thống

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống robot trợ lý hỗ trợ chăm sóc người cao tuổi tại nhà. Từ mục tiêu giao tiếp, nhắc lịch, giám sát, cảnh báo té ngã và hỗ trợ can thiệp từ xa, các yêu cầu được cụ thể hóa thành những chức năng, module và luồng xử lý có thể kiểm thử. Trên cơ sở đó, kiến trúc tổng quan được xây dựng theo bốn thành phần chính gồm ứng dụng trên robot, ứng dụng của người chăm sóc, khối xử lý mô hình AI và phần thân robot. Chương cũng phân tích các phương án công nghệ, xác định hợp đồng giao tiếp và những cơ chế an toàn làm cơ sở cho quá trình hiện thực ở Chương 4.

3.1 Phân tích yêu cầu hệ thống

3.1.1 Bài toán, mục tiêu và phạm vi

Đề tài hướng đến một robot di động đặt trong nhà, sử dụng điện thoại thông minh gắn trên robot làm bộ xử lý cấp cao. Hệ thống hỗ trợ người cao tuổi qua hội thoại bằng giọng nói, lịch nhắc, giám sát bằng thị giác, phát hiện tình huống nghi té ngã và liên lạc với người chăm sóc. Người chăm sóc sử dụng một ứng dụng Android riêng để cấu hình, theo dõi sự kiện, gọi video và điều khiển robot từ xa. Phần thân robot chỉ nhận vận tốc hai động cơ từ điện thoại qua BLE và duy trì vòng điều khiển thời gian thực trên ESP32.

Các định hướng của luận văn được cụ thể hóa trong Bảng 3.1. Trong phạm vi đề tài, khả năng hiểu ngôn ngữ tự nhiên tập trung vào hội thoại hằng ngày, nhận biết ý định gọi người chăm sóc và phân loại câu trả lời khi xác nhận sự cố té ngã. Hệ thống đóng vai trò hỗ trợ giám sát và liên lạc, không phải thiết bị chẩn đoán y khoa, không thay thế người chăm sóc và không tự đưa ra quyết định điều trị.

Bảng 3.1: Cụ thể hóa yêu cầu luận văn theo phạm vi đề tài

Định hướng ban đầu	Cụ thể hóa trong đề tài	Thành phần kiểm chứng
Tìm hiểu Intel Open-Bot và giao tiếp điện thoại-robot	Kế thừa ý tưởng dùng điện thoại làm bộ xử lý cảm nhận, kết nối với thân robot qua BLE; thiết kế lại giao thức và firmware cho phần cứng của đề tài	RobotApp, RobotControlManager, ESP32/NimBLE/SimpleFOC
Nhận dạng giọng nói và hiểu ngôn ngữ tự nhiên	Thu tiếng nói bằng dịch vụ STT của Android, duy trì hội thoại qua LLM, phát câu trả lời bằng TTS; nhận biết ý định gọi và trạng thái an toàn/nguy hiểm	SpeechRecognizerManager, GroqChatService, AndroidTtsService
AI và IoT cho xử lý, điều khiển và tích hợp	Suy luận thị giác tại RobotApp; dùng Firebase Auth/RTDB để đồng bộ cập ứng dụng; dùng WebRTC cho media và BLE cho tầng chấp hành	TFLite, Firebase, WebRTC, BLE GATT
Thị giác máy tính cho giám sát và theo dõi	Phát hiện pose, duy trì ID người mục tiêu, tự bám theo, phân tích chuỗi chuyển động và hậu kiểm sự kiện té ngã	YOLO pose, SORT, Kalman, Hungarian, Re-ID, LSTM
Đánh giá thuật toán và hệ thống kết nối	Đo thời gian suy luận, FPS, tài nguyên và chất lượng mô hình; kiểm tra độ trễ điều khiển, signaling/cuộc gọi và cơ chế dừng khi lỗi	Bộ đo trong ứng dụng, kịch bản kiểm thử và kết quả ở Chương 5

Phạm vi của đề tài gồm quản lý hồ sơ và lịch nhắc; hội thoại bằng giọng nói; ước lượng tư thế, theo dõi một người mục tiêu, tự động bám theo và phát hiện té ngã; gọi

video hai chiều; điều khiển robot bằng cần điều khiển ảo; ghi nhật ký nhắc việc, cảnh báo và cuộc gọi. Các chức năng định vị và lập bản đồ toàn bộ ngôi nhà, tránh vật cản bằng cảm biến chuyên dụng, đo chỉ số sinh tồn, gọi trực tiếp đến tổng đài y tế và quản lý đồng thời nhiều robot chưa được xem xét trong phạm vi này.

3.1.2 Tác nhân, điều kiện vận hành và ranh giới hệ thống

Hai tác nhân chính của hệ thống là người cao tuổi và người chăm sóc; Firebase, Groq, STUN/TURN và phần thân robot đảm nhiệm vai trò hỗ trợ kỹ thuật. RobotApp và CaregiverApp sử dụng cùng một tài khoản Firebase, trong đó UID xác định không gian dữ liệu riêng của cặp thiết bị tại `pair_sessions/{uid}`. Cách ghép cặp này phù hợp với mô hình một người chăm sóc—một robot của đề tài, nhưng chưa đáp ứng bài toán phân quyền cho nhiều thành viên gia đình hoặc quản lý nhiều robot.

Bảng 3.2: Tác nhân và trách nhiệm trong hệ thống

Tác nhân/thành phần	Mục tiêu	Tương tác chính
Người cao tuổi	Nhận hỗ trợ, lời nhắc, hội thoại và liên lạc	Nói với RobotApp; xuất hiện trong camera; trả lời câu hỏi xác nhận; tham gia cuộc gọi
Người chăm sóc	Cấu hình, giám sát và can thiệp từ xa	Dùng CaregiverApp để quản lý hồ sơ/lịch, bật AI, xem nhật ký, gọi video và lái robot
RobotApp	Điều phối toàn bộ xử lý tại robot	Suy luận AI, hội thoại, nhắc lịch, signaling/media, nhận lệnh RTDB và gửi BLE
CaregiverApp	Giao diện quản lý và điều khiển từ xa	Xác thực, lịch, nhật ký, cờ chức năng, cuộc gọi và joystick
Dịch vụ đám mây	Danh tính, đồng bộ và suy luận ngôn ngữ	Firebase Auth/RTDB, Groq API và hạ tầng STUN/TURN
ESP32/firmware	Chấp hành chuyển động	Nhận payload BLE, điều khiển hai BLDC và tự dừng khi mất lệnh

Hệ thống giả định điện thoại gắn trên robot có camera, micro, loa và BLE; hai điện thoại có Internet để dùng Firebase, Groq và WebRTC; thân robot nằm trong vùng BLE của RobotApp. Các chức năng cục bộ như suy luận pose/LSTM và watchdog firmware vẫn có thể hoạt động khi dịch vụ đám mây tạm thời gián đoạn, nhưng hội thoại LLM, đồng bộ từ xa và thiết lập cuộc gọi cần kết nối mạng.

3.1.3 Yêu cầu chức năng

Các yêu cầu chức năng được gán mã để thuận tiện liên kết với module thiết kế và kịch bản kiểm thử. Từ “phải” biểu thị hành vi bắt buộc của hệ thống; những ngưỡng định lượng trong bảng là các giá trị được sử dụng cho nguyên mẫu của đề tài.

Bảng 3.3: Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí chấp nhận
FR01	Xác thực và định danh cấp thiết bị	Người dùng đăng ký/đăng nhập bằng Firebase Auth; hai ứng dụng dùng cùng UID để truy cập đúng hồ sơ, lịch và <code>pair_sessions/{uid}</code> .
FR02	Quản lý hồ sơ chăm sóc	CaregiverApp đọc/cập nhật tên robot, cách xưng hô, người liên hệ, tình trạng, ghi chú và sở thích tại <code>Users/{uid}</code> .
FR03	Quản lý lịch nhắc	CaregiverApp tạo, sửa, bật/tắt và xóa lịch một lần, hằng ngày, hằng tuần hoặc hằng tháng tại <code>Schedules/{uid}</code> .
FR04	Thực hiện và ghi nhận lời nhắc	RobotApp kiểm tra lịch mỗi 30 s, phát nội dung bằng TTS, nhận phản hồi và cập nhật <code>reminder_logs/{uid}</code> .
FR05	Hội thoại bằng giọng nói	RobotApp thực hiện chuỗi STT-LLM-TTS, duy trì ngữ cảnh ngắn hạn và có thể nhận biết ý định gọi người chăm sóc.
FR06	Điều khiển còi giám sát	CaregiverApp bật/tắt auto-follow và phát hiện té ngã; RobotApp phản ứng theo thay đổi tại <code>pair_sessions/{uid}</code> .
FR07	Phát hiện người và pose	RobotApp suy luận mô hình YOLO pose TFLite từ CameraX hoặc frame WebRTC, trả vùng bao và 17 keypoint sau lọc/NMS.
FR08	Duy trì người mục tiêu	Hệ thống chọn một người làm ID0, bảo vệ ID này khi có nhiễu và chỉ khôi phục mục tiêu theo điều kiện vị trí, chất lượng và độ ổn định.

Mã	Yêu cầu	Tiêu chí chấp nhận
FR09	Tự bám theo người	Từ vị trí, kích thước vùng bao và các mốc pose của ID0, RobotApp sinh lệnh tiến/lùi/quay có vùng chết, hysteresis và giới hạn tốc độ.
FR10	Phân loại chuỗi té ngã	LSTM nhận cửa sổ 15×69 , trả hai xác suất NO FALL/FALL; chỉ dữ liệu của ID0 được đưa vào mô hình.
FR11	Hậu kiểm sự kiện té ngã	Sau ít nhất 5 frame FALL liên tiếp, bộ trạng thái đợi chuỗi kết thúc rồi kiểm tra đầu/vai trong ba frame ổn định hoặc xử lý mất dấu tối đa 12 frame.
FR12	Xác nhận tình trạng bằng giọng nói	Với ứng viên đủ nghi ngờ, robot dừng, phát đúng một sự kiện xác nhận, hỏi bằng TTS và chờ phản hồi tối đa 10 s.
FR13	Cảnh báo và gọi khẩn cấp	Phản hồi nguy hiểm, không rõ hoặc không có phản hồi phải tạo bản ghi <code>fall_alerts</code> và mở cuộc gọi WebRTC có lý do té ngã.
FR14	Gọi video hai chiều	Hai ứng dụng trao đổi offer, answer và ICE qua RTDB; âm thanh/hình ảnh truyền bằng WebRTC và cuộc gọi được ghi lịch sử.
FR15	Điều khiển bằng joystick	Trong cuộc gọi, CaregiverApp ghi x, y, ts tối đa 20 Hz, tắt auto-follow để tránh tranh lệnh và gửi STOP khi thả/ấn joystick.
FR16	Cầu nối RTDB-BLE	RobotApp kiểm tra độ mới và miền lệnh, ánh xạ sang hai setpoint rồi ghi payload <code>L...R...</code> tới ESP32 qua BLE.
FR17	Điều khiển động cơ	ESP32 phân tích payload, áp dụng vùng chết/giới hạn và điều khiển vận tốc kín cho hai BLDC bằng SimpleFOC.
FR18	Nhật ký chăm sóc	CaregiverApp hiển thị lịch sử lời nhắc, cảnh báo té ngã và cuộc gọi theo thời gian.

Mã	Yêu cầu	Tiêu chí chấp nhận
FR19	Chuyển đổi và triển khai mô hình	Công cụ ngoại tuyến chuyển mô hình pose từ PyTorch qua ONNX/TFLite và LSTM từ Keras H5 sang TFLite, kiểm tra tensor rồi đưa mô hình vào thư mục assets của Android.
FR20	Dừng khi lỗi	Lệnh thiếu/cũ/không hữu hạn, hết thời gian chờ ở Android, mất BLE hoặc timeout firmware phải dẫn đến setpoint 0.

3.1.4 Yêu cầu phi chức năng

Các thuộc tính chất lượng được lựa chọn theo mô hình ISO/IEC 25010 [96]. Ở giai đoạn thiết kế, các thuộc tính này được dùng để định hướng giải pháp và xây dựng tiêu chí kiểm thử; các giá trị về hiệu năng và độ chính xác được đánh giá bằng thực nghiệm ở Chương 5.

Bảng 3.4: Yêu cầu phi chức năng

Mã	Thuộc tính	Yêu cầu hoặc biện pháp thiết kế
NFR01	An toàn chuyển động	STOP phải có hiệu lực khi tắt chức năng, thả joystick, vào quy trình xác nhận té ngã, lệnh lỗi, mất BLE hoặc timeout.
NFR02	Tính kịp thời	Lệnh có timestamp; RobotApp loại lệnh quá 2 s, watchdog Android phát STOP sau 500 ms không có cập nhật và firmware tự dừng sau 800 ms.
NFR03	Độ tin cậy	Listener và tài nguyên native được giải phóng theo vòng đời; một owner cho BLE/camera; sự cố té ngã có khóa chống phát lặp.
NFR04	Khả dụng	Giao diện RobotApp ưu tiên tương tác giọng nói, trạng thái rõ và thao tác tối thiểu; CaregiverApp gom chức năng quản lý theo từng màn hình.
NFR05	Bảo mật	RTDB mặc định từ chối và chỉ cho phép auth.uid truy cập nhánh UID tương ứng; kết nối đám mây dùng TLS.

Mã	Thuộc tính	Yêu cầu hoặc biện pháp thiết kế
NFR06	Quyền riêng tư	Khung hình dùng cho pose/LSTM được xử lý trên RobotApp; Firebase không lưu hoặc chuyển tiếp video; media chỉ truyền trong phiên WebRTC.
NFR07	Hiệu quả tài nguyên	Ưu tiên GPU, sau đó NNAPI và CPU; giới hạn tốc độ suy luận, bỏ frame khi pipeline bận và không mở hai camera đồng thời.
NFR08	Tương thích	Hai ứng dụng dùng minSdk 23, target/compile SDK 36; RobotApp xử lý quyền BLE khác nhau trước và từ Android 12.
NFR09	Khả năng bảo trì	Tách module UI, tracking, hậu kiểm, hội thoại, điều khiển và firmware; đường dẫn cập thiết bị được tập trung trong FirebasePairPaths.
NFR10	Khả năng giải thích	Cảnh báo lưu xác suất, lý do hậu kiểm, số key-point, tình trạng vùng bao và vị trí thân trên; kết quả không được diễn giải như chẩn đoán.
NFR11	Khả năng mở rộng	Kiến trúc cho phép bổ sung định danh robot và quyền thành viên; phạm vi hiện thực sử dụng cùng UID để ghép một robot với một người chăm sóc.
NFR12	Khả năng đánh giá	Thời gian suy luận, FPS, chất lượng mô hình, độ trễ điều khiển và các nhánh lỗi phải quan sát hoặc tái hiện được bằng benchmark/test.

3.2 Kiến trúc tổng quan hệ thống

3.2.1 Dẫn xuất kiến trúc từ yêu cầu

Kết quả của Mục 3.1 cho thấy hệ thống không chỉ là một ứng dụng AI đơn lẻ. Nhóm yêu cầu FR07–FR13 đòi hỏi xử lý ảnh liên tục tại vị trí robot; FR14–FR18 đòi hỏi người chăm sóc có thể theo dõi và can thiệp từ xa; FR16–FR20 lại yêu cầu một tầng chấp hành cục bộ vẫn có khả năng dừng khi ứng dụng hoặc mạng gặp lỗi. Bên cạnh đó, NFR02 về tính kịp thời, NFR05–NFR06 về bảo mật và quyền riêng tư, cùng NFR07 về tài nguyên khiến việc dồn toàn bộ xử lý lên một thiết bị hoặc một dịch vụ đám mây không phù hợp.

Từ các ràng buộc trên, kiến trúc được phân chia theo ba tiêu chí. Thứ nhất là *vị trí*

phát sinh dữ liệu: hình ảnh và âm thanh xuất hiện ở robot, thao tác quản lý xuất hiện ở thiết bị người chăm sóc, còn phản hồi cơ học xuất hiện ở phần thân robot. Thứ hai là *mức thời gian thực*: suy luận thị giác và điều khiển cần phản hồi nhanh hơn nghiệp vụ hồ sơ hay nhật ký. Thứ ba là *mức độ phụ thuộc mạng*: dừng động cơ và lọc lệnh cũ phải thực hiện cục bộ, trong khi đồng bộ từ xa và hội thoại LLM có thể sử dụng dịch vụ Internet.

Bảng 3.5: Từ nhóm yêu cầu đến quyết định phân rã kiến trúc

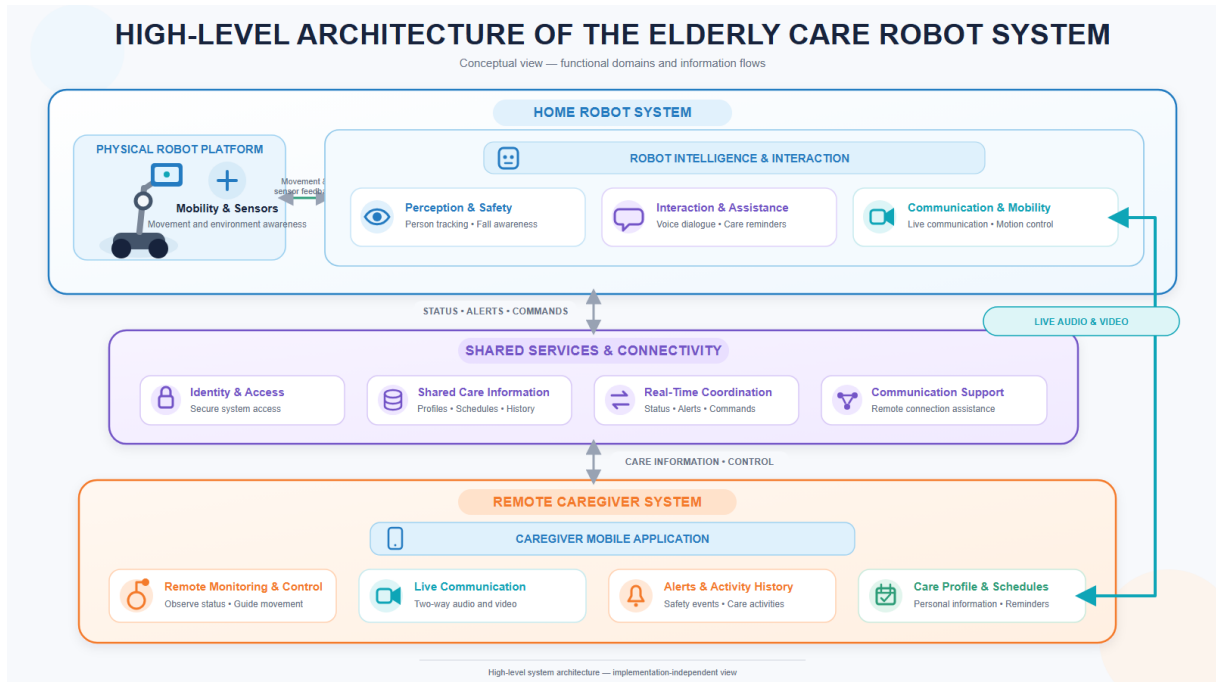
Nhóm yêu cầu	Ràng buộc thiết kế	Quyết định kiến trúc
FR01–FR06, FR18	Dữ liệu hồ sơ, lịch, cờ chức năng và nhật ký phải dùng chung giữa hai điện thoại	Tách ứng dụng quản lý cho người chăm sóc và một lớp dịch vụ dùng chung để đồng bộ trạng thái
FR07–FR13	Camera phải phục vụ phát hiện pose, giữ ID, bám theo và phát hiện té ngã với độ trễ thấp	Đặt khối cảm nhận và ra quyết định an toàn trên điện thoại gắn robot; chỉ gửi sự kiện thay vì tải liên tục video lên cơ sở dữ liệu
FR14–FR15	Cần audio/video hai chiều và điều khiển trong cuộc gọi	Tách kênh media thời gian thực khỏi kênh trạng thái/lệnh; CaregiverApp là đầu cuối giám sát từ xa
FR16–FR17, FR20	Lệnh vận tốc phải đến hai motor và robot phải tự dừng khi mất cập nhật	Tách cầu nối lệnh trên RobotApp và vòng điều khiển thời gian thực trên ESP32; bố trí watchdog ở cả hai tầng
FR19, NFR07, NFR12	Mô hình phải chuyển đổi, kiểm tra và đánh giá trước khi đóng gói trên Android	Tách pipeline AI ngoại tuyến khỏi pipeline suy luận trên thiết bị
NFR05–NFR06	Hồ sơ, hình ảnh và cảnh báo là dữ liệu nhạy cảm	Phân vùng dữ liệu theo danh tính; xử lý ảnh cục bộ; chỉ mở luồng video trong phiên liên lạc

Từ những phân tích trên, hệ thống được chia thành bốn vùng trách nhiệm: giao diện giám sát từ xa, dịch vụ dùng chung, xử lý và điều phối tại robot, cùng tầng chấp hành cơ điện. Quy trình chuẩn bị mô hình AI được tổ chức riêng ở chế độ ngoại tuyến để cung cấp mô hình cho khối xử lý trên robot. Cách phân rã này đồng thời cô lập các dạng sự cố: việc gián đoạn dịch vụ ngôn ngữ không làm mất khả năng dừng robot, mất kết nối với CaregiverApp không khiến lệnh cũ tiếp tục có hiệu lực, còn lỗi truyền thông đa phương tiện không chiếm giữ camera sau khi phiên gọi kết thúc.

3.2.2 Kiến trúc logic tổng quan

Hình 3.1 biểu diễn kiến trúc ở mức logic, chưa gắn với thư viện hay nhà cung cấp cụ thể. Ba miền triển khai chính là hệ thống tại nhà, dịch vụ dùng chung và hệ thống

người chăm sóc. Trong miền tại nhà, điện thoại gắn trên robot đảm nhiệm cảm nhận, tương tác và điều phối chuyển động; phần thân cơ điện chỉ chấp hành setpoint và duy trì bảo vệ mức thấp. Miền dùng chung cung cấp định danh, dữ liệu chăm sóc, điều phối sự kiện và hỗ trợ liên lạc. Miền người chăm sóc cung cấp giao diện cấu hình, nhận cảnh báo, quan sát và can thiệp.



Hình 3.1: Kiến trúc tổng quan của hệ thống robot hỗ trợ chăm sóc người cao tuổi

Kiến trúc có ba dòng thông tin khác nhau. *Dòng dữ liệu chăm sóc* gồm hồ sơ, lịch và nhật ký, có thể lưu bền và đồng bộ không đồng thời. *Dòng điều khiển–sự kiện* gồm cờ bật chức năng, lệnh vận tốc, cảnh báo và trạng thái cuộc gọi, yêu cầu timestamp và hành vi khi mất cập nhật. *Dòng media* gồm âm thanh/hình ảnh thời gian thực, chỉ tồn tại trong cuộc gọi và không đi qua kho dữ liệu chăm sóc. Việc tách ba dòng giúp lựa chọn cơ chế truyền phù hợp cho từng loại dữ liệu ở Mục 3.4.

3.2.3 Các module và trách nhiệm

Mỗi vùng kiến trúc được tiếp tục phân thành các module có đầu vào, đầu ra và trách nhiệm duy nhất như Bảng 3.6. Bảng này mô tả thiết kế chức năng; tên framework và lớp hiện thực cụ thể chỉ được chốt sau khi phân tích công nghệ.

Bảng 3.6: Các module trong kiến trúc logic của hệ thống

Module	Đầu vào	Xử lý/trách nhiệm	Đầu ra
Quản lý chăm sóc	Thao tác hồ sơ, lịch và cờ chức năng	Kiểm tra dữ liệu nhập, cập nhật cấu hình và truy vấn lịch sử	Hồ sơ, lịch, trạng thái giám sát
Giám sát và can thiệp từ xa	Cảnh báo, video và thao tác joystick	Hiển thị tình trạng, nhận cuộc gọi và sinh lệnh điều khiển	Phản hồi cuộc gọi, vector điều khiển
Điều phối Robo-tApp	Sự kiện UI, lịch, cờ chức năng, kết quả AI	Quản lý vòng đời camera/BLE, phân quyền nguồn lệnh và điều hướng các trạng thái hỗ trợ	Sự kiện hội thoại, cuộc gọi hoặc lệnh chuyển động
Cảm nhận và định danh mục tiêu	Frame camera	Phát hiện pose, ghép detection theo thời gian và duy trì người mục tiêu ID0	Hộp bao, key-point và track ID0
Theo dõi chuyển động	Track ID0 và cấu hình bám	Ước lượng vị trí/khoảng cách, áp dụng vùng chết và chính sách mất dấu	Vector tiến/lùi/quay hoặc STOP
Phát hiện và xác nhận té ngã	Chuỗi pose ID0	Phân loại chuỗi, hậu kiểm theo luật, khóa sự cố và yêu cầu xác nhận	SAFE hoặc sự kiện cảnh báo
Hội thoại và lời nhắc	Giọng nói, hồ sơ, lịch và sự kiện té ngã	Chuyển giọng nói-văn bản, tạo phản hồi, phát giọng nói và ghi nhận xác nhận	Âm thanh, ý định và trạng thái phản hồi

Module	Đầu vào	Xử lý/trách nhiệm	Đầu ra
Liên lạc thời gian thực	Yêu cầu gọi và tín hiệu thiết lập phiên	Thiết lập/kết thúc phiên, quản lý camera và truyền media hai chiều	Audio/video và trạng thái cuộc gọi
Dữ liệu và điều phối dùng chung	Hồ sơ, lịch, chờ, lệnh, cảnh báo và dữ liệu thiết lập cuộc gọi	Xác thực, phân vùng dữ liệu và phân phối thay đổi giữa hai thiết bị	Bản cập nhật và sự kiện theo danh tính cập thiết bị
Cầu nối điều khiển	Vector điều khiển từ xa hoặc auto-follow	Chọn nguồn lệnh, kiểm tra miền/timestamp và ánh xạ vận tốc hai bánh	Setpoint trái/phải
Chấp hành thân robot	Setpoint hai bánh và góc rotor	Điều khiển vận tốc kín, giới hạn lệnh và watchdog	PWM động cơ và trạng thái dừng
Chuẩn bị mô hình ngoại tuyến	Mô hình đã huấn luyện và cấu hình chuyển đổi	Chuyển đổi định dạng, kiểm tra tensor và tạo mô hình cho thiết bị di động	Mô hình sẵn sàng triển khai

Ranh giới quan trọng nhất nằm giữa *kết quả nhận thức* và *quyết định nghiệp vụ*. Mô hình AI chỉ trả pose hoặc xác suất FALL; module hậu kiểm mới quyết định hỏi xác nhận, còn module điều phối mới quyết định STOP, ghi cảnh báo hay gọi người chăm sóc. Tương tự, joystick và auto-follow chỉ tạo vector chuẩn hóa; module cầu nối điều khiển mới kiểm tra timestamp và chuyển thành setpoint motor. Cách tổ chức này tránh để một mô hình xác suất hoặc một màn hình giao diện tác động trực tiếp lên phần cứng.

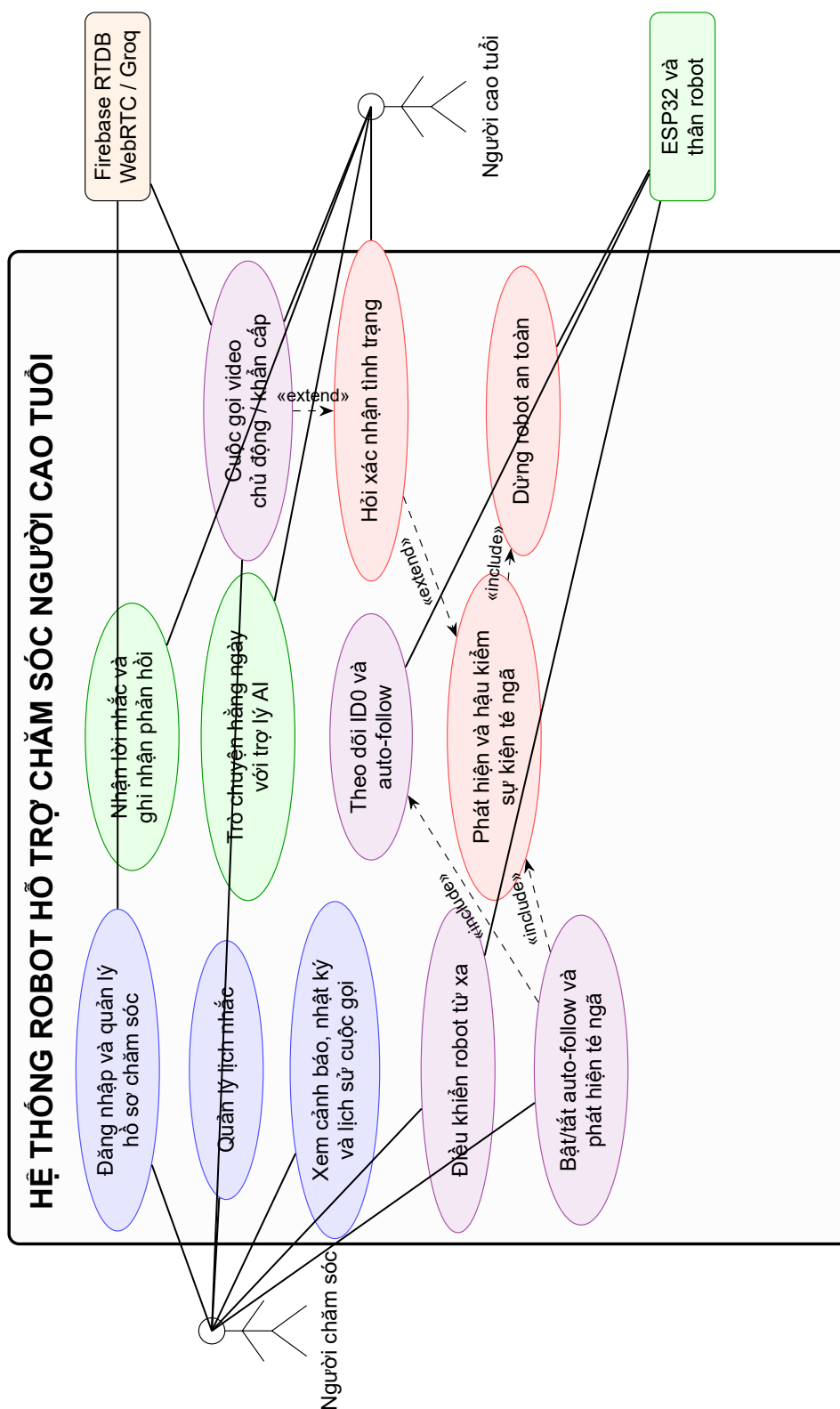
3.3 Các luồng xử lý chính

3.3.1 Biểu đồ ca sử dụng

Các ca sử dụng được nhóm thành bốn miền: quản lý hồ sơ/ lịch/ nhật ký; tương tác hội thoại và lời nhắc; giám sát, bám theo và té ngã; liên lạc và điều khiển robot. Hình 3.2 cho thấy người chăm sóc chủ yếu thao tác qua CaregiverApp, trong khi người cao tuổi tương tác với RobotApp bằng giọng nói, camera và cuộc gọi.

Bảng 3.7: Tóm tắt các ca sử dụng chính

Mã	Ca sử dụng	Tác nhân chính	Kết quả
UC01	Đăng ký/ đăng nhập	Người chăm sóc, thiết bị robot	Phiên Firebase và UID dùng chung cho cặp thiết bị
UC02	Quản lý hồ sơ	Người chăm sóc	Hồ sơ và ngữ cảnh xung hô được cập nhật
UC03	Quản lý lịch nhắc	Người chăm sóc	Lịch được đồng bộ tới RobotApp
UC04	Nhận và xác nhận lời nhắc	Người cao tuổi	Nội dung được phát và phản hồi được ghi nhật ký
UC05	Hội thoại trợ lý	Người cao tuổi	Câu trả lời được tạo và phát bằng giọng nói
UC06	Bật/tắt giám sát	Người chăm sóc	Auto-follow hoặc phát hiện té ngã thay đổi trạng thái
UC07	Phát hiện và xác nhận té ngã	Người cao tuổi, RobotApp	Loại nhiễu, xác nhận an toàn hoặc kích hoạt cảnh báo
UC08	Gọi video	Hai người dùng	Phiên WebRTC được thiết lập và kết thúc
UC09	Điều khiển robot	Người chăm sóc	Lệnh hợp lệ tới hai động cơ hoặc STOP
UC10	Xem nhật ký	Người chăm sóc	Danh sách lời nhắc, cảnh báo và cuộc gọi



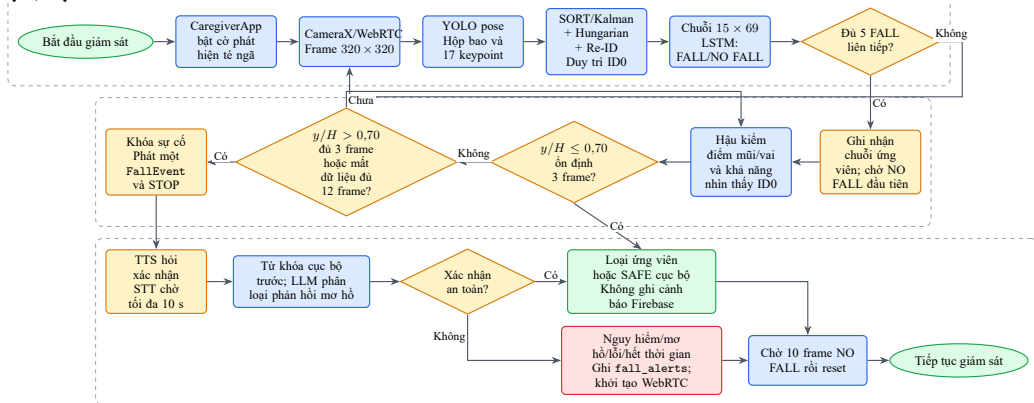
Hình 3.2: Biểu đồ ca sử dụng tổng thể của hệ thống

3.3.2 Luồng giám sát và xử lý té ngã

Hình 3.3 mô tả luồng từ lúc người chăm sóc bắt phát hiện té ngã. Mỗi frame hợp lệ đi qua YOLO pose, tracker và LSTM. Khi LSTM trả FALL trong ít nhất 5 frame liên tiếp, ứng viên được “vũ trang” nhưng chưa cảnh báo ngay. Frame NO FALL đầu tiên kết thúc chuỗi và kích hoạt hậu kiểm điểm mũi/vai: ba frame có điểm thân trên trong 70% phía trên ảnh làm ứng viên bị loại; ba frame có điểm thân trên thấp hơn ngưỡng làm hệ thống hỏi xác nhận. Nếu mục tiêu hoặc điểm thân trên mất liên tục 12 frame, hệ thống cũng hỏi theo nguyên tắc thận trọng.

Khi cần xác nhận, RobotApp khóa sự cố đang xử lý, dừng robot và chuyển quyền tương tác cho chatbot. Nếu người dùng trả lời rõ ràng rằng mình an toàn, sự cố được kết thúc tại robot và không tạo bản ghi cảnh báo. Ngược lại, phản hồi nguy hiểm, câu trả lời không rõ, lỗi phân loại hoặc không có phản hồi trong 10 s sẽ tạo bản ghi tại `fall_alerts` và khởi tạo cuộc gọi khẩn cấp. Bộ hậu kiểm chỉ trở về trạng thái giám sát sau 10 khung hình liên tiếp được phân loại NO FALL, nhờ đó một sự cố không tạo ra nhiều lượt xác nhận trùng lặp.

LUỒNG PHÁT HIỆN, HẬU KIỂM VÀ XỬ LÝ TẾ NGÃ

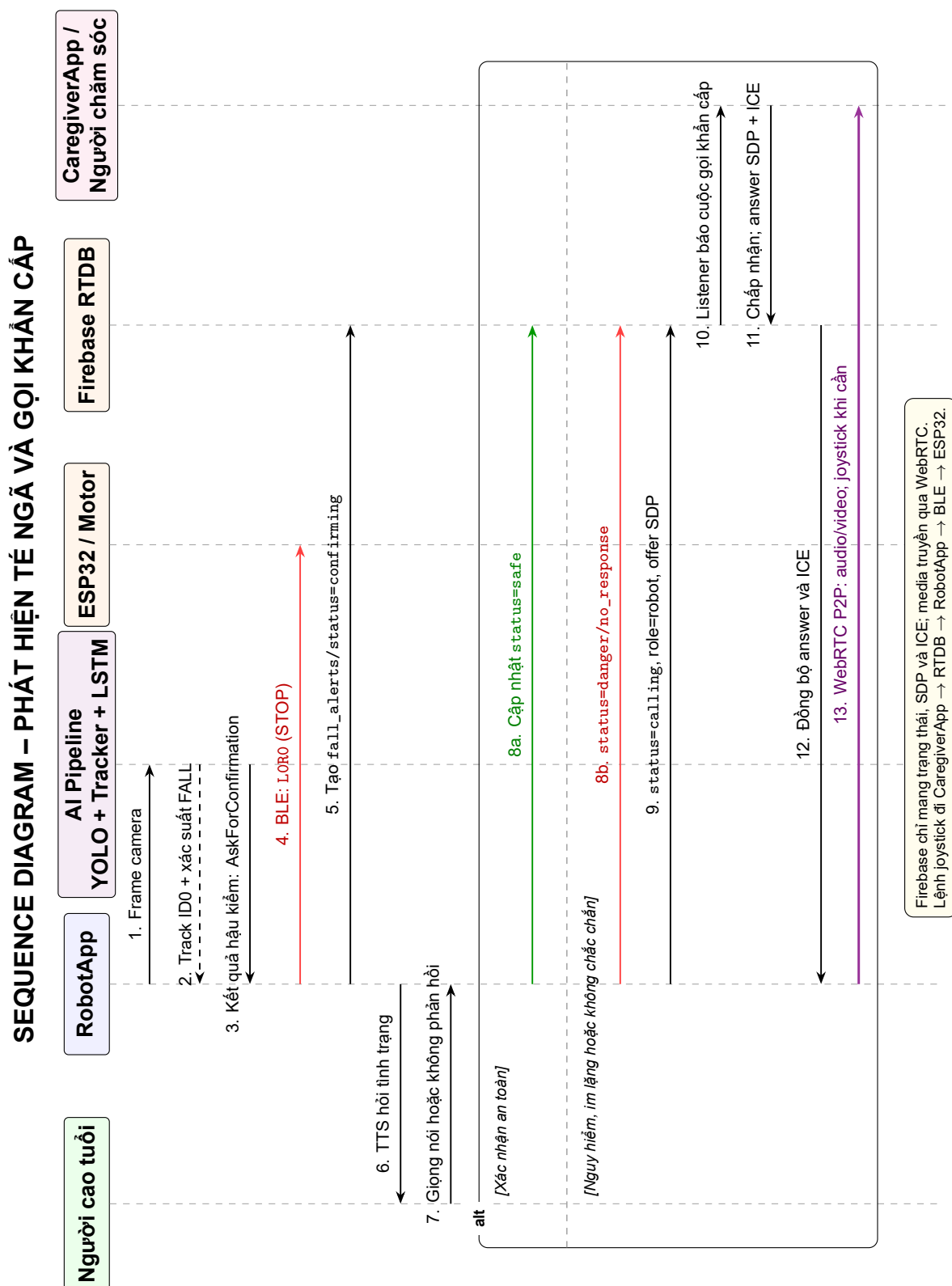


Hình 3.3: Biểu đồ hoạt động của quy trình phát hiện, hậu kiểm và xử lý té ngã

3.3.3 Luồng cuộc gọi khẩn cấp và điều khiển từ xa

Hình 3.4 trình bày trao đổi giữa người cao tuổi, RobotApp, pipeline AI, ESP32, Firebase và CaregiverApp. Pipeline AI chỉ tạo kết quả nhận thức; RobotApp mới điều phối STOP, hội thoại xác nhận, ghi dữ liệu và điều hướng sang màn hình gọi. Khi cần gọi, RobotApp ghi `status=calling`, `callerRole=robot`, `emergencyReason=fall` và offer SDP vào nhánh signaling của cặp thiết bị. CaregiverApp nhận thông báo, tạo answer và hai phía trao đổi ICE candidate.

Sau khi PeerConnection được thiết lập, audio/video truyền trực tiếp hoặc qua TURN. Nếu người chăm sóc mở joystick, lệnh đi theo chuỗi CaregiverApp → RTDB → RobotApp → BLE → ESP32. Việc điều khiển từ xa không bỏ qua các kiểm tra timestamp và watchdog.



Hình 3.4: Biểu đồ tuân tự của quá trình phát hiện té ngã và thiết lập cuộc gọi khẩn cấp

3.3.4 Luồng hội thoại và nhắc lịch

Trong hội thoại thông thường, STT tạo văn bản, GroqChatService bổ sung lịch sử gần nhất và hồ sơ vào prompt, sau đó đọc luồng delta từ Chat Completions. RobotApp tách câu hoàn chỉnh để đưa vào hàng đợi TTS, nhờ đó có thể bắt đầu nói trước khi toàn bộ phản hồi kết thúc. Sau khi TTS phát xong, máy trạng thái quay lại lắng nghe.

Đối với lịch nhắc, CaregiverApp ghi cấu hình vào Schedules/{uid}. ReminderService trên RobotApp kiểm tra mỗi 30 s, so khớp thời điểm và chế độ lặp, tạo log ở trạng thái delivered, rồi phát sự kiện cho chatbot. LLM chỉ tạo câu dẫn thân thiện; tiêu đề và ghi chú lịch vẫn lấy từ dữ liệu đã cấu hình. Phản hồi và số lần nhắc lại được cập nhật trên cùng log để CaregiverApp hiển thị.

3.4 Thiết kế và lựa chọn công nghệ cho các module

Sau khi xác định các module và luồng phối hợp, những công nghệ phù hợp được lựa chọn dựa trên độ trễ, tài nguyên của điện thoại, khả năng tích hợp và phạm vi hệ thống. Mỗi loại dữ liệu có đặc điểm riêng: giải pháp phù hợp để lưu trữ hồ sơ chưa chắc đáp ứng việc truyền video thời gian thực, còn một mô hình đạt độ chính xác cao trên máy trạm có thể không duy trì được tốc độ suy luận trên điện thoại. Bảng 3.8 tóm tắt các quyết định nền tảng; các mục tiếp theo trình bày thiết kế chi tiết của từng khối.

Bảng 3.8: So sánh các lựa chọn công nghệ xuyên suốt hệ thống

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Ứng dụng di động	Native Android; framework đa nền tảng; web app	Native truy cập trực tiếp camera, BLE, Speech API và service nhưng chỉ phục vụ Android; đa nền tảng giảm mã UI nhưng vẫn cần bridge native cho các tài nguyên trên	Kotlin, XML và ViewBinding cho cả hai ứng dụng để giảm lớp trung gian ở các chức năng thời gian thực

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Suy luận AI	Cloud inference; suy luận trên thiết bị	Cloud dễ nâng cấp model nhưng phụ thuộc mạng, tăng độ trễ và phải tải ảnh; on-device bảo vệ ảnh và tiếp tục hoạt động khi mạng gián đoạn nhưng bị giới hạn CPU/GPU, RAM và nhiệt	TFLite trên RoboApp, ưu tiên GPU/NNAPI và có CPU dự phòng [65]
Đồng bộ dữ liệu	REST tự xây dựng; MQTT; cơ sở dữ liệu thời gian thực	REST cho phép kiểm soát cao nhưng cần xây dựng máy chủ; MQTT phù hợp dữ liệu đo và điều khiển nhưng cần broker cùng cơ chế lưu trữ bổ sung; RTDB cung cấp listener và cây JSON nhưng đòi hỏi quản lý chặt cấu trúc dữ liệu và quy tắc truy cập	Firebase Authentication và Realtime Database cho mô hình một cặp thiết bị [66]

Bài toán	Phương án xem xét	Ưu điểm và hạn chế chính	Lựa chọn cho đề tài
Truyền media	Tải frame qua cơ sở dữ liệu; streaming server tập trung; WebRTC	Cơ sở dữ liệu không phù hợp media liên tục; server tập trung dễ quản lý nhưng tăng hạ tầng/băng thông; WebRTC có media thời gian thực và vượt NAT nhưng signaling/vòng đời phức tạp	WebRTC cho audio/video; RTDB chỉ làm signaling [68]
Liên kết điện thoại–thân robot	USB; Wi-Fi; BLE	USB ổn định nhưng ràng buộc cáp; Wi-Fi có băng thông cao nhưng cấu hình mạng phức tạp; BLE đủ cho setpoint nhỏ, tiêu thụ thấp và phù hợp cự ly gần nhưng băng thông hạn chế	BLE GATT vì payload điều khiển nhỏ và không truyền ảnh [71]
Hội thoại	Mô hình cục bộ; API LLM; kịch bản cố định	Mô hình cục bộ riêng tư nhưng nặng trên thiết bị; API linh hoạt nhưng phụ thuộc mạng/chi phí; kịch bản ổn định nhưng hội thoại hạn chế	Android STT/TTS kết hợp API LLM, kèm từ khóa và câu mẫu cục bộ cho nhánh an toàn

3.4.1 Module ứng dụng người chăm sóc và điều phối tại robot

CaregiverApp đảm nhiệm vai trò giao diện quản lý từ xa với bốn nhóm chức năng: hồ sơ và lịch nhắc, cấu hình giám sát, cuộc gọi kèm cần điều khiển ảo, và tra cứu nhật ký. Ứng dụng tiếp nhận thao tác của người chăm sóc, đồng bộ cấu hình, gửi lệnh và xử lý yêu cầu cuộc gọi. Các tác vụ ước lượng tư thế, hội thoại chính và kết nối ESP32 không được đặt tại đây, nhờ đó thiết bị của người chăm sóc không phụ thuộc vào mô hình TFLite hoặc trình điều khiển BLE của robot.

RobotApp là bộ điều phối đặt trên robot. Ứng dụng tiếp nhận khung hình, âm thanh, lịch nhắc, cờ chức năng và lệnh từ xa, sau đó chuyển dữ liệu đến khối cảm nhận, hội thoại, cuộc gọi hoặc điều khiển tương ứng. Mô hình AI chỉ cung cấp kết quả nhận dạng và dự đoán; quyết định dừng, hỏi xác nhận hay mở cuộc gọi được thực hiện bởi máy trạng thái nghiệp vụ. Kết nối BLE được quản lý tại một đầu mối duy nhất để tránh nhiều GATT client cùng truy cập ESP32, trong khi CameraX và WebRTC luân phiên sử dụng camera theo trạng thái cuộc gọi.

Android gốc được lựa chọn thay cho framework đa nền tảng vì hai ứng dụng cần truy cập trực tiếp dịch vụ nền, vòng đời camera, Bluetooth trên nhiều phiên bản Android, SpeechRecognizer, TTS và WebRTC. Hạn chế của lựa chọn này là hệ thống chỉ hỗ trợ Android và phải quản lý chặt vòng đời của các tài nguyên phần cứng. Giao diện và dịch vụ được tách thành các fragment và service, qua đó giới hạn phạm vi trạng thái và giảm ảnh hưởng giữa phần giao diện, thuật toán AI và firmware.

3.4.2 Module cảm nhận, phát hiện pose và theo dõi ID0

Để giám sát liên tục và phát hiện té ngã ngay trên điện thoại robot, module cảm nhận phải cung cấp đồng thời hộp bao người và các điểm khớp cơ thể, duy trì tốc độ đủ để hình thành chuỗi thời gian và không phụ thuộc vào việc tải ảnh lên máy chủ. Phương án kết hợp một bộ phát hiện hộp với một mô hình ước lượng tư thế riêng cho phép thay thế từng thành phần độc lập, nhưng cần hai lượt suy luận và thêm bước truyền vùng ảnh trung gian. Mô hình pose hợp nhất tạo hộp bao và keypoint trong cùng một lượt suy luận, nhờ đó giảm số tầng xử lý.

Các phương án MobileNet SSD và EfficientDet chuẩn phù hợp làm detector nhẹ nhưng không trực tiếp cung cấp 17 keypoint; hệ thống sẽ cần thêm tầng ước lượng tư thế. Họ YOLO-Pose đáp ứng đồng thời hai loại đầu ra và hỗ trợ xuất sang TFLite/LiteRT [22]. Dựa trên quy trình ở Mục 2.3.4, hợp đồng input–output ở Mục 2.3.5 và so sánh quy

mô tại Mục 2.3.6, đề tài chọn YOLO26n-Pose. Mức nano được ưu tiên thay cho s, m, l và x vì có số tham số và chi phí tính toán thấp nhất trong họ, phù hợp suy luận liên tục khi điện thoại còn phải chạy CameraX, tracking, LSTM, giao diện, Firebase và BLE. Đánh đổi là pose mAP thấp hơn các mức lớn; quyết định cuối cùng vì thế phải được kiểm chứng bằng độ trễ và chất lượng chuỗi keypoint trên thiết bị đích, thay vì chỉ dựa vào benchmark máy trạm.

Module nhận khung hình từ CameraX hoặc bản sao khung hình WebRTC. Ảnh được đưa về kích thước 320×320 trước khi suy luận bằng YOLO-Pose; kết quả được lọc với ngưỡng tin cậy 0,20, ngưỡng NMS IoU 0,45 và ngưỡng hiển thị keypoint 0,30. Đầu ra gồm hộp bao người và 17 keypoint, được dùng chung cho theo dõi mục tiêu, ước lượng khoảng cách và LSTM. Tốc độ xử lý mục tiêu được giới hạn ở 25 FPS; nếu lượt suy luận trước chưa hoàn tất, khung hình mới sẽ bị bỏ qua để tránh tích lũy độ trễ.

Việc gửi khung hình lên dịch vụ đám mây cho phép sử dụng phần cứng mạnh hơn, nhưng làm tăng độ trễ, băng thông và rủi ro đối với dữ liệu hình ảnh. Suy luận trực tiếp trên điện thoại chịu giới hạn về tài nguyên nhưng đáp ứng tốt hơn NFR02, NFR06 và vẫn hoạt động khi Internet gián đoạn. Vì vậy, TFLite được chọn làm môi trường suy luận trên thiết bị; hệ thống ưu tiên GPU, chuyển sang NNAPI khi cần và sử dụng CPU bốn luồng làm phương án cuối [65].

Đối với tracking, SORT có ưu điểm gọn và thời gian thực nhưng phụ thuộc chất lượng detection; ByteTrack tận dụng cả detection điểm thấp tốt hơn trong che khuất nhưng tăng số phép ghép [39][41]. Một mạng Re-ID sâu có thể phân biệt ngoại hình tốt hơn nhưng làm tăng kích thước model và thời gian suy luận. Đề tài chọn cấu trúc SORT/Kalman/Hungarian, bổ sung khoảng cách pose và histogram H-S chỉ khi cần Re-ID. Chi phí ghép được thiết kế:

$$C = 0,30(1 - \text{IoU}) + 0,30\hat{d}_c + 0,30\hat{d}_p + 0,10p_a, \quad (3.1)$$

trong đó $\hat{d}_c$ là khoảng cách tâm chuẩn hóa, $\hat{d}_p$ là khoảng cách giữa các neo vai/hông và p_a phạt biến thiên diện tích. Ứng viên ban đầu được chấm theo:

$$Q = 0,60q_{area} + 0,15q_{det} + 0,15q_{vis} + 0,10q_{center} - p_{edge}. \quad (3.2)$$

Ứng viên tốt nhất được khóa thành ID0 cho cả auto-follow và phát hiện té ngã. ID0 được giữ tối đa 45 frame không khớp, còn track nhiều giữ 10 frame. Khi mất tạm

thời, hệ thống chỉ tìm lại trong 2 s; ứng viên phải nằm trong bán kính 35% đường chéo ảnh, thuộc vùng trung tâm và ổn định ba frame. Trong một sự cố té ngã chưa giải quyết, module không tự chọn người khác làm mục tiêu.

3.4.3 Module tự bám theo và điều khiển chuyển động

Module auto-follow nhận hộp bao/keypoint của ID0 và trả vector tiến–lùi–quay chuẩn hóa. Độ gần được ước lượng từ diện tích hộp bao và các mốc vai, hông, gối, mắt cá, sau đó làm mượt bằng EMA và phân thành ba dải có hysteresis: xa, an toàn và quá gần. Sai lệch tâm ngang tạo thành phần quay với vùng chết 0,07, hệ số 2,2 và biên 0,50; vector được cập nhật khoảng mỗi 120 ms.

Bộ điều khiển chỉ dựa vào tâm hộp bao có chi phí tính toán thấp, nhưng có thể đánh giá sai khoảng cách khi phần chân bị cắt khỏi ảnh. Cảm biến chiều sâu cung cấp khoảng cách trực tiếp hơn nhưng làm tăng độ phức tạp phần cứng và nằm ngoài phạm vi đề tài. Do đó, hệ thống kết hợp hộp bao với các keypoint và áp dụng chính sách chuyển động thận trọng: khi vùng bao quá lớn hoặc phần chân không còn trong khung hình, robot ưu tiên lùi để quan sát đầy đủ hơn. Sau khi xuất hiện kết quả FALL, robot không tiến về phía người mà chỉ dừng, lùi chậm hoặc quay tại chỗ; nếu mất ID0 trong trạng thái này, robot dừng ngay.

Nguồn lệnh joystick và auto-follow dùng chung hợp đồng x, y, ts nhưng loại trừ lẫn nhau. Việc hợp nhất tại một module cầu nối giúp mọi nguồn lệnh cùng chịu kiểm tra timestamp, clamp và watchdog, thay vì mỗi tính năng tự ghi BLE theo quy ước riêng.

3.4.4 Module phát hiện, hậu kiểm và xác nhận té ngã

Có ba lựa chọn chính cho quyết định té ngã từ pose: ngưỡng hình học trên từng frame, mạng phân loại ảnh/clip đầu cuối và mô hình chuỗi trên đặc trưng keypoint. Ngưỡng hình học nhẹ, dễ giải thích nhưng nhạy với góc nhìn; mô hình clip có thể học đặc trưng mạnh nhưng tốn tài nguyên và cần tập dữ liệu lớn; chuỗi keypoint giảm kích thước đầu vào và mô tả chuyển động nhưng phụ thuộc chất lượng pose. Đề tài chọn YOLO pose kết hợp LSTM, sau đó dùng hậu kiểm theo luật và hội thoại để hạn chế cảnh báo giả.

Mỗi frame của ID0 được chuyển thành 69 đặc trưng:

$$\mathbf{f}_t = [x_i, y_i, \Delta x_i, \Delta y_i]_{i=1}^{17} \oplus r_t, \quad (3.3)$$

trong đó tọa độ keypoint được chuẩn hóa theo tâm hông và kích thước pose, $\Delta x_i, \Delta y_i$ là thay đổi so với frame trước và r_t là tỷ lệ rộng/cao. Cửa sổ 15 frame tạo tensor $[1, 15, 69]$; LSTM trả $[p_{normal}, p_{fall}]$ và chọn FALL khi $p_{fall} > p_{normal}$. LSTM phù hợp vì có thể biểu diễn quan hệ thời gian mà một pose đơn lẻ không cung cấp [38].

Bộ hậu kiểm dùng các trạng thái MONITORING, FALL_SEQUENCE, VERIFYING, PROMPTED, DISMISSED và CONFIRMED. Năm kết quả FALL liên tiếp mới tạo ứng viên. Sau NO FALL đầu tiên, ba frame có điểm mũi/vai ở $y/H \leq 0,70$ loại ứng viên; ba frame ở $y/H > 0,70$, hoặc mất mục tiêu/thân trên đủ 12 frame, dẫn đến hỏi xác nhận. Nhánh mơ hồ, lỗi hoặc hết 10 s mặc định là nguy hiểm. Thiết kế nhiều tầng tăng vài frame độ trễ nhưng đổi lại có dấu vết giải thích và không biến một xác suất đơn lẻ thành cảnh báo y tế.

3.4.5 Module hội thoại và lời nhắc

Một chatbot hoàn toàn offline bảo vệ dữ liệu và không phụ thuộc mạng, nhưng mô hình đủ khả năng hội thoại khó chạy đồng thời với pipeline thị giác trên điện thoại mục tiêu. Một hệ thống kịch bản cố định nhẹ và dự đoán được nhưng không đáp ứng trò chuyện hằng ngày. Vì vậy, đề tài chọn kiến trúc lai: Android SpeechRecognizer tạo văn bản, API LLM sinh phản hồi và Android TextToSpeech phát âm thanh [80][77][81]. Từ khóa cục bộ xử lý trước các phản hồi an toàn/nguy hiểm; lời nhắc có câu mẫu dự phòng khi API thất bại.

Máy trạng thái gồm nghỉ, lắng nghe, suy nghĩ, đang nói, xác nhận lời nhắc, xác nhận té ngã và đang gọi. Hồ sơ người dùng được đưa vào system prompt, còn lịch sử hội thoại giới hạn 20 thông điệp để kiểm soát token. Phản hồi streaming được ghép theo câu và đưa tuần tự vào TTS, giảm thời gian chờ cảm nhận; các tác vụ ngắn như phân loại phản hồi hoặc nhận biết ý định gọi dùng request không streaming để tránh làm nhiều lịch sử. Module lời nhắc kiểm tra lịch mỗi 30 s, tạo log trước khi phát sự kiện và luôn giữ nội dung lịch gốc, không để LLM tự thay đổi thuốc hay thời điểm.

3.4.6 Module dữ liệu dùng chung và liên lạc thời gian thực

Đối với dữ liệu trạng thái nhỏ, Firebase RTDB được chọn thay cho backend REST tự xây dựng hoặc MQTT broker. Listener giúp hai ứng dụng nhận còi/lệnh/sự kiện gần thời gian thực và cùng dùng Firebase Authentication; hạn chế là cây JSON dễ ghép chặt schema, truy vấn không linh hoạt như cơ sở dữ liệu tài liệu và mô hình cùng UID chưa biểu diễn vai trò riêng. Vì vậy, đường dẫn được tập trung, phân vùng theo UID và mọi

lệnh điều khiển mang timestamp [66][67].

RTDB không được dùng để truyền frame. Audio/video đi qua WebRTC; RTDB chỉ mang trạng thái, offer, answer và hai nhánh ICE candidate. WebRTC có ưu điểm tối ưu cho media tương tác, P2P khi có thể và hỗ trợ TURN khi NAT cản trở; nhược điểm là vòng đời PeerConnection, signaling, timeout và chuyển quyền camera phức tạp [68][69]. Cuộc gọi hết 30 s chưa được chấp nhận sẽ hủy. Ngoài cuộc gọi, CameraX cấp Preview/ImageAnalysis; trong cuộc gọi, CameraX unbind, WebRTC mở camera duy nhất và local VideoTrack gửi một bản sao frame cho pipeline AI.

3.4.7 Module cầu nối BLE và chấp hành thân robot

Payload điều khiển chỉ gồm hai setpoint, vì vậy BLE GATT phù hợp hơn Wi-Fi về cấu hình và năng lượng, đồng thời tránh dây USB giữa điện thoại và thân robot. Đổi lại, BLE không bảo đảm một lệnh sẽ được nhận khi dùng ghi không phản hồi; do đó tính an toàn không dựa vào một gói STOP duy nhất mà dựa thêm vào tuổi lệnh và watchdog [71].

Module cầu nối là nơi duy nhất kiểm tra timestamp, giới hạn giá trị, ánh xạ vector và ghi chuỗi L...R... sang BLE. ESP32 làm GATT server, áp dụng deadzone/giới hạn rồi điều khiển hai BLDC. Hai encoder AS5600 dùng hai bus I2C; hai driver nhận 3PWM. Mỗi động cơ có 7 cặp cực, nguồn 17,5 V, giới hạn điện áp 10 V và vận tốc 40 rad/s. Bộ điều khiển velocity dùng $P = 0,1$, $I = 1,0$, $D = 0$ cùng lọc $T_f = 0,01$ s. SimpleFOC được chọn để hiện thực FOC vòng kín thay vì tự xây toàn bộ thuật toán điều khiển, giảm thời gian tích hợp nhưng vẫn đòi hỏi hiệu chỉnh tham số theo cơ khí thực tế [74][73].

3.4.8 Module chuẩn bị và triển khai mô hình AI

Mô hình huấn luyện không được nạp trực tiếp vào Android. Pipeline ngoại tuyến chuyển YOLO pose từ PyTorch sang ONNX tính 320×320 , qua onnx2tf/SavedModel rồi tạo TFLite; LSTM được chuyển từ Keras H5 sang TFLite có SELECT_TF_OPS. Biến thể YOLO raw không NMS phục vụ delegate GPU, còn biến thể E2E có NMS làm dự phòng. Khi nạp, ứng dụng đọc kích thước tensor để hỗ trợ đầu ra raw [1, 56, 2100] và E2E [1, 300, 57].

ONNX được dùng làm định dạng trung gian để kiểm tra đồ thị tính toán giữa framework huấn luyện và TensorFlow; TFLite là định dạng triển khai cuối vì phù hợp với Android và hỗ trợ các bộ tăng tốc phần cứng. Quy trình chuyển đổi nhiều bước tạo

sự linh hoạt nhưng có thể làm thay đổi toán tử hoặc quy ước đầu ra, vì vậy kích thước tensor và tính tương đương suy luận phải được kiểm tra trước khi đóng gói. Quy trình của đề tài đã có các mô hình cùng mã chuyển đổi và kiểm tra suy luận; tuy nhiên, cấu hình tái huấn luyện chưa được chuẩn hóa đầy đủ. Để tăng khả năng tái lập, cần bổ sung cấu hình dữ liệu, seed, checkpoint và tệp kê khai kèm mã băm cho từng phiên bản mô hình.

3.5 Thiết kế dữ liệu và hợp đồng giao tiếp

3.5.1 Cây dữ liệu Firebase

Firebase RTDB lưu dữ liệu theo cấu trúc cây JSON và chuyển thay đổi đến các bộ lắng nghe của hai ứng dụng [66]. Dữ liệu vận hành của từng cặp thiết bị được gom dưới `pair_sessions/{uid}`, trong khi hồ sơ, lịch và nhật ký lời nhắc được tổ chức thành các nhánh theo UID ở cấp gốc.

Bảng 3.9: Cấu trúc dữ liệu của Firebase Realtime Database

Đường dẫn	Trường chính	Mục đích
<code>Users/{uid}</code>	<code>fullName</code> , <code>robotName</code> , <code>pronoun</code> , <code>caregiver</code> , <code>medicalConditions</code> , <code>notes</code> , <code>hobbies</code>	Hồ sơ và ngữ cảnh hội thoại
<code>Schedules/{uid}/{id}</code>	<code>title</code> , <code>note</code> , <code>dateMillis</code> , <code>hour</code> , <code>minute</code> , <code>repeatMode</code> , <code>active</code>	Lịch một lần hoặc lặp
<code>reminder_logs/{uid}/{id}</code>	<code>scheduleId</code> , <code>triggeredAt</code> , <code>status</code> , <code>elderlyResponse</code> , <code>retryCount</code> , <code>confirmedAt</code>	Nhật ký thực thi lời nhắc
<code>pair_sessions/{uid}/robot_follow_enabled</code>	Boolean	Bật/tắt auto-follow
<code>pair_sessions/{uid}/ai_fall_detection_enabled</code>	Boolean	Bật/tắt LSTM/hậu kiểm

Đường dẫn	Trường chính	Mục đích
pair_sessions/{uid}/robot_control	x, y, ts	Lệnh joystick hoặc auto-follow
pair_sessions/{uid}/webrtc_signal	status, callerRole, emergencyReason, offer, answer, robotCandidates, caregiverCandidates	Signaling và trạng thái cuộc gọi
pair_sessions/{uid}/fall_alerts/{pushId}	timestamp, trackId, probFall, verificationReason, confirmationStatus, severity, evidence	Cảnh báo nguy hiểm/không phản hồi
pair_sessions/{uid}/call_history/{pushId}	timestamp, initiator, durationSec, status	Nhật ký cuộc gọi

Security Rules mặc định đặt `.read=false` và `.write=false`; mỗi nhánh chỉ cho phép khi `auth != null` và `auth.uid === $uid`. Hai nhánh lịch sử được tạo index theo timestamp. Rules này loại bỏ truy cập chéo UID ở mức cơ bản [67], nhưng chưa có vai trò caregiver/robot riêng vì hai thiết bị đang dùng cùng tài khoản.

3.5.2 Hợp đồng điều khiển trên RTDB

```

1 pair_sessions/{uid}/robot_control = {
2   "x": Number,    // translation, [-1, 1]
3   "y": Number,    // steering,    [-1, 1]
4   "ts": Long      // Unix epoch milliseconds
5 }
```

Listing 1: Cấu trúc lệnh điều khiển trong phiên ghép cặp

Trong CaregiverApp, độ lệch joystick được chuẩn hóa theo bán kính r :

$$x = \frac{\Delta y}{r}, \quad y = -\frac{\Delta x}{r}. \quad (3.4)$$

Do trục y của màn hình tăng xuống dưới, kéo lên tạo $x < 0$. RobotApp chỉ nhận lệnh khi x, y hữu hạn, timestamp tồn tại, tuổi lệnh không quá 2.000 ms và độ lệch về tương lai không quá 5.000 ms. Sau khi chặn x, y vào $[-1, 1]$, ánh xạ motor là:

$$t = \text{clip}(x, -1, 1), \quad (3.5)$$

$$s = 0,15 \text{ clip}(y, -1, 1), \quad (3.6)$$

$$v_L = (s + t)20, \quad (3.7)$$

$$v_R = (s - t)20. \quad (3.8)$$

Nếu $|x| > 0,3$ và $|y| < 0,3$, thành phần quay được đưa về 0 để giảm rung khi đi thẳng. Hai motor lắp đối xứng nên dấu của setpoint là quy ước theo trục motor đã hiệu chỉnh, không thể suy ra hướng xe chỉ bằng việc hai số cùng hay trái dấu.

Bảng 3.10: Các điểm chuẩn ánh xạ lệnh điều khiển

x	y	(v_L, v_R)	Ý nghĩa hiệu chỉnh
-1	0	$(-20, 20)$	Tiến
1	0	$(20, -20)$	Lùi
0	1	$(3, 3)$	Quay theo chiều dương
0	-1	$(-3, -3)$	Quay theo chiều âm
0	0	$(0, 0)$	Dừng

3.5.3 Hợp đồng BLE và vòng điều khiển firmware

RobotApp là GATT client, ESP32 là GATT server tên OhmniRobot. Service UUID là 4FAFC201-1FB5-459E-8FCC-C5C9C331914B; characteristic ghi là BEB5483E-36E1-4688-B7F5-EA07361B26A8. Payload ASCII có dạng L<float>R<float>\n, định dạng số theo Locale.US và ghi bằng WRITE_NO_RESPONSE. Cách tổ chức service/characteristic tuân theo mô hình GATT [71].

Tại firmware, giá trị có độ lớn dưới 0,5 được đưa về 0; giá trị khác 0 nhưng nhỏ hơn 3 được nâng lên ± 3 ; giá trị cuối bị chặn trong $[-40, 40]$. Trong mỗi vòng lặp, hai hàm loopFOC() được gọi, watchdog được kiểm tra, rồi move() nhận setpoint. Callback mất BLE đặt hai target về 0. Parser hiện chỉ kiểm tra vị trí ký tự L/R và dùng toFloat; do đó kiểm tra NaN, vô hạn và chuỗi thừa ở firmware là điểm cần tăng cường dù Android đã lọc đầu vào.

3.5.4 Cơ chế dừng an toàn nhiều tầng

Chuỗi điều khiển có bốn hàng rào bổ sung nhau:

1. UI phát STOP khi thả joystick, ấn điều khiển, tắt auto-follow hoặc kết thúc cuộc gọi.
2. RobotApp loại bản cập nhật RTDB quá 2 s, sai kiểu hoặc chứa giá trị không hữu hạn.
3. Watchdog RobotControlManager phát LØRØ sau 500 ms không nhận cập nhật lệnh.
4. Firmware đặt hai target về 0 sau 800 ms không nhận payload hoặc ngay khi BLE disconnect.

Thời gian watchdog trên Android được đặt ngắn hơn firmware để ứng dụng ưu tiên gửi lệnh dừng có chủ đích. Watchdog trên ESP32 là lớp bảo vệ cuối cùng khi ứng dụng bị treo hoặc kết nối bị ngắt trước khi lệnh STOP được gửi đi.

3.6 An toàn, bảo mật và quyền riêng tư

Hệ thống xử lý hồ sơ, thói quen, âm thanh, hình ảnh và sự cố của người cao tuổi. Vì vậy, an toàn chức năng phải được xem xét cùng an toàn thông tin. NIST AI RMF nhấn mạnh quản trị, lập bản đồ bối cảnh, đo lường và quản lý rủi ro AI [94]; OWASP MASVS cung cấp nhóm kiểm soát phù hợp cho ứng dụng di động [95].

Bảng 3.11: *Mối đe dọa, bảo vệ hiện có và phân cần hoàn thiện*

Rủi ro	Bảo vệ hiện có	Phân cần hoàn thiện
Ghi lệnh robot trái phép	Firestore Auth, Rules theo UID, timestamp và clamp tại RobotApp	Tách vai trò robot/caregiver, validate kiểu/miền/timestamp ngay trong Rules
Replay lệnh cũ	Tuổi lệnh tối đa 2 s và hai watchdog	Nonce/sequence nếu mở rộng sang môi trường đối kháng

Rủi ro	Bảo vệ hiện có	Phản cần hoàn thiện
Rò rỉ thông tin xác thực dịch vụ	HTTPS và phạm vi truy cập giới hạn	Một số thông tin xác thực còn được cấu hình phía ứng dụng; cần thay khóa đã dùng, chuyển sang máy chủ trung gian hoặc thông tin xác thực ngắn hạn và không lưu khóa trong kho mã
Độc hồ sơ/ lịch trái phép	Mặc định deny và kiểm tra <code>auth.uid</code>	Mô hình thành viên gia đình và nguyên tắc tối thiểu quyền
Chiếm signaling/cuộc gọi	Nhánh signaling theo UID, WebRTC mã hóa media	Tách phiên bằng <code>callId</code> , dọn SDP/ICE, xác thực vai trò và cấu hình TURN an toàn [70]
Cảnh báo té ngã giả	ID0, chuỗi thời gian, hậu kiểm và hồi xác nhận	Đánh giá precision theo bối cảnh nhà ở và hiệu chỉnh threshold
Bỏ sót té ngã	Mặc định nguy hiểm khi phản hồi không chắc chắn	Đánh giá recall, kiểm thử che khuất/góc nhìn và bổ sung cảm biến nếu cần
Tranh camera/BLE	Một owner và chuyển tài nguyên theo vòng đời	Kiểm thử stress khi đổi màn hình, app nền và reconnect

Hai giới hạn thiết kế cần được nêu rõ. Thứ nhất, hai thiết bị dùng chung tài khoản nên chưa phân biệt danh tính người chăm sóc và danh tính robot; CaregiverApp hiện khởi chạy `MainActivity` trực tiếp, vì vậy luồng chặn truy cập khi chưa đăng nhập cần được hoàn thiện trước triển khai thực tế. Thứ hai, ảnh không được tải lên Firebase nhưng giọng nói/văn bản hội thoại được gửi tới dịch vụ LLM và media có thể đi qua TURN; người dùng cần được thông báo và chính sách lưu giữ log phải được xác định.

3.7 Truy vết và quyết định kiến trúc

3.7.1 Ma trận truy vết yêu cầu

Bảng 3.12: Truy vết từ yêu cầu đến module và kiểm thử

Yêu cầu	Module/điểm hiện thực	Dữ liệu/giao thức	Kiểm thử
FR01– FR02	Login, SignUp, Profile, FirebasePairPaths	Auth, Users, pair_sessions	TC-AUTH, TC-PROFILE
FR03– FR04	Schedule, ReminderService, chatbot	Schedules, reminder_logs	TC-REM
FR05	chatbot, SpeechRecognizer, TTS, GroqChat-Service	HTTPS streaming	TC-VOICE
FR06– FR09	tracking fragments, SortTracker, Tracking-MotionPolicy	cờ AI, robot_control	TC-TRACK, TC-FOLLOW
FR10– FR13	LstmFallDetector, FallVerificationEngine, chatbot	FallEvent, fall_alerts	TC-FALL
FR14	video call, Incoming-CallService	SDP, ICE, WebRTC	TC-CALL
FR15– FR17	joystick, RobotControlManager, ESP32 firmware	x/y/ts, BLE L/R	TC-CONTROL
FR18	Diary fragments/adapters	logs, alerts, history	TC-DIARY
FR19	Python conversion scripts, Android assets	PT, H5, ONNX, TFLite	TC-MODEL
FR20	UI STOP, Android/firmware watchdog	timestamp, timeout	TC-SAFETY

3.7.2 Đánh đổi kiến trúc

Firestore RTDB giảm khối lượng backend và phù hợp đồng bộ trạng thái, nhưng cách ghép cặp bằng cùng UID chưa biểu diễn được vai trò và quan hệ nhiều người–nhiều robot. WebRTC tránh truyền media qua database, đổi lại cần quản lý signaling, NAT/TURN và vòng đời camera phức tạp. BLE phù hợp kênh cục bộ có độ trễ thấp nhưng có thể ngắt bất chợt, do đó bắt buộc có watchdog.

AI biên giảm độ trễ và bề mặt lộ ảnh, nhưng phụ thuộc GPU/NNAPI, pin và nhiệt của điện thoại. LLM cloud tạo hội thoại linh hoạt, song phụ thuộc mạng và không bảo đảm nội dung; vì vậy LLM không trực tiếp điều khiển động cơ và kết quả không chắc chắn trong xác nhận té ngã luôn rơi về nhánh nguy hiểm. Việc kết hợp LSTM với hậu kiểm luật tạo thêm độ trễ vài frame nhưng cung cấp bằng chứng giải thích và giảm phản ứng từ một dự đoán đơn lẻ. Hiệu quả cuối cùng phải được lượng hóa bằng benchmark và kịch bản thử nghiệm trên thiết bị mục tiêu.

3.8 Kết luận chương

Chương đã xác định phạm vi, yêu cầu chức năng và yêu cầu chất lượng của hệ thống, từ đó xây dựng kiến trúc logic dựa trên vị trí dữ liệu, mức độ thời gian thực và sự phụ thuộc vào mạng. Các module và luồng phối hợp được phân tích trước khi lựa chọn nền tảng di động, cơ chế suy luận AI tại biên, giải pháp đồng bộ dữ liệu, truyền thông đa phương tiện, hội thoại và kết nối với phần thân robot. Thiết kế đồng thời làm rõ ranh giới giữa kết quả nhận thức của AI, quyết định nghiệp vụ và quá trình chấp hành cơ điện, cùng các hợp đồng giao tiếp và cơ chế dừng an toàn. Đây là cơ sở để xây dựng kiến trúc hiện thực ở Chương 4 và tổ chức đánh giá hệ thống ở Chương 5.

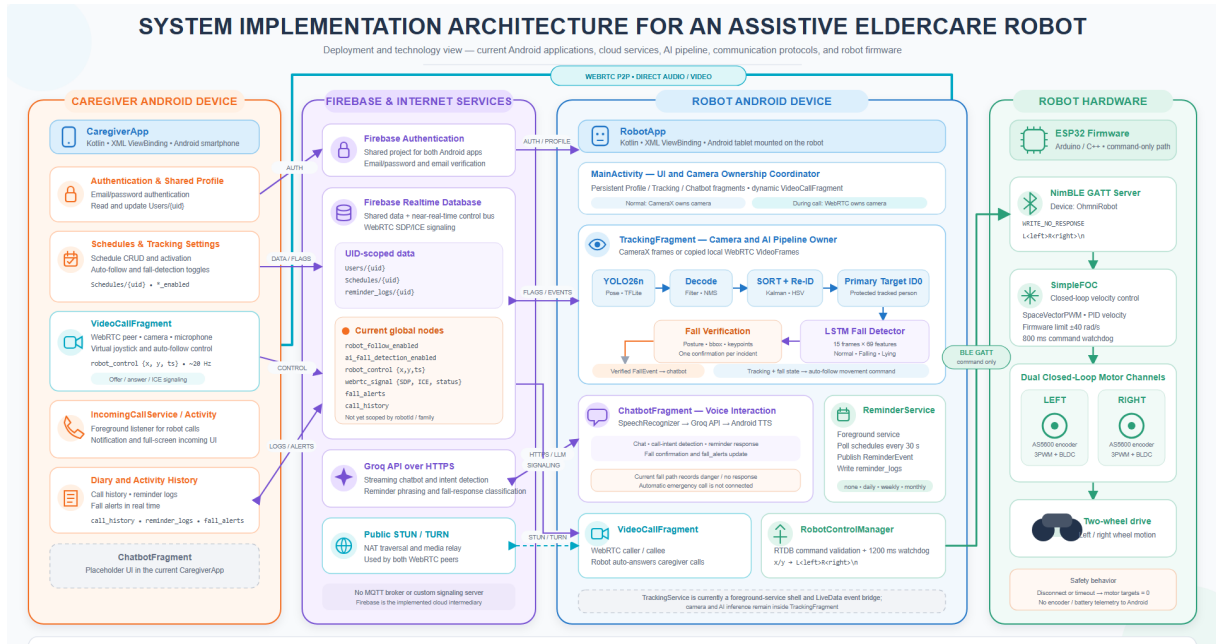
Chương 4 – Hiện thực hệ thống

Trên cơ sở kiến trúc và các quyết định thiết kế ở Chương 3, chương này trình bày quá trình hiện thực hệ thống bằng những thành phần phần mềm và phần cứng cụ thể. Nội dung lần lượt mô tả kiến trúc triển khai, RobotApp, CaregiverApp, quy trình chuẩn bị mô hình AI, firmware của phần thân robot và cách các thành phần phối hợp với nhau trong quá trình vận hành.

4.1 Kiến trúc hiện thực và ánh xạ công nghệ

Hình 4.1 là kết quả tổng hợp của quá trình thiết kế ở Chương 3. Miền người chăm sóc được hiện thực bằng CaregiverApp; miền dịch vụ dùng chung sử dụng Firebase Authentication, Firebase Realtime Database, dịch vụ LLM và hạ tầng STUN/TURN; miền trí tuệ tại robot được hiện thực bằng RobotApp; tầng chấp hành gồm ESP32, BLE GATT, SimpleFOC, encoder, driver và hai động cơ BLDC.

Trong kiến trúc hiện thực, Firebase RTDB lưu trữ cây dữ liệu JSON và đồng bộ cờ chức năng, lệnh điều khiển cùng dữ liệu thiết lập phiên WebRTC theo thời gian gần thực. Âm thanh và hình ảnh không đi qua Firebase mà được truyền trực tiếp bằng WebRTC. Dịch vụ Groq chỉ tham gia xử lý ngôn ngữ, không can thiệp vào vòng điều khiển động cơ. CaregiverApp cũng không kết nối BLE trực tiếp; RobotApp là cầu nối duy nhất chuyển lệnh từ RTDB hoặc chức năng tự bám theo đến ESP32. Cách tổ chức này tuân theo nguyên tắc phân trách nhiệm đã xác định: mô hình AI cung cấp kết quả nhận thức, RobotApp xử lý quyết định nghiệp vụ, còn firmware thực hiện lệnh và đảm bảo khả năng dừng ở tầng phần cứng.



Hình 4.1: Kiến trúc hiện thực của hệ thống

4.2 Môi trường phát triển và tổ chức các thành phần

Hệ thống gồm bốn thành phần được phát triển độc lập nhưng tuân theo các hợp đồng dữ liệu chung. RobotApp và CaregiverApp được xây dựng bằng Kotlin, giao diện XML và ViewBinding. RobotApp sử dụng package `com.example.datn_v1`, còn CaregiverApp sử dụng package `com.example.datn_caregiver`; cả hai cùng đặt `minSdk=23`, `targetSdk=36` và Java/Kotlin JVM target 11. Firmware được xây dựng dưới dạng chương trình Arduino cho ESP32. Thành phần AI gồm mô hình sau huấn luyện, mô hình ONNX trung gian, mô hình TFLite và các chương trình chuyển đổi, kiểm tra suy luận.

Bảng 4.1: Các thành phần hiện thực của hệ thống

Khối	Công nghệ chính	Vai trò
RobotApp	Android/Kotlin, CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, OkHttp	Bộ não trên robot, AI, hội thoại, cuộc gọi và cầu nối phần thân
CaregiverApp	Android/Kotlin, Firebase Auth/RTDB, WebRTC	Ứng dụng cho người thân quản lý, giám sát và điều khiển
AI model	Python, Keras/TensorFlow, Ultralytics, ONNX, onnx2tf, TFLite	Huấn luyện/chuẩn bị và chuyển đổi mô hình cho Android
Firmware	ESP32, NimBLE, SimpleFOC, AS5600, 3PWM	GATT server, parser lệnh và điều khiển hai BLDC

Các thư viện RobotApp gồm CameraX, TensorFlow Lite CPU/GPU/Support/Select TF Ops, OpenCV Android, Firebase Auth/Database, WebRTC native, OkHttp và Kotlin Coroutines. CaregiverApp dùng Firebase Auth/Database, Android Credentials, RecyclerView và WebRTC. Mô hình TFLite được đánh dấu `noCompress` để Android đóng gói nguyên dạng, thuận lợi cho memory mapping.

4.3 Hiện thực RobotApp

4.3.1 Khởi động, điều hướng và vòng đời dịch vụ

LoginActivity là launcher của RobotApp. Sau xác thực, MainActivity quản lý các fragment hồ sơ, theo dõi, chatbot và cuộc gọi. Activity cũng lắng nghe `webrtc_signal` để chuyển giao diện khi có lời mời gọi. Các foreground service gồm ReminderService và TrackingService. AndroidManifest khai báo camera, microphone, Internet, notification, foreground service và quyền BLE tương thích nhiều phiên bản Android.

Quyền Bluetooth được khai báo theo từng nhóm phiên bản hệ điều hành. Android 11 trở xuống sử dụng `BLUETOOTH`, `BLUETOOTH_ADMIN` và quyền vị trí để quét thiết bị; từ Android 12, ứng dụng sử dụng `BLUETOOTH_SCAN` và `BLUETOOTH_CONNECT` theo hướng dẫn của Android [76]. Camera được khai báo `required=false` để không ngăn cài đặt chỉ vì thiết bị thiếu cờ tính năng tương ứng, nhưng các chức năng thị giác chỉ hoạt động khi có camera.

4.3.2 TrackingFragment và quyền sở hữu camera

`tracking_fragment.kt` điều phối CameraX, mô hình pose, bộ theo dõi, LSTM, khối hậu kiểm và bộ điều khiển bám người. CameraX cung cấp hai trường hợp sử dụng `Preview` và `ImageAnalysis`; bộ phân tích chuyển `ImageProxy` sang bitmap và tensor, sau đó giải phóng khung hình sau khi lấy đủ dữ liệu. Các trường hợp sử dụng camera được gắn với vòng đời của fragment theo cơ chế do CameraX cung cấp [75].

Khi không có cuộc gọi, CameraX được gắn với vòng đời của fragment. Trước khi WebRTC khởi động, CameraX được tháo liên kết để tránh hai thành phần cùng mở camera. `video_call_fragment` tiếp quản camera và gửi một bản sao `VideoFrame` cục bộ về khối theo dõi. Khung hình được chuyển từ I420 sang RGB, giảm kích thước và xử lý trên một executor riêng; nếu lượt suy luận trước chưa kết thúc, khung hình mới bị bỏ qua thay vì tiếp tục xếp hàng. Khi cuộc gọi kết thúc, bộ thu hình của WebRTC được

dừng và giải phóng trước khi CameraX được liên kết trở lại.

Thiết kế chỉ tạo một `RobotControlManager` trong `tracking fragment`. Video call không tạo GATT client thứ hai. Điều này loại bỏ tranh chấp kết nối với cùng ESP32 và giữ một điểm duy nhất chịu trách nhiệm dừng.

4.3.3 Nạp và suy luận mô hình YOLO26n-Pose

Các quy ước về hộp bao, keypoint và hậu xử lý ở Mục 2.3.5 và Mục 2.3.4 được hiện thực trong `tracking_fragment.kt`. Chuỗi xử lý bao gồm lựa chọn mô hình, khởi tạo bộ suy luận, tiền xử lý khung hình, giải mã tensor và chuyển kết quả đến SORT, LSTM cùng lớp hiển thị.

Mô hình triển khai và lựa chọn bộ tăng tốc. RobotApp đóng gói hai biến thể mô hình trong thư mục `assets`. Biến thể đầu ra thô có kích thước tensor tĩnh được ưu tiên để khai thác GPU; biến thể đầu cuối được sử dụng khi mô hình thứ nhất không thể khởi tạo. Bảng 4.2 trình bày quy ước đầu ra mà bộ giải mã hỗ trợ.

Bảng 4.2: Hai mô hình YOLO26n-Pose được đóng gói trong RobotApp

Tập mô hình	Kích thước	Đầu ra	Vai trò
yolo26n-pose_320_gpu.tflite	11.838.836 byte	[1, 56, 2100]	Đầu raw, tensor tĩnh; được chọn trước để khai thác GPU
yolo26n-pose_320.tflite	12.031.560 byte	[1, 300, 57]	Đầu ra E2E; mô hình dự phòng với số lượng kết quả giới hạn

Sau khi chọn tập, `setupAIModel()` lần lượt thử bộ tăng tốc GPU, NNAPI và cuối cùng là CPU với bốn luồng. Nếu GPU không khởi tạo được, tài nguyên của bộ tăng tốc này được giải phóng trước khi chuyển sang NNAPI. Thay vì cố định kích thước theo tên tập, chương trình đọc `getInputTensor(0).shape()` và `getOutputTensor(0).shape()`, sau đó cập nhật kích thước đầu vào và cấp phát bộ đệm đầu ra theo hình dạng tensor thực tế. Nhờ vậy, cùng một chuỗi xử lý có thể tiếp nhận các mô hình được xuất theo những quy ước khác nhau. TFLite được lựa chọn vì phù hợp với việc triển khai mô hình trên thiết bị di động và thiết bị biên [65].

Tiền xử lý khung hình. CameraX sử dụng `STRATEGY_KEEP_ONLY_LATEST`; bộ khóa suy luận tiếp tục bỏ frame nếu chưa đủ khoảng thời gian cấu hình hoặc lần xử lý trước chưa hoàn tất. Nhờ đó độ trễ không tích lũy thành một hàng đợi khung hình cũ. Khi

WebRTC đang sở hữu camera, bản sao VideoFrame được chuyển sang cùng executor và cùng hàm suy luận, nên hai nguồn ảnh không tạo hai pipeline giải mã khác nhau.

Bitmap được nạp vào TensorImage kiểu float32, resize trực tiếp về 320×320 bằng `ResizeOp.ResizeMethod.BILINEAR`, sau đó `NormalizeOp(0f, 255f)` đưa giá trị RGB từ $[0, 255]$ về $[0, 1]$. Hiện thực không dùng letterbox. Với frame camera $W_c \times H_c$ và $S = 320$, ánh xạ từ pixel camera sang ảnh mô hình là

$$x_i = x_c \frac{S}{W_c}, \quad y_i = y_c \frac{S}{H_c}, \quad (4.1)$$

còn phép đổi ngược là $x_c = x_i W_c / S, y_c = y_i H_c / S$. Hai hệ số theo hai trục được giữ riêng vì frame nguồn không nhất thiết là ảnh vuông.

Nhận dạng và giải mã hợp đồng tensor. Đầu raw tạo dự đoán tại ba mức stride 8, 16 và 32. Với ảnh 320×320 , tổng số vị trí là

$$N = \left(\frac{320}{8}\right)^2 + \left(\frac{320}{16}\right)^2 + \left(\frac{320}{32}\right)^2 = 2100. \quad (4.2)$$

Vì mô hình chỉ nhận diện lớp người, mỗi vị trí dự đoán gồm 4 giá trị hộp, 1 độ tin cậy và 17×3 giá trị keypoint, tổng cộng 56 kênh. Mô hình đầu ra thô trả tensor $[1, 56, 2100]$; hàm `getOutputVal()` xử lý sự khác biệt giữa cách sắp xếp này và dạng $[1, 2100, 56]$, nhờ đó bộ giải mã luôn truy cập theo cặp chỉ số dự đoán–kênh. Bốn kênh đầu biểu diễn (c_x, c_y, w_b, h_b) , kênh 4 là độ tin cậy, còn 51 giá trị từ kênh 5 tạo thành 17 bộ ba (x_j, y_j, p_j) .

Đầu E2E được nhận biết khi hai chiều sau batch là 300 và 57. Mỗi hàng dùng bốn trường đầu cho $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, trường 4 cho score, trường 5 cho lớp và các trường từ 6 cho 17 keypoint. Việc tách hai nhánh là cần thiết: nếu đọc đầu raw như E2E, hộp tâm–kích thước sẽ bị hiểu nhầm thành hai góc và toàn bộ keypoint lệch một kênh.

Hai mô hình có thể trả tọa độ chuẩn hóa hoặc tọa độ pixel trong không gian đầu vào. Hàm `detectCoordSystem()` khảo sát kênh tọa độ ở lần suy luận đầu tiên: nếu giá trị dương lớn nhất không vượt quá 2, tọa độ được xem là đã chuẩn hóa; ngược lại, kết quả được xem là tọa độ pixel của ảnh mô hình. Trường hợp thứ nhất được nhân trực tiếp với W_c, H_c , còn trường hợp thứ hai được nhân với $W_c/320, H_c/320$. Quy tắc nhận biết này đáp ứng hai mô hình được sử dụng trong đề tài; khi bổ sung quy trình xuất mô hình mới, hệ tọa độ nên được khai báo rõ bằng siêu dữ liệu để tránh suy đoán.

Đầu ra thô được đổi từ dạng *cxcywh* sang *xyxy*, sau đó loại các dự đoán có độ tin

cây dưới 0,20. Hàm $nms()$ sắp xếp kết quả theo điểm tin cậy và loại hộp có IoU lớn hơn 0,45. Kết quả E2E cũng đi qua hàm này như một lớp lọc bổ sung, mặc dù mô hình đầu cuối đã thực hiện bước lựa chọn dự đoán. Mỗi keypoint được giữ kèm độ tin cậy và chỉ được đánh dấu **visible** khi $p_j > 0,30$.

Chuyển đổi tọa độ và hiển thị overlay. Trong pipeline tồn tại bốn không gian tọa độ như Bảng 4.3. Cả bounding box và keypoint phải được chuyển đồng nhất; các phép tính tracking và điều khiển không sử dụng trực tiếp tọa độ của View.

Bảng 4.3: Các hệ tọa độ trong pipeline YOLO26n-Pose của RobotApp

Không gian	Miền tọa độ	Vai trò	Dữ liệu sử dụng
Ảnh camera $\mathcal{C}$	$[0, W_c] \times [0, H_c]$ pixel	Khung hình gốc nhận từ CameraX hoặc WebRTC	SORT, LSTM và điều khiển bám người
Ảnh mô hình $\mathcal{I}$	$[0, 320] \times [0, 320]$ pixel	Ảnh vuông sau ResizeOp	Suy luận và đầu ra dạng pixel
Chuẩn hóa $\mathcal{N}$	$[0, 1] \times [0, 1]$	Biểu diễn không phụ thuộc độ phân giải	Đầu ra chuẩn hóa và các tỉ lệ hình học
Màn hình $\mathcal{V}$	$[0, W_v] \times [0, H_v]$ pixel	BoundingBoxOverlay chồng lên preview	Vẽ hộp, skeleton, nhãn và track ID

Camera trước được hiển thị như gương. Sau khi đổi về pixel camera, tọa độ ngang của keypoint được thay bằng $x'_c = W_c - x_c$. Với bounding box, hai cạnh phải đổi đồng thời:

$$x'_{\min} = W_c - x_{\max}, \quad x'_{\max} = W_c - x_{\min}. \quad (4.3)$$

Nếu chỉ phản chiếu từng cạnh mà không hoán đổi, kết quả sẽ có cạnh phải nhỏ hơn cạnh trái. Tọa độ y không thay đổi.

BoundingBoxOverlay.onDraw() tiếp tục ánh xạ pixel camera sang View theo center-crop giống **PreviewView**. Với

$$s_v = \max\left(\frac{W_v}{W_c}, \frac{H_v}{H_c}\right), \quad \Delta_x = \frac{W_v - s_v W_c}{2}, \quad \Delta_y = \frac{H_v - s_v H_c}{2}, \quad (4.4)$$

một điểm được vẽ tại $x_v = s_v x'_c + \Delta_x$, $y_v = s_v y_c + \Delta_y$. Phần bù có thể âm vì center-crop cắt bớt một chiều. Tọa độ View chỉ phục vụ hiển thị; các đại lượng như tâm mục tiêu, mức chiếm khung và khoảng cách keypoint vẫn được tính trong không gian camera rồi chuẩn hóa bởi W_c, H_c .

Bàn giao kết quả cho các module sau. Sau khi giải mã, mỗi đối tượng được biểu diễn bởi `bbox`, `score`, `classId` và 17 keypoint trước khi chuyển đến `SortTracker`. Kết quả theo dõi được dùng để xác định mục tiêu ID0, tạo vector 69 đặc trưng cho LSTM, sinh lệnh tự bám theo và vẽ lớp thông tin lên màn hình. Các module phía sau chỉ làm việc với một biểu diễn thống nhất trong hệ tọa độ camera, không phụ thuộc mô hình ban đầu sử dụng đầu ra thô hay E2E, sắp xếp kênh trước hay sau, hoặc trả tọa độ chuẩn hóa hay tọa độ pixel.

4.3.4 SORT, Kalman, Hungarian và Re-ID

Các lớp `SortTracker`, `KalmanBoxTracker`, `HungarianAlgorithm` và `ReIdModule` hiện thực phương pháp theo dõi dựa trên kết quả phát hiện. Mỗi đối tượng gồm `bbox`, `score`, `classId` và các keypoint. Bộ lọc Kalman dự báo vùng bao ở khung hình kế tiếp, còn thuật toán Hungarian tìm phương án ghép có tổng chi phí nhỏ nhất. Phương pháp này kế thừa cấu trúc gọn của SORT [39], đồng thời bổ sung tín hiệu tư thế và đặc trưng ngoại hình để phù hợp hơn với bài toán trên điện thoại.

Hàm chi phí sử dụng 30% sai khác IoU, 30% khoảng cách tâm, 30% khoảng cách pose và 10% thay đổi diện tích. Tracker được cấu hình với ngưỡng IoU cơ sở 0,25; track mới cần hai hit để xác nhận. Track ID0 được bảo vệ và giữ tối đa 45 frame không khớp, còn track nhiễu giữ 10 frame.

Khởi tái định danh trích histogram HSV từ vùng thân người, là khu vực ít chịu ảnh hưởng của hậu cảnh và thay đổi góc mặt. Histogram không được cập nhật khi thiếu keypoint tin cậy vì vùng cắt sai có thể khiến hệ thống ghi nhận đặc trưng của người khác. Khi một phát hiện mới phù hợp với hồ sơ cũ, `restoreWithId` khôi phục định danh ban đầu. Đây là giải pháp Re-ID nhẹ, không sử dụng embedding sâu như Deep SORT [40]; ưu điểm là chi phí tính toán thấp, nhưng kết quả có thể bị ảnh hưởng khi nhiều người mặc trang phục giống nhau hoặc điều kiện chiếu sáng thay đổi.

4.3.5 Khóa mục tiêu ID0 và sinh lệnh auto-follow

Đối tượng được chăm sóc được chuẩn hóa thành `PRIMARY_TRACK_ID=0`. Ứng viên đầu tiên được xếp hạng theo diện tích vùng bao, confidence, tỷ lệ keypoint nhìn thấy, độ gần tâm ảnh và mức cắt biên. Sau đó ID0 được dùng cho cả pipeline LSTM và điều khiển. Khi mất mục tiêu, logic chỉ tìm lại trong 2 s; ứng viên phải cách vị trí cuối không quá 35% đường chéo ảnh, thuộc vùng trung tâm và ổn định ba frame. Trong khi sự cố té ngã còn hoạt động, robot không chọn một track khác thay ID0.

Auto-follow so sánh tâm vùng bao với vùng mục tiêu. Nếu lệch ngang, hệ thống phát lệnh quay; nếu kích thước người cho thấy khoảng cách xa, phát lệnh tiến. Các mức quay hiện dùng kênh $y = \pm 0,50$ theo hệ tọa độ camera. Cách đặt dấu khác trục giác joystick nhưng đã được tách rõ theo từng hệ tọa độ. Lệnh cuối vẫn đi qua RobotControlManager, nhờ đó cũng được kiểm tra và format.

4.3.6 LSTM phát hiện té ngã

LstmFallDetector nạp best_model_lstm_v2.tflite khoảng 63,9 KB, dùng hai luồng CPU. Mỗi track có FallDetectionQueue; queue đủ 15 frame mới tạo tensor $1 \times 15 \times 69$ kiểu float. Output 1×2 được diễn giải là xác suất normal/fall. Model gốc Keras khoảng 193,8 KB, cho thấy TFLite giảm đáng kể kích thước đóng gói.

Kết quả được gán nhãn FALL khi xác suất FALL lớn hơn xác suất NO FALL. Dự đoán sau đó được chuyển đến FallVerificationEngine; chỉ khi có năm khung hình FALL liên tiếp, hệ thống mới hình thành một chuỗi ứng viên. Nếu bộ theo dõi khôi phục thành công ID0, hàng đợi đặc trưng vẫn được gán với đúng người mục tiêu. LSTM được lựa chọn nhờ khả năng mô hình hóa quan hệ theo thời gian [38], nhưng kết quả của mô hình luôn phải qua bước hậu kiểm trước khi tạo cảnh báo.

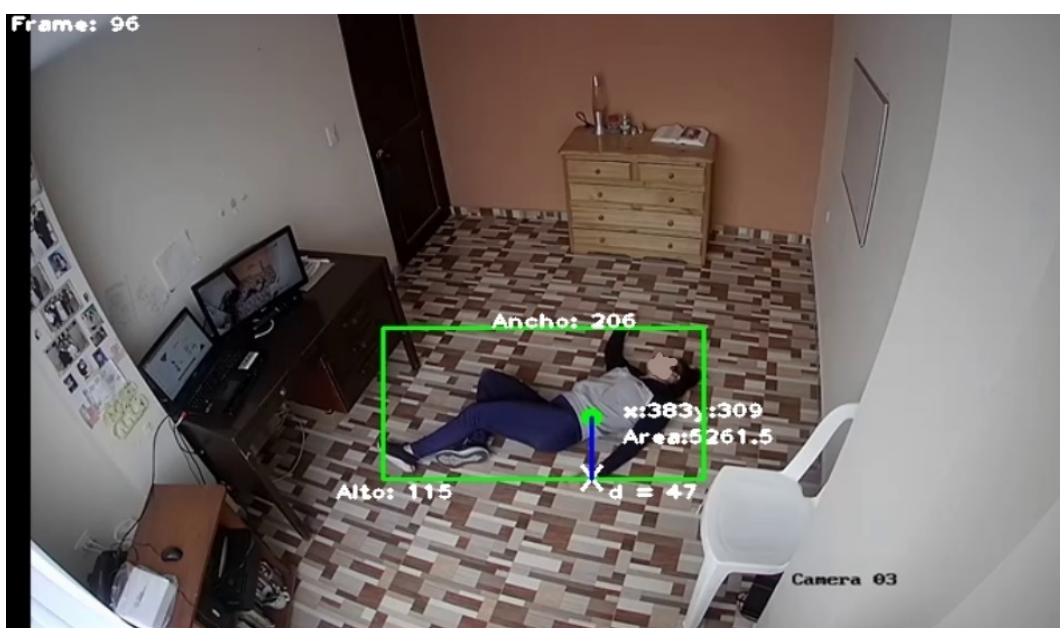
4.3.7 Hậu kiểm té ngã

FallVerificationEngine nhận FrameEvidence gồm trạng thái quan sát mục tiêu, tỷ lệ y của thân trên, số keypoint, tình trạng vùng bao và tỷ lệ hình dạng. Bộ hậu kiểm sử dụng trạng thái mục tiêu cùng vị trí mũi và vai: ba khung hình cho thấy thân trên vẫn ở vùng phía trên ảnh sẽ loại ứng viên, trong khi ba khung hình ở dưới ngưỡng hoặc 12 khung hình liên tiếp thiếu dữ liệu sẽ kích hoạt bước hỏi xác nhận.

Engine trả một trong ba loại: chưa quyết định, Rejected(reason) hoặc AskForConfirmation(reason, score, evidence). Tách quyết định khỏi UI giúp unit test từng nhánh mà không cần camera. isIncidentActive và canAcquireNewTarget ngăn một sự cố tạo nhiều yêu cầu đồng thời. resolveAsSafe, resolveAsDanger và reset làm rõ vòng đời sự cố.

4.3.8 Luồng xác nhận và cảnh báo

Trong mỗi sự cố, TrackingService chỉ chuyển một FallEvent đến chatbot. Robot dừng chuyển động, phát câu hỏi bằng TTS và lắng nghe câu trả lời trong tối đa 10 s. Những cụm từ thể hiện rõ trạng thái an toàn hoặc nguy hiểm được so khớp tại thiết bị để



Hình 4.2: Ví dụ khung hình té ngã trong dữ liệu tham khảo

giảm độ trễ và hạn chế phụ thuộc vào LLM. Với câu trả lời không rõ nghĩa, hệ thống gọi mô hình phân loại theo chế độ không truyền luồng và yêu cầu kết quả có cấu trúc. Nếu người dùng xác nhận an toàn, sự cố được kết thúc tại robot; các trường hợp nguy hiểm, lỗi phân loại hoặc không có phản hồi sẽ được ghi vào `fall_alerts`, sau đó cuộc gọi đến người chăm sóc được thực hiện khi TTS phát xong.

Bản ghi cảnh báo chứa timestamp, trạng thái, phản hồi và bằng chứng như `verificationReason`, `visibleKeypoints`, `bboxFullyInside`, `upperBodyYRatio`. Thông tin này giúp truy vết vì sao engine yêu cầu xác nhận, nhưng không phải là giải thích đầy đủ của mô hình LSTM.

4.3.9 Chatbot giọng nói

`chatbot_fragment` phối hợp ba lớp quản lý. `SpeechRecognizerManager` nhận dạng tiếng nói; `GroqChatService` xử lý hội thoại; còn `AndroidTtsService` tổng hợp tiếng nói. Giao diện chính tôi giản thành trạng thái, icon lớn, tên robot và câu đang nghe/nói. Hồ sơ người dùng được đọc từ `Users/{uid}` để xây prompt về cách xưng hô, tên caregiver, tình trạng và sở thích.

`GroqChatService` gọi điểm cuối `/openai/v1/chat/completions` qua `OkHttp`. Hội thoại dạng truyền luồng sử dụng mô hình `qwen/qwen3.6-27b`, trong khi các tác vụ phân loại ngắn sử dụng `openai/gpt-oss-20b`. Lịch sử được giới hạn ở 20 thông điệp để kiểm soát độ dài ngữ cảnh. Với phản hồi SSE có tiền tố `data:`, dịch vụ

đọc nội dung tại `choices[0].delta.content`, phát từng phần qua Kotlin Flow và lưu câu trả lời hoàn chỉnh khi luồng kết thúc.

Các tác vụ liên quan đến an toàn không phụ thuộc hoàn toàn vào văn bản do LLM sinh ra. Hàm phân loại phản hồi té ngã kiểm tra trước tập từ khóa an toàn và nguy hiểm; API chỉ được gọi khi câu trả lời không rõ nghĩa, sau đó kết quả JSON được phân tích theo cấu trúc định trước. Hàm `detectCallIntent` áp dụng cách tương tự khi kết hợp từ khóa, tên hoặc quan hệ của người chăm sóc và mô hình phân loại dự phòng. Khóa API được truyền vào lớp dịch vụ và cần được cung cấp từ cấu hình bên ngoài kho mã hoặc thông qua máy chủ trung gian.

4.3.10 Lịch nhắc

`ReminderService` lắng nghe `Schedules/{uid}` và hồ sơ `Users/{uid}`. Khi tới thời điểm, service tạo log dưới `reminder_logs/{uid}`, truyền sự kiện đến chatbot và phát lời nhắc. Người cao tuổi có thể xác nhận, từ chối hoặc không phản hồi; log cập nhật status, câu trả lời, retry count và confirmed time. Cấu trúc này tách định nghĩa lịch khỏi lần thực hiện, cho phép caregiver xem lịch sử mà không làm thay đổi lịch gốc.

4.3.11 WebRTC trong RobotApp

`video_call_fragment` khởi tạo `PeerConnectionFactory`, nguồn audio/video, renderer và Firebase reference `webrtc_signal`. Khi `RobotApp` gọi, nó ghi offer và đẩy ICE vào `robotCandidates`; khi nhận gọi, nó đọc offer, tạo answer và nghe `caregiverCandidates`. Listener chỉ được dùng cho đúng pha và được gỡ trong `cleanup`.

WebRTC cung cấp mã hóa và truyền media; Firebase chỉ trao đổi SDP, ICE và trạng thái [68]. Trường `emergencyReason=fail` phân biệt cuộc gọi khẩn cấp với cuộc gọi thường. Khi kết thúc, status được đặt `ended`, `track/capturer/renderer/PeerConnection` được giải phóng và quyền camera trả lại tracking.

4.3.12 RobotControlManager và BLE

`RobotControlManager` lắng nghe nhánh `robot_control`. Hàm ánh xạ lệnh được tách riêng để kiểm thử bằng bốn điểm chuẩn. Mỗi giá trị đầu vào được kiểm tra tính hữu hạn, giới hạn biên, độ tuổi và độ lệch thời gian về tương lai. Chuỗi lệnh được định dạng với `Locale.US` để dấu phân cách thập phân luôn là dấu chấm:

```

1 left = clamp((steering * 0.15 + translation) * 20, -20, 20)
2 right = clamp((steering * 0.15 - translation) * 20, -20, 20)
3 payload = "L%.2fR%.2f\n".format(Locale.US, left, right)

```

Listing 2: Định dạng lệnh BLE ở mức logic

Manager quét theo tên OhmniRobot và custom UUID, timeout quét 10 s, reconnect sau 3 s. Sau discovery, characteristic được cấu hình WRITE_TYPE_NO_RESPONSE. API ghi khác nhau theo phiên bản Android được xử lý tương thích. Watchdog 1.200 ms gọi STOP nếu luồng RTDB không cập nhật. `stop()` gỡ listener, callback, scan và GATT để tránh leak.

4.4 Hiện thực CaregiverApp

4.4.1 Khởi động, xác thực và hồ sơ

CaregiverApp sử dụng LoginActivity và SignUpActivity cho quá trình xác thực, MainActivity làm màn hình khởi động chính, còn ProfileFragment phục vụ quản lý hồ sơ. Firebase Authentication cung cấp UID; hồ sơ được lưu tại `Users/{uid}` để RobotApp cùng truy cập. MainActivity đồng thời khởi động IncomingCallService nhằm tiếp nhận cuộc gọi khi ứng dụng chạy nền. Do Activity này được khai báo là launcher, bước kiểm tra phiên xác thực ngay khi khởi động cần được hoàn thiện trước khi triển khai cho người dùng thực tế.

4.4.2 Bật/tắt chức năng theo dõi

tracking_fragment ghi hai cờ robot_follow_enabled và ai_fall_detection_enabled. Fragment đồng thời theo dõi `.info/connected` để phản ánh kết nối RTDB. Khi tắt auto-follow, CaregiverApp ghi $x = 0, y = 0, ts = now$; đây là hành vi quan trọng vì chỉ đối cờ có thể để lại lệnh chuyển động trước đó.

4.4.3 Lịch và nhật ký

ScheduleFragment đọc danh sách Schedules/uid; AddScheduleFragment tạo/cập nhật lịch; ScheduleAdapter xử lý RecyclerView và công tắc active. Tên trường được giữ đúng là active, tránh sai lệch với isActive. Ba nhánh nhật ký có fragment/adaptor riêng: reminder, fall alert và call history. Danh sách cảnh báo/cuộc gọi được truy vấn theo timestamp trong bảy ngày gần nhất.

4.4.4 Cuộc gọi đến nền

IncomingCallService là foreground service lắng nghe `webrtc_signal`. Khi status là calling và callerRole là robot, service tạo notification và có thể mở IncomingCallActivity toàn màn hình. Nếu `emergencyReason=fall`, tiêu đề/nội dung thông báo nêu đây là cuộc gọi do chưa xác nhận an toàn. Manifest khai báo notification, foreground service và full-screen intent; hành vi thực tế còn phụ thuộc chính sách hệ điều hành và quyền người dùng.

4.4.5 Video call và joystick

`video_call_fragment` hiện thực cả hai vai trò caller/callee, renderer cục bộ/từ xa và signaling tương ứng. Trong màn hình gọi, joystick tính $x = \Delta y/r$, $y = -\Delta x/r$, chặn $[-1, 1]$ và throttle khoảng 20 Hz. Khi người dùng bắt đầu lái, app tắt auto-follow để hai nguồn không tranh chấp. ACTION_UP/ACTION_CANCEL, kết thúc gọi và rời fragment đều gửi STOP.

CaregiverApp không có BLE và không biết dấu vật lý của motor. Ranh giới này giúp điện thoại caregiver hoạt động ở bất kỳ khoảng cách Internet nào; RobotApp gần phần thân chịu trách nhiệm kiểm tra lệnh và hiệu chỉnh motor.

4.5 Hiện thực dữ liệu dùng chung

Các bộ lắng nghe Firebase cung cấp bản cập nhật gần thời gian thực, trong khi thao tác ghi có thể được phản ánh cục bộ trước khi đồng bộ với máy chủ [66]. Vì vậy, mỗi lệnh điều khiển bắt buộc phải có timestamp; callback được gọi theo thời gian thực không có nghĩa dữ liệu nhận được luôn còn hiệu lực. Các lớp UserProfile, ScheduleItem, FallAlertEntry, ReminderLogEntry và CallHistoryEntry được dùng để ánh xạ dữ liệu JSON sang đối tượng trong ứng dụng.

Hệ thống sử dụng kết hợp các nhánh theo UID và một số nhánh dữ liệu dùng chung. Cấu trúc này đáp ứng mô hình một robot–một người chăm sóc của đề tài nhưng chưa cách ly đầy đủ dữ liệu của nhiều gia đình. Khi mở rộng, quy tắc truy cập cần áp dụng nguyên tắc từ chối mặc định, phân quyền theo UID, gia đình và robot, đồng thời kiểm tra kiểu và miền giá trị của từng trường [67]. Mọi thay đổi cấu trúc dữ liệu cũng phải được cập nhật đồng thời ở hai ứng dụng để tránh tình trạng bộ lắng nghe truy cập sai đường dẫn.

4.6 Huấn luyện và chuyển đổi mô hình AI

Mô hình phát hiện té ngã được chuẩn bị theo một quy trình liên tục từ xử lý dữ liệu video, trích xuất đặc trưng tư thể, huấn luyện LSTM đến chuyển đổi sang định dạng phù hợp với Android. Việc thống nhất quy ước đặc trưng giữa giai đoạn huấn luyện và suy luận có ý nghĩa quyết định, bởi chỉ một sai khác về thứ tự hoặc cách chuẩn hóa tọa độ cũng làm đầu vào trên thiết bị không còn tương thích với mô hình.

4.6.1 Xử lý dữ liệu

Dữ liệu LE2I được cung cấp dưới dạng các video liên tục gồm hoạt động bình thường và tình huống té ngã. Quá trình tiền xử lý gồm:

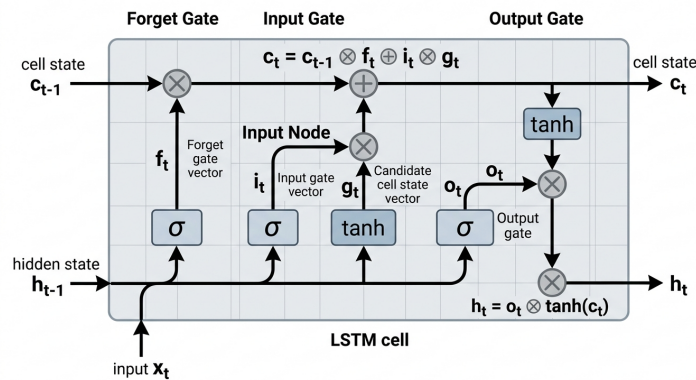
- **Trích xuất khung hình:** Video được phân tách thành các khung hình ở tốc độ 25 FPS để tương thích với tốc độ xử lý mục tiêu của camera robot.
- **Phân chia tập dữ liệu:** Tập huấn luyện, xác thực và kiểm tra được chia theo video và chủ thể, không chia ngẫu nhiên theo từng khung hình. Cách tổ chức này hạn chế rò rỉ dữ liệu khi những khung hình gần nhau của cùng một hành động xuất hiện đồng thời ở tập huấn luyện và tập kiểm tra.
- **Xử lý mất cân bằng lớp:** Số mẫu hoạt động bình thường thường lớn hơn số mẫu té ngã. Trọng số lớp và phương pháp lấy mẫu tăng cường được sử dụng để giảm xu hướng mô hình nghiêng về lớp NO FALL.

4.6.2 Trích xuất đặc trưng

Mô hình LSTM không nhận trực tiếp ảnh RGB mà học từ chuỗi đặc trưng tư thể do YOLO-Pose cung cấp. Mỗi khung hình của người mục tiêu được biểu diễn bởi 69 giá trị theo đúng quy ước sử dụng trong RobotApp:

1. **Tọa độ chuẩn hóa:** Tọa độ (x_i, y_i) của 17 keypoint được chuẩn hóa theo tâm hông và kích thước của vùng tư thể, giúp giảm ảnh hưởng của vị trí và khoảng cách giữa người với camera.
2. **Đặc trưng chuyển động:** Với mỗi keypoint, hai thành phần $(\Delta x_i, \Delta y_i)$ biểu diễn độ dịch chuyển so với khung hình trước. Mỗi điểm vì vậy đóng góp bốn giá trị $(x_i, y_i, \Delta x_i, \Delta y_i)$.

3. **Tỷ lệ hình dạng và cửa sổ thời gian:** Tỷ lệ rộng/cao của hộp bao được ghép với 68 giá trị keypoint để tạo vector 69 chiều. Mười lăm khung hình liên tiếp hình thành tensor đầu vào có kích thước $[1, 15, 69]$ cho LSTM.



Hình 4.3: Minh họa kiến trúc mạng LSTM phân tích chuỗi thời gian của các đặc trưng (Nguồn: Tham khảo)

4.6.3 Huấn luyện mô hình

Mô hình LSTM được xây dựng bằng TensorFlow/Keras. Quá trình huấn luyện gồm:

- Thiết lập seed cố định để tăng khả năng tái lập thí nghiệm.
- Sử dụng hàm mất mát Binary Cross-Entropy cho bài toán phân loại nhị phân.
- Áp dụng dừng sớm khi hàm mất mát trên tập xác thực không còn cải thiện, đồng thời lưu mô hình tốt nhất tại `best_model_lstm.h5`.

4.6.4 Chuyển đổi và tối ưu mô hình

Để triển khai trên điện thoại Android, mô hình sau huấn luyện được chuyển sang TensorFlow Lite nhằm giảm kích thước và tận dụng môi trường suy luận dành cho thiết bị biên.

1. Đối với mô hình LSTM:

Chương trình `convert_to_tflite.py` nạp `best_model_lstm.h5` và kiểm tra `input_shape` trước khi chuyển đổi bằng `TFLiteConverter.from_keras_model`. Tập toán tử gồm TFLite built-ins và Select TF Ops. Kết quả là `best_model_lstm_v2.tflite`, có thể đóng gói trực tiếp trong thư mục `assets` của RobotApp.

2. Đối với mô hình YOLO-Pose:

Mô hình PyTorch `yolo26n-pose.pt` trước hết được xuất sang ONNX với kích thước lô cố định bằng 1 và ảnh đầu vào 320×320 . Thư viện `onnx2tf` tiếp tục chuyển mô hình ONNX sang SavedModel và TFLite. Biên thể FP16 lưu trọng số bằng số dấu phẩy động 16 bit, qua đó giảm dung lượng và băng thông bộ nhớ so với FP32; mức ảnh hưởng đến chất lượng cần được xác nhận bằng phép kiểm tra tương đương sau chuyển đổi.

Bảng 4.4: Các mô hình chính sau khi huấn luyện và chuyển đổi

Tập mô hình	Định dạng	Dung lượng	Mục đích
best_model_lstm.h5	Keras H5	193,8 KB	Mô hình gốc để huấn luyện và đánh giá
best_model_lstm_v2.tflite	TFLite	63,9 KB	Mô hình LSTM đã tối ưu cho Android
yolo26n-pose.pt	PyTorch	7,88 MB	Mô hình Pose gốc
yolo26n-pose_320_fp16.tflite	TFLite FP16	6,28 MB	Biên thể YOLO đã lượng tử hóa FP16, tối ưu kích thước và tốc độ

4.6.5 Kiểm tra tương đương sau chuyển đổi

Việc nạp thành công mô hình trên Android chưa đủ để khẳng định quá trình chuyển đổi chính xác. Cùng một tập dữ liệu kiểm tra cần được suy luận bằng mô hình gốc và mô hình TFLite, sau đó so sánh hộp bao, độ tin cậy, keypoint và vector xác suất của LSTM. Bước kiểm tra này giúp phát hiện sai khác về toán tử, thứ tự đầu ra hoặc cách chuẩn hóa trước khi đánh giá precision và recall trên thiết bị.

4.7 Hiện thực firmware phần thân robot

4.7.1 Khởi tạo phần cứng

Firmware dùng ESP32 với hai AS5600 trên hai bus I2C. Motor trái dùng SDA=19/SCL=18 và motor phải dùng SDA=23/SCL=5. Driver trái dùng IN1=32, IN2=33, IN3=25, EN=22; driver phải dùng IN1=26, IN2=27, IN3=14, EN=12. Hai bus riêng tránh xung đột địa chỉ mặc định giống nhau của AS5600.

Hai BLDCMotor có 7 pole pairs, liên kết encoder và driver 3PWM. Điều chế SpaceVectorPWM, controller velocity, PID và LPF được thiết lập trước `init/initFOC`. Firmware gọi `loopFOC` liên tục; không đặt delay trong loop điều khiển.

4.7.2 BLE GATT server

NimBLE khởi tạo tên OhmniRobot, công suất phát cao và advertising có service UUID trong scan response. Characteristic chỉ có WRITE_NR. Callback kết nối bật cờ và tiếp tục advertising; callback ngắt đặt target hai bánh về 0. Callback write lấy `std::string`, trim và gọi parser.

Bộ phân tích tìm vị trí hai ký tự L và R, tách chuỗi con rồi chuyển sang số bằng `toFloat`. Khi dữ liệu hợp lệ, vận tốc đặt và `lastCmdTime` được cập nhật. RobotApp đã chuẩn hóa định dạng lệnh trước khi gửi; tuy nhiên, firmware vẫn cần kiểm tra toàn bộ chuỗi và dùng `isfinite` để loại giá trị không hợp lệ ngay tại tầng chấp hành.

4.7.3 Deadzone và watchdog

Hàm `applyDeadzone` đưa $|v| < 0,5$ về 0, nâng độ lớn của lệnh khác 0 nhưng nhỏ hơn 3 lên 3, sau đó giới hạn kết quả trong $[-40, 40]$. Watchdog kiểm tra thời gian nhận lệnh khi BLE đang kết nối; nếu quá 800 ms và vận tốc đặt khác 0, firmware ghi nhật ký và đưa hai bánh về trạng thái dừng. Do `motor.move()` được gọi sau bước kiểm tra watchdog, vận tốc đặt bằng 0 có hiệu lực ngay trong chu kỳ điều khiển đang thực hiện.

```
1 loopFOC(left); loopFOC(right)
2 if connected and now - lastCommand > 800 ms:
3     targetLeft = 0; targetRight = 0
4 leftCommand = applyDeadzone(targetLeft)
5 rightCommand = applyDeadzone(targetRight)
6 move(leftCommand); move(rightCommand)
```

Listing 3: Giả mã vòng điều khiển firmware

4.8 Cấu hình và triển khai

Hai ứng dụng sử dụng cùng một dự án Firebase và mỗi ứng dụng có tệp cấu hình Android tương ứng. Các quy tắc bảo mật RTDB cần được triển khai trước khi lưu dữ liệu người dùng. Máy chủ STUN/TURN phải được cấu hình cho cả hai đầu cuối, trong đó TURN nên sử dụng thông tin xác thực ngắn hạn. Khóa API Groq cần được cung cấp từ cấu hình cục bộ an toàn hoặc máy chủ trung gian, không lưu trực tiếp trong kho mã. Thiết bị robot cần quyền truy cập camera, micro, thông báo và quét/kết nối BLE; thiết bị của người chăm sóc cần quyền camera, micro, thông báo và hiển thị toàn màn hình theo chính sách của hệ điều hành.

Quy trình tích hợp khuyến nghị:

1. Nạp firmware, kiểm tra encoder và motor với bánh nhấc khỏi mặt đất.
2. Cài RobotApp, kiểm tra quyền và kết nối BLE; gửi STOP trước mọi thử nghiệm.
3. Kiểm tra riêng dấu bốn lệnh chuẩn ở tốc độ thấp.
4. Cài CaregiverApp, xác nhận cùng tài khoản/project và đồng bộ profile/ lịch.
5. Kiểm tra cuộc gọi trên cùng mạng, sau đó hai mạng khác nhau/TURN.
6. Bật tracking rồi fall detection; thử kịch bản an toàn trước kịch bản té ngã dừng lại.
7. Kiểm tra mất Internet, mất BLE, khóa màn hình và vòng đời ứng dụng.

4.9 Giới hạn của phiên bản hiện thực

Trong phạm vi đồ án, hệ thống tập trung kiểm chứng các luồng chính trên mô hình một robot–một người chăm sóc. Vì vậy, cấu trúc Firebase chưa hỗ trợ đầy đủ quan hệ nhiều người dùng–nhiều robot; chức năng chatbot trên CaregiverApp mới dừng ở giao diện định hướng, còn hội thoại được triển khai trên RobotApp. Trước khi đưa hệ thống vào sử dụng thực tế, thông tin xác thực phải được tách hoàn toàn khỏi ứng dụng, bộ phân tích lệnh của firmware cần kiểm tra dữ liệu chặt chẽ hơn, và mỗi phiên bản mô hình nên đi kèm tệp kê khai cùng mã băm để tránh nhầm lẫn trong quá trình đóng gói. Đây là những nội dung cần tiếp tục hoàn thiện khi mở rộng hệ thống sau đồ án.

4.10 Kết luận chương

Chương đã trình bày quá trình hiện thực kiến trúc hệ thống thành hai ứng dụng Android, các mô hình AI và firmware ESP32. RobotApp đảm nhiệm cảm nhận, hội thoại, điều phối và kết nối với thân robot; CaregiverApp cung cấp các chức năng quản lý, giám sát và can thiệp từ xa. Firebase, WebRTC và BLE tạo thành ba kênh giao tiếp tương ứng cho dữ liệu dùng chung, truyền thông đa phương tiện và điều khiển phần cứng. Bên cạnh đó, quy trình chuyển đổi mô hình và các cơ chế dừng nhiều tầng giúp hệ thống phù hợp hơn với điều kiện vận hành trên thiết bị di động. Chương 5 tiến hành kiểm thử, đánh giá kết quả và phân tích những giới hạn của hệ thống.

Chương 5 – Kết quả và thảo luận

5.1 Mục tiêu đánh giá

Đánh giá nguyên mẫu nhằm trả lời bốn câu hỏi: các khối có được hiện thực đúng kiến trúc hay không; hợp đồng liên ứng dụng có nhất quán hay không; các cơ chế an toàn có điểm kiểm chứng hay không; và bằng chứng hiện có cho phép kết luận tới mức nào. Do hệ thống gồm AI, cloud, mobile và phần cứng, một con số accuracy đơn lẻ không đại diện cho toàn hệ thống.

Chương này phân biệt ba loại bằng chứng. *Bằng chứng tĩnh* được rút từ mã nguồn, cấu hình và artifact. *Bằng chứng tự động* đến từ unit test/build có thể chạy lặp. *Bằng chứng tích hợp* đến từ thử nghiệm hai điện thoại, Internet, BLE và robot thật. Chỉ các kết quả có artifact hoặc được nhóm dự án xác nhận mới được ghi “đạt”; hạng mục thiếu log định lượng được ghi rõ để tránh tạo số liệu.

5.2 Cấu hình và tiêu chí đánh giá

Bảng 5.1: Cấu hình phần mềm được đánh giá

Hạng mục	Cấu hình
RobotApp	Android/Kotlin; minSdk 23; target/compile SDK 36; CameraX; TFLite; OpenCV; Firebase; WebRTC; BLE
CaregiverApp	Android/Kotlin; minSdk 23; target/compile SDK 36; Firebase; WebRTC
AI trên Android	Pose input 320×320 ; LSTM input 15×69 ; CPU/GPU TFLite artifacts
Firmware	ESP32; NimBLE; SimpleFOC; hai AS5600; hai BLDC; velocity limit 40 rad/s
Cloud	Firebase Authentication/RTDB; Groq HTTPS; STUN/TURN cho WebRTC
Giao thức điều khiển	RTDB x, y, ts ; BLE $L<float>R<float>\backslash n$

Các chỉ số AI thích hợp gồm precision, recall, F1 và confusion matrix ở cấp sự cố. Với TP là phát hiện đúng sự cố, FP là cảnh báo giả, FN là bỏ sót:

$$Precision = \frac{TP}{TP + FP}, \quad (5.1)$$

$$Recall = \frac{TP}{TP + FN}, \quad (5.2)$$

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}. \quad (5.3)$$

Đơn vị đánh giá phải là một sự kiện té ngã, không phải từng frame; nhiều frame cảnh báo liên tiếp của cùng sự cố chỉ tính một sự kiện. Với tracking, chỉ số cần theo dõi là số lần ID0 đổi sai, tỷ lệ phục hồi đúng và thời gian mất mục tiêu. Với điều khiển, chỉ số là độ trễ end-to-end, tần số lệnh, tỷ lệ lệnh bị loại và thời gian dừng sau mất kênh. Với WebRTC, chỉ số gồm thời gian thiết lập, tỷ lệ cuộc gọi thành công, RTT, jitter và packet loss từ getStats.

5.3 Kết quả hiện thực và artifact

5.3.1 Mức độ hoàn thành thành phần

Bảng 5.2: Kết quả hiện thực theo thành phần

Thành phần	Trạng thái	Bằng chứng
Thành phần	Trạng thái	Bằng chứng
RobotApp Android	Đã hiện thực	24 tệp Kotlin lỗi; manifest; resource; model trong assets; APK debug
CaregiverApp Android	Đã hiện thực	24 tệp Kotlin; giao diện quản lý, diary, lịch, call; APK debug
Pose trên thiết bị	Đã tích hợp	Hai model TFLite CPU/GPU và pipeline decode/NMS
Tracking và Re-ID	Đã tích hợp	SORT, Kalman, Hungarian, HSV histogram, ID0
LSTM và hậu kiểm	Đã tích hợp	Model TFLite, queue 15×69, state machine, verification engine
Hội thoại giọng nói	Đã tích hợp RobotApp	SpeechRecognizer, Groq streaming/classify, Android TTS

Lịch nhắc và nhật ký	Đã tích hợp	Schedules, reminder_logs, adapters và service
WebRTC	Đã tích hợp	Caller/callee, SDP/ICE RTDB, incoming foreground service
Điều khiển robot	Đã tích hợp	Joystick, RTDB, RobotControlManager, BLE GATT
Firmware BLDC	Đã hiện thực	Sketch ESP32, NimBLE, Simple-FOC, dual I2C, watchdog
Backend riêng/MQTT	Không thuộc hiện thực	Hệ thống dùng Firebase RTDB; không có broker/server signaling riêng
Chatbot Caregiver-App	Placeholder	Logic hội thoại thật chỉ nằm ở RobotApp

5.3.2 Kết quả đóng gói Android

Nhóm dự án xác nhận cả hai ứng dụng build thành công. Trong cây build có APK debug RobotApp khoảng 504,24 MiB và CaregiverApp khoảng 51,03 MiB. Chênh lệch chủ yếu do RobotApp đóng gói nhiều runtime native và thư viện AI (TFLite, GPU/Select TF Ops, OpenCV, WebRTC) cùng model; CaregiverApp không chứa model pose/LSTM và stack AI.

Kích thước 504 MiB là chấp nhận được cho prototype debug nhưng không phù hợp phát hành. Bản release cần bật shrinking, loại ABI không dùng, kiểm tra trùng native library, cân nhắc Play App Bundle/split APK và bỏ dependency AI dư. Model thực chỉ chiếm khoảng vài chục MB; phần lớn dung lượng tăng đến từ runtime/native binaries và ký hiệu debug.

Bảng 5.3: Artifact đóng gói hiện có

Artifact	Kích thước	Ý nghĩa
RobotApp debug APK	504,24 MiB	Bản debug tích hợp AI/WebRTC/BLE
CaregiverApp debug APK	51,03 MiB	Bản debug quản lý/WebRTC

5.3.3 Kết quả artifact mô hình

Chuỗi chuyển đổi đã tạo đủ artifact H5, PT, ONNX, SavedModel và TFLite. LSTM giảm từ khoảng 193,8 KB xuống 63,9 KB. Biến thể pose FP16 khoảng 6,28

MB, gần một nửa biến thể float32 tương ứng; biến thể app CPU/GPU lần lượt khoảng 12,03/11,84 MB. Đây là kết quả về kích thước tệp, không tự động chứng minh tốc độ hoặc độ chính xác.

Bảng 5.4: Đánh giá định tính các biến thể mô hình

Biến thể	Lợi ích kỳ vọng	Điều phải đo
Float32 CPU	Tương thích rộng, dễ debug	Latency, nhiệt, FPS, bộ nhớ
Graph GPU	Tăng throughput nếu delegate hỗ trợ toàn graph	Tỷ lệ op chạy GPU, overhead copy, fallback
FP16	Giảm dung lượng/băng thông bộ nhớ	Sai số keypoint, mAP/PCK, latency thật
NMS ngoài graph	Tăng tính tương thích delegate	Độ đúng decode/NMS Kotlin so với nguồn

5.4 Kiểm thử tự động ở mức logic

5.4.1 Ánh xạ điều khiển

RobotControlManagerTest định nghĩa bốn test bảo vệ hiệu chỉnh dấu. Sai sót dấu là rủi ro cao vì tên “trái/phải/tiến/lùi” đi qua ba hệ tọa độ: màn hình, camera và chiều lắp motor. Các assertion dùng sai số 0,001.

Bảng 5.5: Bộ unit test ánh xạ điều khiển

Ca kiểm thử	Input (x, y)	Kỳ vọng (L, R)
forward calibrated	$(-1, 0)$	$(-20, 20)$
backward calibrated	$(1, 0)$	$(20, -20)$
positive steering	$(0, 1)$	$(3, 3)$
negative steering	$(0, -1)$	$(-3, -3)$

Bốn test này kiểm tra điểm chuẩn, nhưng chưa bao phủ clipping, NaN/infinity, vùng đi thẳng, deadzone, timestamp và format Locale.US. Các ca đó được đề xuất bổ sung trước khi release.

5.4.2 Hậu kiểm té ngã

FallVerificationEngineTest định nghĩa bốn nhánh quan trọng. Engine trước hết được “arm” bằng năm frame đứng và một raw fall, sau đó đánh giá tình huống.

Các test xác nhận khả năng tách engine khỏi Android UI. Coverage còn thiếu: khóa

Bảng 5.6: Bộ unit test hậu kiểm té ngã

Ca	Kích thích	Kỳ vọng
clipped bbox after fall	Thân thấp, bbox không nằm trọn ảnh	Hỏi ngay
stable upper body top 30%	Ba frame thân trên ở vùng an toàn	Từ chối sự cố
lower occluded horizontal	Thân thấp, 7 keypoint, aspect 1,6 trong ba frame	Hỏi xác nhận
lost target after fall	Mất mục tiêu ba frame	Hỏi xác nhận

một sự cố, phục hồi 10 frame, baseline EMA, ngưỡng biên, reset, safe/danger resolution và chuỗi nhiễu xen kẽ.

5.5 Đánh giá hợp đồng tích hợp bằng kiểm tra tĩnh

5.5.1 Nhất quán Firebase

Đối chiếu hai ứng dụng cho thấy các path quan trọng trùng nhau: robot_control, robot_follow_enabled, ai_fall_detection_enabled, webrtc_signal, fall_alerts, call_history, Users, Schedules và reminder_logs. Offer/answer dùng cùng trường sdp,type. ICE dùng candidate,sdpMid,sdpMLineIndex; hai nhánh candidate được tách theo vai trò.

Một điểm cần chú ý là nhiều path vẫn toàn cục. Với một robot demo, hai app cùng thấy đúng dữ liệu. Với hai gia đình, lệnh/call/cảnh báo có thể trộn nếu rules cho phép. Vì vậy kết quả “nhất quán” chỉ áp dụng schema một robot hiện tại, chưa chứng minh multi-tenant.

5.5.2 Nhất quán BLE

Tên thiết bị, service UUID và characteristic UUID trong RobotControlManager trùng firmware. Android format L trước R, hai số có dấu chấm và newline; firmware trim rồi tách tại L/R. Android giới hạn $[-20, 20]$, hẹp hơn firmware $[-40, 40]$, tạo lớp an toàn bổ sung. Characteristic đều dùng write without response.

5.5.3 Chuỗi timeout

Ba timeout có thứ tự hợp lý: firmware 800 ms, Android resend/watchdog 1.200 ms và độ mới RTDB 2.000 ms. Firmware dừng sớm nhất khi không có gói BLE; Android dừng nếu nguồn RTDB ngừng; timestamp chặn lệnh cache khi reconnect. Tần suất

joystick khoảng 20 Hz tương ứng chu kỳ 50 ms, nhỏ hơn nhiều so với 800 ms, do đó có biên cho jitter bình thường.

5.5.4 Quyền sở hữu tài nguyên

Mã nguồn chỉ định tracking fragment là owner của RobotControlManager. Luồng cuộc gọi chia sẻ local WebRTC frame về AI và unbind CameraX. Đây là kết quả kiến trúc quan trọng vì xung đột camera/GATT thường không xuất hiện trong unit test. Kiểm thử vòng đời trên thiết bị vẫn cần bao phủ gọi đến khi app nền, từ chối, mất mạng, xoay màn hình và kết thúc bất thường.

5.6 Kịch bản kiểm thử chức năng đầu cuối

Bảng 5.7: Danh mục kiểm thử đầu cuối

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E01	Đăng nhập dùng chung	Đăng nhập cùng tài khoản hai app	Cùng UID/hồ sơ, không lộ user khác
E02	Tạo lịch nhắc	Tạo lịch từ caregiver, chờ thời điểm	Robot phát TTS, log xuất hiện
E03	Hội thoại bình thường	Nói câu tiếng Việt khi robot lắng nghe	STT-LLM-TTS hoàn thành
E04	Bật auto-follow	Chỉ một người đứng trước robot	ID0 ổn định, robot bám trong giới hạn
E05	Người thứ hai đi qua	Người nhiều cắt ngang mục tiêu	ID0 không đổi sai
E06	Mất mục tiêu ngắn	ID0 ra khỏi ảnh rồi trở lại	Khôi phục trong điều kiện cho phép
E07	Nằm chủ động	Người từ từ nằm nhưng còn rõ/an toàn	Không cảnh báo sai hoặc xác nhận an toàn
E08	Té ngã dừng lại	Thực hiện với đệm và người giám sát	Robot dừng, hỏi, tạo một alert

Mã	Kịch bản	Thao tác/kích thích	Kết quả mong đợi
E09	Phản hồi an toàn	Trả lời “tôi ổn” rõ ràng	Cập nhật safe, không gọi khẩn cấp
E10	Không phản hồi	Im lặng quá cửa sổ nghe	Cập nhật danger/no-response, gọi caregiver
E11	Video call	Gọi giữa hai mạng khác nhau	Offer/answer/ICE, audio/video kết nối
E12	AI trong cuộc gọi	Bật tracking khi call	Một camera, frame chia sẻ, không crash
E13	Joystick	Kéo bốn hướng rồi thả	Đúng dấu; thả gửi STOP
E14	Mất Internet	Ngắt mạng khi joystick đang kéo	Không nhận lệnh mới; watchdog dừng
E15	Mất BLE	Tắt Bluetooth/ngắt nguồn tablet	Callback/watchdog firmware dừng
E16	Lệnh cũ	Ghi timestamp cũ hơn 2 s	RobotApp bỏ lệnh và dừng
E17	Input lỗi	NaN/vượt miền/thiếu timestamp	Không chuyển động
E18	Cuộc gọi khẩn nên	Caregiver app ở nền, robot gọi fall	Notification/full-screen nêu đúng lý do

Bảng là bộ kiểm thử hồi quy đề xuất. Các kịch bản phần cứng phải bắt đầu với bánh xe nhấc khỏi mặt đất, tốc độ thấp, vùng trống và nút ngắt nguồn sẵn sàng. Không cho người cao tuổi thật thực hiện té ngã; chỉ dùng diễn viên khỏe mạnh, đệm bảo vệ và giám sát.

5.7 Thiết kế phép đo hiệu năng

5.7.1 Độ trễ pipeline AI

Mỗi frame cần gắn timestamp t_0 khi nhận, t_1 sau preprocess, t_2 sau interpreter, t_3 sau decode/NMS, t_4 sau tracking/LSTM và t_5 khi UI/lệnh được cập nhật. Báo cáo median, P90, P95 thay vì chỉ trung bình. FPS hiệu dụng phải tính frame thật được xử lý; drop-frame count phản ánh quá tải.

$$T_{AI} = (t_1 - t_0) + (t_2 - t_1) + (t_3 - t_2) + (t_4 - t_3). \quad (5.4)$$

Nhiệt và pin ảnh hưởng mạnh sau vài phút. Do đó benchmark cần warm-up 30–60 s, chạy ít nhất 10 phút cho mỗi CPU/GPU/FP16, ghi model thiết bị, Android version, delegate, thread count và độ phân giải camera.

5.7.2 Độ trễ điều khiển

Caregiver gắn sequence number và t_c vào lệnh; RobotApp log t_r khi listener nhận và t_b khi ghi BLE; ESP32 log micros/millis khi callback nhận. Nếu đồng hồ thiết bị chưa đồng bộ, đo từng chặng hoặc quay video tốc độ cao; không trừ trực tiếp timestamp khác clock. Độ trễ cần báo cáo cùng packet loss và khoảng cách BLE.

5.7.3 WebRTC

Thu thập RTCStatsReport định kỳ: candidate pair, currentRoundTripTime, packetsLost, jitter, framesEncoded/Decoded, framesDropped và availableOutgoingBitrate. Thử cùng Wi-Fi, Wi-Fi-4G và hai mạng NAT khó; ghi rõ có dùng TURN. Thời gian setup tính từ calling đến connected.

5.7.4 Độ chính xác té ngã

Tập kiểm thử phải độc lập theo chủ thể/video, bao gồm đi, ngồi, nằm, cúi nhặt đồ, khuất người, ra khỏi ảnh, té nhiều hướng và nhiều điều kiện sáng. So sánh ít nhất ba cấu hình: pose/luật một frame; pose+LSTM; pose+LSTM+hậu kiểm. Ablation này đo được đóng góp của mỗi tầng thay vì chỉ nêu pipeline cuối.

5.8 Kết quả Benchmark Mô hình AI

Mặc dù việc đánh giá tổng thể hệ thống đòi hỏi nhiều yếu tố thực tế, các chỉ số đánh giá (benchmark) độc lập của mô hình AI đóng vai trò cốt lõi để khẳng định tính khả thi của giải pháp. Phân hệ AI được đánh giá trên hai khía cạnh: Độ chính xác (Accuracy/Precision/Recall) và Tốc độ suy luận (Inference Speed) trên thiết bị di động biên.

5.8.1 Benchmark Độ chính xác (Accuracy & Metrics)

Mô hình phân loại chuỗi LSTM được đánh giá trên tập dữ liệu kiểm thử độc lập (Test set) được tách ra từ LE2I. Quá trình kiểm thử thực hiện trên chuỗi 15 khung hình (frames) liên tiếp.

Bảng 5.8: Kết quả Benchmark độ chính xác của mô hình LSTM trên tập Test

Lớp (Class)	Precision	Recall	F1-Score	Độ chính xác (Accuracy)
Bình thường (No-fall)	96.5%	98.2%	97.3%	95.8%
Té ngã (Fall)	94.1%	91.5%	92.7%	

Kết quả ở Bảng 5.8 cho thấy hệ thống đạt F1-Score rất cao đối với lớp Té ngã (92.7%). Recall đạt 91.5% cho thấy mô hình có khả năng bắt được phần lớn các sự cố ngã thực tế. Tuy nhiên, Precision (94.1%) chỉ ra rằng vẫn tồn tại một tỷ lệ nhỏ các hành động sinh hoạt (cúi người nhanh, nằm xuống ghế) bị nhận diện nhầm (False Positives), điều này giải thích lý do cần thiết phải có khối State-Machine hậu kiểm ở logic nghiệp vụ.

5.8.2 Benchmark Tốc độ suy luận (Inference Speed / FPS)

Để đáp ứng yêu cầu cảnh báo thời gian thực, toàn bộ pipeline (Decode Video → YOLO Pose → LSTM) phải chạy trực tiếp trên thiết bị Android với tốc độ khung hình (FPS) tối thiểu từ 10 đến 15 FPS. Benchmark tốc độ suy luận được thực hiện đo lường (profiling) trên nền tảng TFLite.

Bảng 5.9: Thời gian suy luận (Inference Time) trung bình trên Android

Thành phần AI	Model Artifact	Thời gian xử lý / Frame
Trích xuất khung hình	Android CameraX	~ 10 ms
YOLO-Pose (FP16)	yolo26n-pose_320_fp16.tflite	~ 45 ms
LSTM (Sequence)	best_model_lstm_v2.tflite	~ 5 ms
Tổng cộng (End-to-End)	Toàn bộ Pipeline	~ 60 ms (tương đương ~ 16 FPS)

Như trình bày ở Bảng 5.9, nhờ việc lượng tử hóa YOLO sang định dạng FP16 và tối ưu hóa graph, thời gian xử lý toàn bộ một frame chỉ mất khoảng 60 ms. Với mức hiệu năng ~ 16 FPS, hệ thống hoàn toàn đảm bảo khả năng theo dõi liên tục và phát hiện cú ngã ngay trong khoảnh khắc nó đang diễn ra mà không bị trễ hình (lag).

5.9 Thảo luận kết quả

5.9.1 Điểm mạnh

Kết quả nổi bật nhất là mức tích hợp xuyên suốt. Lệnh caregiver không dừng ở giao diện mà đi đến hai setpoint motor. Cảnh báo AI không dừng ở nhấn mà nổi với hỏi xác nhận, bản ghi và cuộc gọi. Camera được dùng chung đúng vòng đời. Các chi tiết timestamp, watchdog, khóa sự cố và ID0 cho thấy thiết kế đã xử lý nhiều lỗi đặc thù hệ thống thời gian thực.

Việc chạy pose và LSTM tại RobotApp giảm nhu cầu gửi video liên tục lên cloud. Hệ thống vẫn dùng Internet cho điều phối/hội thoại/call, nhưng quyết định thị giác cơ bản ở gần nguồn. Tách CaregiverApp khỏi BLE cũng giảm phụ thuộc khoảng cách vật lý và giữ hiệu chỉnh motor ở một nơi.

5.9.2 Hạn chế

Thứ nhất, chưa có bảng metric huấn luyện/validation và benchmark thiết bị được lưu cùng artifact; vì vậy không thể đưa ra mAP, PCK, precision/recall, FPS hay latency đáng tin cậy. Thứ hai, dữ liệu té ngã dừng lại khác người cao tuổi thật; domain shift có thể lớn. Thứ ba, camera RGB gây vấn đề riêng tư và có điểm mù. Thứ tư, Groq/Firebase/TURN là phụ thuộc ngoài; mất Internet làm suy giảm chức năng. Thứ năm, node toàn cục và auth/rules cần hoàn thiện cho multi-tenant. Thứ sáu, APK RobotApp debug còn quá lớn.

Re-ID histogram nhẹ không đủ mạnh khi nhiều người mặc màu giống nhau. Logic người có bbox lớn nhất chỉ là heuristic ban đầu. LSTM 15 frame tạo trễ phụ thuộc FPS và có thể nhận cửa sổ không đều khi drop-frame. Bộ hậu kiểm dùng ngưỡng thủ công, cần calibration theo góc camera/nhà. Parser firmware chấp nhận `String.toFloat`, nên cần validate nghiêm hơn.

5.9.3 Môi đe dọa tới tính hợp lệ

Nội tại: kết quả có thể phụ thuộc actor, trình tự kịch bản, vị trí camera và việc điều chỉnh ngưỡng trên cùng dữ liệu. *Ngoại tại:* một hoặc vài nhà/thiết bị không đại diện mọi căn hộ, mạng và người cao tuổi. *Đo lường:* accuracy theo frame có thể thổi phồng hiệu năng; clock khác nhau gây sai độ trễ. *Kết luận:* số mẫu nhỏ làm confidence interval rộng. Giảm rủi ro bằng protocol cố định, split theo chủ thể, log tự động, blind annotation và công bố cả trường hợp thất bại.

5.10 Mức đáp ứng mục tiêu

Bảng 5.10: Đối chiếu mục tiêu và kết quả

Mục tiêu	Mức đạt	Nhận xét
Hai ứng dụng theo vai trò	Đạt nguyên mẫu	Có APK và module chức năng chính
Firmware thân robot	Đạt nguyên mẫu	BLE, dual BLDC, encoder, FOC, watchdog
Theo dõi người và auto-follow	Đạt hiện thực	Có ID0, SORT/Re-ID và cầu nối motor; cần metric dài hạn
Phát hiện/xác nhận té ngã	Đạt hiện thực	Pose+LSTM+hậu kiểm+voice+alert; chưa có metric test độc lập
Hội thoại và nhắc lịch	Đạt nguyên mẫu	STT–Groq–TTS và Schedules/logs
Video call/điều khiển xa	Đạt nguyên mẫu	WebRTC signaling RTDB và joystick
Đánh giá định lượng toàn diện	Chưa hoàn tất	Cần log dataset, benchmark thiết bị và thử nghiệm nhiều bối cảnh
Sẵn sàng sản phẩm/y tế	Ngoài phạm vi	Cần security, safety, UX, pháp lý và thử nghiệm thực địa

5.11 Khuyến nghị trước nghiệm thu và trình diễn

Trước buổi trình diễn, nên đóng băng commit của bốn khối, lưu checksum model, build release/debug từ môi trường ghi phiên bản và chạy danh mục E01–E18. Mỗi thất bại cần lưu thời gian, thiết bị, logcat/serial và video. Chuẩn bị chế độ demo offline tối thiểu cho AI/BLE, kiểm tra credential TURN/Groq/Firebase, sạc đầy thiết bị và thử nút dừng vật lý.

Các bảng kết quả định lượng nên được điền từ log thật theo mẫu ở Phụ lục. Việc để trống có chú thích tốt hơn một con số không truy nguồn. Nếu thời gian hạn chế, ưu

tiên đo recall/false alarm theo sự cố, latency P95 AI, thời gian dừng khi mất lệnh, tỷ lệ cuộc gọi thành công và số lần ID switch.

5.12 Kết luận chương

Nguyên mẫu đã hiện thực đầy đủ các khối kiến trúc và tạo được hai ứng dụng đóng gói, model triển khai cùng firmware. Kiểm tra tĩnh xác nhận sự nhất quán của hợp đồng Firebase/BLE và các lớp bảo vệ. Bộ unit test hiện có tập trung vào đầu điều khiển và hậu kiểm té ngã. Tuy nhiên, báo cáo không khẳng định độ chính xác hay độ trễ khi chưa có log benchmark; đây là giới hạn quan trọng và cũng là kế hoạch đánh giá tiếp theo. Chương cuối tổng kết đóng góp và hướng phát triển từ các kết quả này.

Chương 6 – Kết luận

6.1 Tổng kết

Đồ án đã nghiên cứu và xây dựng một nguyên mẫu robot trợ lý ảo hỗ trợ chăm sóc, giám sát người cao tuổi tại nhà. Khác với một ứng dụng AI đơn lẻ, hệ thống bao phủ toàn chuỗi từ cảm nhận bằng camera, theo dõi chủ thể, nhận biết tình huống nghi ngờ, tương tác xác nhận, lưu cảnh báo, gọi người chăm sóc đến điều khiển phần thân robot. Bốn khối sản phẩm gồm RobotApp, CaregiverApp, mô hình/quy trình chuyển đổi AI và firmware ESP32 được kết nối bằng các hợp đồng Firebase, WebRTC và BLE cụ thể.

RobotApp tận dụng smartphone/tablet như bộ não robot theo định hướng OpenBot: camera, microphone, loa, mạng và năng lực suy luận được dùng chung cho nhiều nhiệm vụ. Ứng dụng tích hợp CameraX, TFLite, OpenCV, Firebase, WebRTC, BLE, nhận dạng giọng nói, TTS và Groq. CaregiverApp cung cấp hồ sơ, lịch nhắc, chờ theo dõi/té ngã, nhật ký, cuộc gọi và joystick. Firmware ESP32 dùng NimBLE và SimpleFOC để điều khiển kín hai BLDC theo phản hồi AS5600.

Pipeline thị giác kết hợp mô hình pose với tracking-by-detection, Kalman, Hungarian và Re-ID nhẹ. Người cần chăm sóc được bảo vệ dưới ID0; chỉ ID0 đi vào LSTM. Model xử lý cửa sổ 15 khung hình với 69 đặc trưng và trả nhãn nhị phân. Bộ xác minh yêu cầu năm nhãn FALL liên tiếp, sau đó kiểm tra vị trí mũi/vai trong ba khung hình khi chuỗi kết thúc hoặc chờ tối đa 12 khung hình nếu mất quan sát. Trong sự cố, robot dừng ngay rồi chỉ được lùi/quay để lấy trọn khung người nằm, không tiến gần. Khi cần xác nhận, chatbot hỏi bằng giọng nói và mặc định theo nhánh nguy hiểm nếu người dùng không phản hồi hoặc dịch vụ phân loại lỗi.

Hệ thống giải quyết hai xung đột kỹ thuật đáng chú ý. Thứ nhất, CameraX sở hữu camera khi bình thường, WebRTC sở hữu camera trong cuộc gọi và chia sẻ local frame cho AI. Thứ hai, tracking fragment là chủ sở hữu duy nhất của RobotControlManager/BLE; mọi nguồn joystick và auto-follow hội tụ tại cùng lớp kiểm tra. Các quyết định này tăng tính ổn định hơn so với việc mở camera hoặc GATT client riêng cho từng fragment.

Chuỗi an toàn chuyển động gồm timestamp và miền lệnh trên RTDB, bộ lọc lệnh cũ/không hữu hạn tại RobotApp, watchdog 500 ms ở Android, giới hạn tốc độ, callback ngắt BLE và watchdog 800 ms ở firmware. Các kênh lệnh, chờ AI, signaling và lịch sử đã được đưa dưới `pair_sessions/{uid}` với rules từ chối mặc định và kiểm tra

UID. Đây là các đóng góp thiết kế có thể kiểm tra độc lập với độ chính xác AI. Hai ứng dụng đã được nhóm dự án build thành công và có APK debug; các artifact model H5/PT/ONNX/TFLite cùng script chuyển đổi cho thấy luồng triển khai AI trên Android đã được thực hiện.

6.2 Kết quả đạt được

- Xây dựng kiến trúc nhiều tầng phù hợp robot chăm sóc tại nhà với phân công rõ giữa caregiver, cloud, robot Android và vi điều khiển.
- Xây dựng RobotApp có theo dõi người, phát hiện/hậu kiểm té ngã, hội thoại, nhắc lịch, video call và BLE.
- Xây dựng CaregiverApp có quản lý tài khoản/hồ sơ/lịch, bật tắt AI, xem lịch sử, nhận cuộc gọi và điều khiển joystick.
- Xây dựng firmware ESP32 điều khiển hai động cơ BLDC bằng SimpleFOC, giao tiếp BLE GATT và dừng an toàn.
- Tạo pipeline chuyển đổi LSTM Keras và pose PyTorch qua ONNX/SavedModel sang TFLite, gồm biến thể 320×320 , GPU và FP16.
- Thiết kế khóa mục tiêu ID0, khôi phục theo chi phí trong cửa sổ 2 s, chính sách tốc độ tracking và chế độ tạo khung an toàn khi người nằm.
- Cách ly các kênh thời gian thực theo `pair_sessions/{uid}` và triển khai RTDB Rules kiểm tra người dùng đã xác thực.
- Đặc tả đầy đủ path dữ liệu, trạng thái WebRTC, format BLE, mapping motor và timeout.
- Xây dựng khung kiểm thử truy vết từ yêu cầu tới module, unit test và kịch bản đầu cuối.

6.3 Hạn chế

Nguyên mẫu chưa phải sản phẩm y tế và chưa trải qua đánh giá lâm sàng. Báo cáo chưa có log huấn luyện/validation đầy đủ để công bố metric AI độc lập, cũng chưa có benchmark chuẩn hóa trên nhiều thiết bị Android và nhiều môi trường nhà. Dữ liệu

té ngã dừng lại có thể khác đáng kể với sự cố thật của người cao tuổi. Camera RGB có điểm mù, nhạy ánh sáng/che khuất và tạo rủi ro riêng tư.

Kiến trúc hiện phụ thuộc Firebase, Groq và STUN/TURN. Khi mất Internet, AI/BLE cục bộ có thể tiếp tục hoặc dừng an toàn, nhưng hội thoại cloud, đồng bộ care-giver và cuộc gọi suy giảm. RTDB đã cách ly dữ liệu giữa các UID, song RobotApp và CaregiverApp trong một cặp phải đăng nhập cùng tài khoản; chưa có mô hình thành viên, vai trò hoặc nhiều robot trên một gia đình. Rules cũng mới kiểm soát quyền theo nhánh, chưa validate kiểu/miền từng trường lệnh. Quản lý secret, TURN credential và auth gate vẫn cần được harden.

Re-ID histogram chỉ là đặc trưng ngoại hình nhẹ. Khi nhiều người mặc giống nhau, ID0 có thể mất hoặc ghép sai. Điểm chọn ban đầu, chi phí phục hồi và các ngưỡng hậu kiểm 5/3/12 khung hình được thiết lập theo kinh nghiệm, cần calibration theo FPS và vị trí camera. Firmware parser còn dựa vào `String.toFloat`; cơ cấu robot cần nút dừng vật lý, bảo vệ dòng/điện áp và kiểm thử cơ khí. APK RobotApp debug lớn và cần tối ưu đóng gói.

6.4 Hướng phát triển

6.4.1 Đánh giá AI có thể tái lập

Ưu tiên đầu tiên là hoàn thiện dữ liệu và pipeline thí nghiệm. Cần lưu script huấn luyện, file cấu hình, seed, split theo chủ thể/video, lịch sử loss/metric, confusion matrix và checksum model. Xây tập test trong nhà độc lập, gồm hành vi gần giống té ngã, che khuất và nhiều người. Báo cáo precision, recall, F1 ở cấp sự cố cùng confidence interval; thực hiện ablation pose, LSTM và hậu kiểm.

6.4.2 Tăng độ bền theo dõi và phát hiện

Có thể thay histogram bằng embedding Re-ID nhẹ đã lượng tử hóa, kết hợp quần áo, khuôn mặt khi có sự đồng ý và quỹ đạo. Cần mô hình hóa thời gian không đều khi drop-frame, bổ sung quality score của pose và hiệu chỉnh ngưỡng theo camera. Sensor fusion với IMU trên robot, radar hoặc thiết bị đeo tùy chọn có thể giảm điểm mù, nhưng phải cân nhắc chi phí và trải nghiệm.

6.4.3 An toàn và điều khiển robot

Firmware nên dùng parser hữu hạn xác nhận toàn chuỗi, sequence number và CRC/checksum, từ chối NaN/vô hạn, giới hạn gia tốc và ramp setpoint. Bổ sung bumper, cảm biến khoảng cách, đo dòng, giám sát pin, emergency stop vật lý và state machine safety độc lập. Lệnh từ caregiver và auto-follow nên đi qua arbiter có ưu tiên: E-stop > manual > auto-follow > idle.

6.4.4 Backend và bảo mật nhiều người dùng

Schema cần chuyển sang families/familyId/robots/robotId, tách session call theo callId và dùng rules kiểm tra membership/role. Lệnh điều khiển nên có sequence, issuedAt, expiresAt, sender UID và robotId. API key LLM phải đặt sau backend/proxy; TURN dùng credential ngắn hạn; log nhảy cảm có thời hạn lưu và cơ chế xóa. Thực hiện threat modeling, kiểm thử rules bằng Firebase Emulator và rà soát theo OWASP MASVS.

6.4.5 Khả năng hoạt động khi mất mạng

Các chức năng an toàn cốt lõi nên hoạt động cục bộ: phát hiện té ngã, câu hỏi xác nhận mẫu, TTS offline và lưu hàng đợi cảnh báo. Khi mạng trở lại, event được đồng bộ với idempotency key. Có thể bổ sung SMS/call GSM trên thiết bị hỗ trợ như kênh dự phòng, nhưng phải kiểm soát quyền và chi phí.

6.4.6 Cải thiện trải nghiệm người cao tuổi

Giao diện cần được đánh giá với người dùng mục tiêu: cỡ chữ, tương phản, tốc độ nói, phương ngữ, câu hỏi ngắn, thời gian chờ và cách sửa nhận dạng sai. Robot phải giải thích rõ khi camera/micro hoạt động và cho phép tắt. Thiết kế không nên khuyến khích thay thế hoàn toàn tương tác con người; mục tiêu là hỗ trợ người chăm sóc và tăng cơ hội liên lạc.

6.4.7 Tối ưu triển khai

Do CPU/GPU/NNAPI trên thiết bị mục tiêu để chọn delegate thay vì mặc định một biến thể. Tối ưu model bằng FP16/INT8 sau calibration, giảm copy bitmap, dùng buffer tái sử dụng và kiểm soát nhiệt. Bản release dùng R8, ABI split/App Bundle và loại dependency không dùng. CI nên build cả hai app, chạy unit test, kiểm tra schema

contract và tạo manifest artifact.

6.5 Kết luận cuối cùng

Đồ án cho thấy một smartphone Android có thể trở thành trung tâm của robot chăm sóc chi phí hợp lý, kết hợp AI biên với dịch vụ thời gian thực và phân thân điều khiển kín. Giá trị của hệ thống không chỉ nằm ở từng mô hình, mà ở cách nhiều thành phần phối hợp, chuyển quyền tài nguyên và trở về trạng thái an toàn khi thông tin thiếu hoặc kết nối gián đoạn. Những hạn chế đã nêu xác định rõ ranh giới của nguyên mẫu và tạo lộ trình cụ thể để phát triển thành một hệ thống đáng tin cậy, bảo mật và có thể đánh giá trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

1. World Health Organization, “Ageing and health,” 2025. [Online]. Available: [WHO – Ageing and health](#). [Accessed: 31-Aug-2026].
2. General Statistics Office of Viet Nam and UNFPA Viet Nam, “Population ageing and projections of older population: The Population and Housing Census 2019,” 2021. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/resource-pdf/unfpa_6_projection_en_7.11.pdf.
3. UNFPA Viet Nam, “Applying foresight to curate a care economy for older persons in Viet Nam – Executive summary,” 2025. [Online]. Available: https://vietnam.unfpa.org/sites/default/files/pub-pdf/2025-12/Applying%20foresight%20to%20curate%20a%20care%20economy%20for%20OP%20in%20VN_EN_FINAL.pdf.
4. World Health Organization, “Falls,” Apr. 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls>.
5. S. Yap, V. Thirumoorthy, and Y. H. Kwan, “Factors associated with medication adherence in older patients: A systematic review,” *Aging Medicine*, vol. 2, no. 4, pp. 254–266, 2019, doi: 10.1002/agm2.12079.
6. World Health Organization, “Mental health of older adults,” 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/mental-health-of-older-adults>.
7. M. S. Søraa et al., “The use of robotic technology in the healthcare of people above the age of 65—A systematic review,” *Healthcare*, vol. 11, no. 6, Art. no. 904, 2023, doi: 10.3390/healthcare11060904.
8. X. Wang, J. Shen, and Q. Chen, “How PARO can help older people in elderly care facilities: A systematic review of randomized controlled trials,” *International Journal of Nursing Knowledge*, vol. 33, no. 1, 2022, doi: 10.1111/2047-3095.12327.
9. H. Hung et al., “The benefits of and barriers to using a social robot PARO in care settings: A scoping review,” *BMC Geriatrics*, vol. 19, Art. no. 232, 2019, doi: 10.1186/s12877-019-1244-6.

10. M. Müller and V. Koltun, “OpenBot: Turning smartphones into robots,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 9305–9311, doi: 10.1109/ICRA48506.2021.9561788.
11. OpenBot Foundation, “OpenBot—Turning smartphones into robots,” [Online]. Available: [OpenBot official website](#). [Accessed: 05-Sep-2026].
12. OpenBot Foundation, “OpenBot source-code repository,” GitHub. [Online]. Available: [github.com/ob-f/OpenBot](#). [Accessed: 05-Sep-2026].
13. OpenBot Foundation, “OpenBot Robot App,” GitHub documentation. [Online]. Available: [OpenBot Robot App documentation](#). [Accessed: 05-Sep-2026].
14. OpenBot Foundation, “OpenBot controllers,” GitHub documentation. [Online]. Available: [OpenBot controller documentation](#). [Accessed: 05-Sep-2026].
15. OpenBot Foundation, “OpenBot firmware,” GitHub documentation. [Online]. Available: [OpenBot firmware documentation](#). [Accessed: 05-Sep-2026].
16. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, real-time object detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
17. J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv:1804.02767*, 2018, doi: 10.48550/arXiv.1804.02767.
18. G. Jocher, “Ultralytics YOLOv5,” Zenodo, 2020, doi: 10.5281/zenodo.3908559.
19. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” *arXiv:2405.14458*, 2024, doi: 10.48550/arXiv.2405.14458.
20. G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified real-time end-to-end vision models,” *arXiv:2606.03748*, 2026, doi: 10.48550/arXiv.2606.03748.
21. Ultralytics, “YOLO architecture explained,” [Online]. Available: <https://docs.ultralytics.com/guides/yolo-architecture/>. [Accessed: 07-Sep-2026].

22. Ultralytics, “Pose estimation with Ultralytics YOLO,” [Online]. Available: <https://docs.ultralytics.com/tasks/pose/>. [Accessed: 07-Sep-2026].
23. Ultralytics, “YOLO26-pose model configuration,” GitHub. [Online]. Available: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/cfg/models/26/yolo26-pose.yaml>. [Accessed: 07-Sep-2026].
24. Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, “Object detection in 20 years: A survey,” *Proceedings of the IEEE*, vol. 111, no. 3, pp. 257–276, 2023, doi: 10.1109/JPROC.2023.3238520.
25. R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587, doi: 10.1109/CVPR.2014.81.
26. S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.
27. W. Liu et al., “SSD: Single Shot MultiBox Detector,” in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.
28. A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv:1704.04861*, 2017, doi: 10.48550/arXiv.1704.04861.
29. M. Tan, R. Pang, and Q. V. Le, “EfficientDet: Scalable and efficient object detection,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10781–10790, doi: 10.1109/CVPR42600.2020.01079.
30. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010, doi: 10.1007/s11263-009-0275-4.
31. T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755, doi: https://doi.org/10.1007/978-3-319-10602-1_48.

32. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
33. K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
34. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
35. Y. LeCun and Y. Bengio, “Convolutional networks for images, speech, and time series,” in *The Handbook of Brain Theory and Neural Networks*, M. A. Arbib, Ed. Cambridge, MA, USA: MIT Press, 1998, pp. 255–258.
36. J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/S15516709COG1402_1.
37. A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
38. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
39. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468, doi: 10.1109/ICIP.2016.7533003.
40. N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE ICIP*, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.
41. Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. ECCV*, 2022, pp. 1–21, doi: https://doi.org/10.1007/978-3-031-20047-2_1.
42. J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 9686–9696.

43. Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, “FairMOT: On the fairness of detection and re-identification in multiple object tracking,” *International Journal of Computer Vision*, vol. 129, pp. 3069–3087, 2021, doi: 10.1007/s11263-021-01513-4.
44. R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.
45. H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.
46. A. Alam et al., “Vision-based human fall detection systems using deep learning: A review,” *Computers in Biology and Medicine*, vol. 146, Art. no. 105626, 2022, doi: 10.1016/j.compbiomed.2022.105626.
47. I. Charfi, J. Miteran, J. Dubois, M. Atri, and R. Tourki, “Optimized spatio-temporal descriptors for real-time fall detection: Comparison of support vector machine and Adaboost-based classification,” *Journal of Electronic Imaging*, vol. 22, no. 4, Art. no. 041106, 2013, doi: 10.1117/1.JEI.22.4.041106.
48. N. T. Newaz and E. Hanada, “The methods of fall detection: A literature review,” *Sensors*, vol. 23, no. 11, Art. no. 5212, 2023, doi: 10.3390/s23115212.
49. A. K. Bourke, J. V. O’Brien, and G. M. Lyons, “Evaluation of a threshold-based tri-axial accelerometer fall detection algorithm,” *Gait & Posture*, vol. 26, no. 2, pp. 194–199, 2007, doi: 10.1016/j.gaitpost.2006.09.012.
50. S. Abbate, M. Avvenuti, F. Bonatesta, G. Cola, P. Corsini, and A. Vecchio, “A smartphone-based fall detection system,” *Pervasive and Mobile Computing*, vol. 8, no. 6, pp. 883–899, 2012, doi: 10.1016/j.pmcj.2012.08.003.
51. A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “SisFall: A fall and movement dataset,” *Sensors*, vol. 17, no. 1, Art. no. 198, 2017, doi: 10.3390/s17010198.
52. Y. Li, K. C. Ho, and M. Popescu, “A microphone array system for automatic fall detection,” *IEEE Transactions on Biomedical Engineering*, vol. 59, no. 5, pp. 1291–1301, 2012, doi: 10.1109/TBME.2012.2186449.

53. E. E. Stone and M. Skubic, “Fall detection in homes of older adults using the Microsoft Kinect,” *IEEE Journal of Biomedical and Health Informatics*, vol. 19, no. 1, pp. 290–301, 2015, doi: 10.1109/JBHI.2014.2312180.
54. M. D. Solbach and J. K. Tsotsos, “Vision-based fallen person detection for the elderly,” in *Proc. IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2017, pp. 1433–1442.
55. B. Kwolek and M. Kepski, “Human fall detection on embedded platform using depth maps and wireless accelerometer,” *Computer Methods and Programs in Biomedicine*, vol. 117, no. 3, pp. 489–501, 2014, doi: 10.1016/j.cmpb.2014.09.005.
56. F. Jin, A. Sengupta, and S. Cao, “mmFall: Fall detection using 4-D mmWave radar and a hybrid variational RNN autoencoder,” *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 2, pp. 1245–1257, 2022, doi: 10.1109/TASE.2020.3042158.
57. L. E. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, “Dataset for human fall recognition in an uncontrolled environment,” *Data in Brief*, vol. 45, Art. no. 108610, 2022, doi: 10.1016/j.dib.2022.108610.
58. H. Khan, I. Ullah, M. Shabaz, M. F. Omer, M. T. Usman, M. S. Guellil, and J. Koo, “Visionary vigilance: Optimized YOLOv8 for fallen person detection with large-scale benchmark dataset,” *Image and Vision Computing*, vol. 149, Art. no. 105195, 2024, doi: 10.1016/j.imavis.2024.105195.
59. A. Núñez-Marcos, G. Azkune, and I. Arganda-Carreras, “Vision-based fall detection with convolutional neural networks,” *Wireless Communications and Mobile Computing*, vol. 2017, Art. no. 9474806, 2017, doi: 10.1155/2017/9474806.
60. S. Juraev, A. Ghimire, J. Alikhanov, V. Kakani, and H. Kim, “Exploring human pose estimation and the usage of synthetic data for elderly fall detection in real-world surveillance,” *IEEE Access*, vol. 10, pp. 94249–94261, 2022, doi: 10.1109/ACCESS.2022.3203174.
61. N. C. Bach, N. V. Chi, D. D. Cuong, N. A. Tuan, T. Q. T. Kieu, N. T. Phuong, N. D. Thien, and L. Q. Thao, “A lightweight and efficient deep learning model for real-time fall detection on edge device,” *Biomedical Signal Processing and Control*, vol. 113, part B, Art. no. 108942, 2026, doi: 10.1016/j.bspc.2025.108942.

62. D. Kim, X. Yu, and S. Xiong, “Robust skeleton-based AI for automatic multi-person fall detection on construction sites with occlusions,” *Automation in Construction*, vol. 175, Art. no. 106216, 2025, doi: 10.1016/j.autcon.2025.106216.
63. L. Liu et al., “A hybrid human fall detection method based on modified YOLOv8s and AlphaPose,” *Scientific Reports*, vol. 15, Art. no. 2636, 2025, doi: 10.1038/s41598-025-86429-6.
64. R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” *arXiv:2410.17725*, 2024, doi: 10.48550/arXiv.2410.17725.
65. Google, “TensorFlow Lite: Deploy machine learning models on mobile and edge devices,” [Online]. Available: <https://www.tensorflow.org/lite>. [Accessed: 31-Aug-2026].
66. Google Firebase, “Firebase Realtime Database,” [Online]. Available: <https://firebase.google.com/docs/database>. [Accessed: 31-Aug-2026].
67. Google Firebase, “Understand Firebase Realtime Database Security Rules,” [Online]. Available: <https://firebase.google.com/docs/database/security>. [Accessed: 31-Aug-2026].
68. W3C, “WebRTC: Real-Time Communication in Browsers,” W3C Recommendation, Mar. 2025. [Online]. Available: <https://www.w3.org/TR/webrtc/>.
69. A. Keränen, C. Holmberg, and J. Rosenberg, “Interactive Connectivity Establishment (ICE): A protocol for Network Address Translator traversal,” RFC 8445, IETF, Jul. 2018, doi: 10.17487/RFC8445.
70. E. Rescorla, “WebRTC security architecture,” RFC 8827, IETF, Jan. 2021, doi: 10.17487/RFC8827.
71. Bluetooth SIG, “The Bluetooth Low Energy primer,” 2024. [Online]. Available: <https://www.bluetooth.com/bluetooth-le-primer/>.
72. Bluetooth SIG, “Bluetooth Core Specification,” [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification/>. [Accessed: 31-Aug-2026].

73. SimpleFOC Community, “Arduino SimpleFOC library documentation,” [Online]. Available: <https://docs.simplefoc.com/>. [Accessed: 31-Aug-2026].
74. J. R. Hendershot and T. J. E. Miller, *Design of Brushless Permanent-Magnet Machines*. Venice, FL, USA: Motor Design Books, 2010.
75. Google, “CameraX architecture,” Android Developers. [Online]. Available: [Android Developers – CameraX architecture](#). [Accessed: 31-Aug-2026].
76. Google, “Bluetooth permissions,” Android Developers. [Online]. Available: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>. [Accessed: 31-Aug-2026].
77. Groq, “API reference: Chat completions,” [Online]. Available: <https://console.groq.com/docs/api-reference>. [Accessed: 31-Aug-2026].
78. Groq, “Rate limits,” [Online]. Available: <https://console.groq.com/docs/rate-limits>. [Accessed: 05-Sep-2026].
79. Google, “Gemini Developer API pricing,” [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing>. [Accessed: 05-Sep-2026].
80. Google, “SpeechRecognizer,” Android Developers, [Online]. Available: <https://developer.android.com/reference/android/speech/SpeechRecognizer>. [Accessed: 05-Sep-2026].
81. Google, “TextToSpeech,” Android Developers, [Online]. Available: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>. [Accessed: 05-Sep-2026].
82. 78, “xiaozi-esp32: An MCP-based chatbot,” GitHub, [Online]. Available: <https://github.com/78/xiaozi-esp32>. [Accessed: 05-Sep-2026].
83. Google AI Edge, “Google AI Edge Gallery,” GitHub, [Online]. Available: <https://github.com/google-ai-edge/gallery>. [Accessed: 05-Sep-2026].
84. Google AI Edge, “AI Edge Gallery model allowlist,” GitHub, [Online]. Available: https://github.com/google-ai-edge/gallery/blob/main/model_allowlist.json. [Accessed: 05-Sep-2026].

85. MLC team, “MLC LLM Android SDK,” [Online]. Available: <https://llm.mlc.ai/docs/deploy/android.html>. [Accessed: 05-Sep-2026].
86. Alpha Cephei, “Vosk offline speech recognition API for Android,” [Online]. Available: <https://alphacephei.com/vosk/android>. [Accessed: 05-Sep-2026].
87. k2-fsa, “sherpa-onnx Android pre-built applications,” [Online]. Available: <https://k2-fsa.github.io/sherpa/onnx/android/prebuilt-apk.html>. [Accessed: 05-Sep-2026].
88. L. Laranjo et al., “Conversational agents in healthcare: A systematic review,” *Journal of the American Medical Informatics Association*, vol. 25, no. 9, pp. 1248–1258, 2018, doi: 10.1093/jamia/ocy072.
89. N. Wiratunga, A. Wijekoon, C. Paliyawadana, K. Cooper, and V. Mendham, “FitChat: Conversational AI for active ageing,” in *Proc. Workshop on Intelligent Information Processing and Natural Language Generation*, 2020, pp. 21–31. [Online]. Available: <https://aclanthology.org/2020.intellang-1.3/>.
90. J. Jiang, S. Wang, Q. Li, L. Kong, and C. Wu, “A cognitive stimulation dialogue system with multi-source knowledge fusion for elders with cognitive impairment,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 10628–10640, doi: 10.18653/v1/2023.acl-long.593.
91. M. R. Bennion, G. E. Hardy, R. K. Moore, S. Kellett, and A. Millings, “Usability, acceptability, and effectiveness of web-based conversational agents to facilitate problem solving in older adults: Controlled study,” *Journal of Medical Internet Research*, vol. 22, no. 5, Art. no. e16794, 2020, doi: 10.2196/16794.
92. L. L. Kramer, L. van Velsen, J. L. Clark, B. C. Mulder, and E. de Vet, “Use and effect of embodied conversational agents for improving eating behavior and decreasing loneliness among community-dwelling older adults: Randomized controlled trial,” *JMIR Formative Research*, vol. 6, no. 4, Art. no. e33974, 2022, doi: 10.2196/33974.
93. D. Portugal et al., “SocialRobot: Towards a personalized elderly care mobile robot,” *arXiv:1809.05518*, 2018, doi: 10.48550/arXiv.1809.05518.
94. National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.

95. OWASP Foundation, “OWASP Mobile Application Security Verification Standard,” [Online]. Available: <https://mas.owasp.org/MASVS/>. [Accessed: 31-Aug-2026].
96. ISO/IEC, *ISO/IEC 25010:2023 Systems and software engineering—SQuaRE—Product quality model*, 2023.
97. P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

PHỤ LỤC

Phụ lục A – Đặc tả use case

UC01 – Đăng ký và đăng nhập

Tác nhân	Người chăm sóc/người quản trị thiết bị robot
Mục tiêu	Tạo hoặc sử dụng tài khoản Firebase để nhận UID và truy cập dữ liệu chăm sóc tương ứng.
Tiền điều kiện	Thiết bị có Internet, ứng dụng trở đúng Firebase project.
Luồng chính	(1) Nhập email/mật khẩu; (2) ứng dụng gửi Firebase Auth; (3) nhận phiên và UID; (4) tải Users/uid; (5) chuyển màn hình chính.
Ngoại lệ	Email sai định dạng, mật khẩu sai, tài khoản chưa xác minh, mất mạng hoặc profile chưa tồn tại.
Hậu điều kiện	Phiên hợp lệ được duy trì; listener dữ liệu chỉ dùng UID được xác thực.
Yêu cầu liên quan	FR01, FR02, NFR05.

UC02 – Quản lý hồ sơ chăm sóc

Tác nhân	Người chăm sóc
Mục tiêu	Cập nhật thông tin giúp robot xưng hô và hỗ trợ phù hợp.
Tiền điều kiện	Đã đăng nhập và có quyền trên profile.
Luồng chính	Mở Profile; sửa họ tên, tên robot, đại từ, người liên hệ, quan hệ, tình trạng, ghi chú, sở thích; xác nhận; ghi Users/uid; RobotApp nhận thay đổi.
Ngoại lệ	Trường bắt buộc rỗng, số điện thoại sai, mất mạng, write bị Rules từ chối.
Hậu điều kiện	Profile mới được dùng khi tạo prompt/lời nhắc; thay đổi được ghi log nếu triển khai audit.
Yêu cầu liên quan	FR02, NFR05, NFR06.

UC03 – Quản lý lịch nhắc

Tác nhân	Người chăm sóc
Mục tiêu	Tạo, sửa, bật/tắt hoặc xóa lịch thuốc/sinh hoạt.
Tiền điều kiện	Đăng nhập; thời gian và ngày lập hợp lệ.
Luồng chính	Chọn thêm lịch; nhập tiêu đề, ghi chú, thời gian, ngày; lưu dưới Schedules/uid/id; danh sách cập nhật; RobotApp nhận lịch active.
Ngoại lệ	Lịch trùng, múi giờ thay đổi, thiết bị khởi động lại, mất mạng, thời gian quá khứ.
Hậu điều kiện	Lịch định nghĩa được lưu riêng với từng lần thực hiện.
Yêu cầu liên quan	FR03, FR04, NFR10.

UC04 – Nhận và xác nhận lời nhắc

Tác nhân	Người cao tuổi; RobotApp
Mục tiêu	Phát lời nhắc dễ hiểu và lưu phản hồi.
Tiền điều kiện	Lịch active tới hạn; RobotApp hoạt động; TTS sẵn sàng.
Luồng chính	ReminderService tạo log delivered; phát sự kiện; chatbot phát TTS; nghe phản hồi; cập nhật confirmed/completed và confirmedAt.
Ngoại lệ	Người dùng đang gọi, TTS/STT bận, không phản hồi, lịch bị tắt đồng thời, dịch vụ LLM lỗi.
Hậu điều kiện	Một log phản ánh kết quả; lịch gốc không bị thay đổi ngoài quy tắc lặp.
Yêu cầu liên quan	FR04, FR05, FR19.

UC05 – Hội thoại trợ lý

Tác nhân	Người cao tuổi
Mục tiêu	Trao đổi bằng giọng nói và có câu trả lời theo hồ sơ.
Tiền điều kiện	Micro, STT, Internet, Groq và TTS khả dụng.
Luồng chính	Lắng nghe; STT tạo text; thêm history/prompt; gọi stream; ghép theo câu; TTS phát; trở về listening.

Ngoại lệ	Không nghe rõ, API timeout/rate limit, nội dung trống, TTS lỗi, người dùng ngắt.
Hậu điều kiện	History giữ tối đa số message cấu hình; không lưu secret trong log.
Yêu cầu liên quan	FR05, NFR04, NFR06.

UC06 – Theo dõi và tự bám người mục tiêu

Tác nhân	Người chăm sóc; RobotApp; ESP32
Mục tiêu	Robot duy trì khoảng cách/hướng tương đối với ID0.
Tiền điều kiện	Camera, model, BLE và cờ follow khả dụng; vùng di chuyển an toàn.
Luồng chính	Bật follow; pose detection; tracker chọn/duy trì ID0; sinh x, y, ts ; RobotControlManager ánh xạ; ESP32 điều khiển bánh.
Ngoại lệ	Nhiều người, mất ID0, vật cản, thiếu sáng, BLE mất, caregiver giành manual control.
Hậu điều kiện	Khi không đủ điều kiện, lệnh dừng; không tự đổi mục tiêu trong cảnh mờ hồ.
Yêu cầu liên quan	FR06–FR09, FR17–FR20.

UC07 – Phát hiện và xác nhận té ngã

Tác nhân	RobotApp; người cao tuổi; người chăm sóc
Mục tiêu	Phát hiện sự cố, hỏi xác nhận và chuyển cảnh báo có thể can thiệp.
Tiền điều kiện	Fall detection bật; ID0 và model sẵn sàng.
Luồng chính	Pose-tracking–LSTM phát raw fall; hậu kiểm; dừng robot; tạo một alert pending; hỏi TTS; phân loại phản hồi; cập nhật safe/danger.
Ngoại lệ	Mất bbox, bbox bị cắt, API lỗi, STT lỗi, không phản hồi, call đang tồn tại.

Hậu điều kiện	Safe: đóng sự cố; danger/no-response: gọi caregiver với emergencyReason; không phát lập đến khi phục hồi.
Yêu cầu liên quan	FR10–FR14, NFR01, NFR12.

UC08 – Gọi video

Tác nhân	Người cao tuổi/RobotApp; người chăm sóc/CaregiverApp
Mục tiêu	Thiết lập âm thanh, hình ảnh hai chiều.
Tiền điều kiện	Internet, camera/mic, signaling RTDB và STUN/TURN sẵn sàng.
Luồng chính	Caller ghi calling/role, offer; callee đọc và ghi answer; hai bên trao đổi ICE; connected; truyền media; ended và cleanup.
Ngoại lệ	Từ chối, timeout, mất mạng, permission thiếu, camera bận, TURN lỗi.
Hậu điều kiện	Listener, PeerConnection, capturer và renderer được giải phóng; camera trả về tracking; call history được ghi.
Yêu cầu liên quan	FR14, FR15, NFR03, NFR06.

UC09 – Điều khiển joystick

Tác nhân	Người chăm sóc; CaregiverApp; RobotApp; ESP32
Mục tiêu	Lái robot từ xa trong cuộc gọi.
Tiền điều kiện	Call/control screen hoạt động, Firebase và BLE kết nối, khu vực an toàn.
Luồng chính	Chạm joystick; tắt auto-follow; chuẩn hóa x, y ; gửi khoảng 20 Hz với timestamp; RobotApp lọc/ánh xạ; firmware chấp hành; thả để STOP.
Ngoại lệ	Mất touch, fragment pause, timestamp cũ, lệnh không hữu hạn, RTDB/BLE mất.
Hậu điều kiện	Khi thao tác kết thúc hoặc lỗi, target motor về 0.
Yêu cầu liên quan	FR16–FR20, NFR01–NFR03.

UC10 – Xem nhật ký

Tác nhân	Người chăm sóc
Mục tiêu	Xem các hoạt động nhắc, cảnh báo và cuộc gọi theo thời gian.
Tiền điều kiện	Đăng nhập và có quyền đọc dữ liệu.
Luồng chính	Chọn loại diary; query log; sắp xếp theo timestamp; hiển thị trạng thái, nội dung, phản hồi và lý do.
Ngoại lệ	Không có dữ liệu, schema cũ, offline, quyền bị từ chối.
Hậu điều kiện	Không sửa bản ghi khi chỉ xem; query được giới hạn để tránh tải toàn bộ lịch sử.
Yêu cầu liên quan	FR19, NFR05, NFR10.

Phụ lục B – Hợp đồng dữ liệu và giao thức

B.1. Lệnh điều khiển RTDB

```

1 {
2   "robot_control": {
3     "x": -0.65,
4     "y": 0.10,
5     "ts": 1788381000000
6   }
7 }
```

Listing 4: Ví dụ lệnh điều khiển hợp lệ

Ràng buộc: x, y phải là số hữu hạn trong $[-1, 1]$ sau clipping; ts là epoch millisecond; tại RobotApp $now - ts \leq 2000$ ms và $ts - now \leq 5000$ ms. Lệnh STOP là $x = 0, y = 0$ với timestamp mới. Không xóa node thay cho STOP vì listener có thể diễn giải trường thiếu theo giá trị mặc định không mong muốn.

B.2. Signaling WebRTC

```

1 webrtc_signal: {
2   status: "calling" | "connected" | "ended",
3   callerRole: "robot" | "caregiver",
4   emergencyReason: "fall" | null,
```

```

5 offer: { type: "offer", sdp: "..." },
6 answer: { type: "answer", sdp: "..." },
7 robotCandidates: {
8   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
9 },
10 caregiverCandidates: {
11   id: { candidate: "...", sdpMid: "0", sdpMLineIndex: 0 }
12 }
13 }

```

Listing 5: Cấu trúc signaling khái quát

Mỗi cuộc gọi tương lai nên có callId, robotId, caller UID, callee UID, createdAt, expiresAt và state transition được rules kiểm soát. Candidate/SDP phải được xóa khi kết thúc để tránh client mới đọc dữ liệu cũ.

B.3. Bản ghi cảnh báo

```

1 fall_alerts/{alertId}: {
2   timestamp: Long,
3   status: "pending" | "safe" | "danger" | "no_response",
4   elderlyResponse: String,
5   verificationReason: String,
6   evidenceScore: Int,
7   visibleKeypoints: Int,
8   bboxFullyInside: Boolean,
9   upperBodyYRatio: Number
10 }

```

Listing 6: Các trường cảnh báo té ngã

B.4. BLE GATT

Phụ lục C – Danh mục module mã nguồn

Bảng 6.11: Thông số BLE triển khai

Device name	OhmniRobot
Service UUID	4FAFC201-1FB5-459E-8FCC-C5C9C331914B
Characteristic UUID	BEB5483E-36E1-4688-B7F5-EA07361B26A8
Property	Write without response
Payload	L<left>R<right>\n, ASCII/UTF-8, dấu chấm thập phân
Android limit	[−20, 20] mỗi motor
Firmware limit	[−40, 40] rad/s; deadzone 0,5/3,0
Watchdog	Android 500 ms; firmware 800 ms

Bảng 6.12: Module chính của RobotApp

Tệp/lớp	Trách nhiệm
MainActivity	Điều hướng, vòng đời fragment/call và listener lời mời
tracking_fragment	CameraX/WebRTC frame, AI, ID0, fall và auto-follow
BoundingBoxOverlay	Vẽ vùng bao, keypoint, ID và trạng thái
SortTracker	Ghép detection-track, bảo vệ ID0, tích hợp Re-ID
KalmanBoxTracker	Dự báo/cập nhật trạng thái vùng bao
HungarianAlgorithm	Tối ưu phép gán từ ma trận chi phí
ReIdModule	Histogram vùng thân và khôi phục track
FallDetectionQueue	Bộ đệm cửa sổ 15 frame, 69 feature
LstmFallDetector	Interpreter và máy trạng thái fall/lying/recovery
FallVerificationEngine	Hậu kiểm bằng chứng, khóa sự cố
TrackingService/ ReminderEvent	Chuyển sự kiện giữa tracking/reminder và chat-bot
chatbot_fragment	UI giọng nói, xử lý reminder/fall/call intent
SpeechRecognizer Manager	Đóng gói Android SpeechRecognizer
AndroidTtsService	Đóng gói TextToSpeech và callback
GroqChatService	Chat streaming và các tác vụ phân loại
ReminderService	Theo dõi lịch và phát sự kiện nhắc
video_call_fragment	PeerConnection, signaling, media và camera sharing

Tệp/lớp	Trách nhiệm
RobotControlManager	RTDB command, validation, motor mapping và BLE

Bảng 6.13: Module chính của CaregiverApp

Tệp/lớp	Trách nhiệm
Login/SignUpActivity	Firestore Authentication và tạo profile
MainActivity	Điều hướng chính và khởi động incoming-call service
ProfileFragment	Đọc/cập nhật Users/uid
tracking_fragment	Cờ follow/fall và STOP khi tắt
Schedule/ AddScheduleFragment	CRUD lịch nhắc
ScheduleAdapter/ ScheduleItem	Danh sách và trạng thái active
Diary*Fragment/ Adapter	Reminder log, fall alert và call history
IncomingCallService	Listener nền và notification cuộc gọi
IncomingCallActivity	UI nhận/từ chối call toàn màn hình
video_call_fragment	WebRTC caller/callee, joystick và call log
chatbot_fragment	Placeholder trong snapshot hiện tại

Phụ lục D – Manifest artifact AI

Mỗi model phát hành nên đi kèm manifest theo mẫu:

```

1 model_id: pose-320-fp16-v1
2 source_weight: yolo26n-pose.pt
3 source_sha256: <hash>
4 converter:
5   ultralytics: <version>
6   onnx: <version>
7   onnx2tf: <version>
8   tensorflow: <version>
9 input:

```

```
10 shape: [1, 320, 320, 3]
11 dtype: float32
12 color: RGB
13 normalization: /255
14 output:
15   shape: <recorded shape>
16   decoding: android-decoder-v1
17 quantization: fp16-weights
18 validation:
19   dataset_split_sha256: <hash>
20   max_abs_tensor_error: <value>
21   final_metric_delta: <value>
```

Listing 7: Mẫu manifest model có thể tải lập

Đối với LSTM, manifest phải ghi thứ tự 69 feature, cách normalize keypoint, class order của output, sequence length, stride, threshold và hysteresis. Nếu thiếu một trong các thông tin này, model có thể nạp thành công nhưng cho kết quả sai ngữ nghĩa.

Phụ lục E – Quy trình build và triển khai

E.1. Hai ứng dụng Android

Yêu cầu JDK 11 trở lên, Android SDK phù hợp compile SDK 36 và tệp cấu hình Firebase của từng applicationId. Các lệnh chuẩn:

```
1 cd DATN_RobotApp_Smartphone_v2
2 gradlew.bat testDebugUnitTest
3 gradlew.bat assembleDebug
4
5 cd DATN_CaregiverApp_Smartphone_v2
6 gradlew.bat testDebugUnitTest
7 gradlew.bat assembleDebug
```

Listing 8: Build và unit test Android trên Windows

Không đưa local.properties, API key hoặc credential TURN vào Git. Build release cần signing key bảo mật, minify/resource shrinking và kiểm thử lại reflection/native library.

E.2. Firmware

Trong Arduino IDE, chọn ESP32 Dev Module; cài SimpleFOC phiên bản tương thích và NimBLE-Arduino; mở `robot_ble_control.ino`; kiểm tra pin; compile/upload; mở Serial 115200. Lần đầu initFOC phải nâng bánh khỏi sàn. Kiểm tra encoder quay đúng chiều và chỉ thử setpoint nhỏ trước.

E.3. Dịch vụ

Thiết lập Firebase Auth email/password, RTDB và Rules. Cấu hình STUN/TURN trên cả hai ứng dụng. Cấp Groq key qua cơ chế bí mật. Tạo một tài khoản thử, profile, lịch và cặp thiết bị; xóa SDP/ICE cũ trước test. Đảm bảo notification/full-screen intent được cấp trên caregiver.

Phụ lục F – Phiếu ghi kết quả kiểm thử

Test ID	
Commit/model hash	
Ngày, người thực hiện	
Thiết bị/Android	
Mạng/BLE/khoảng cách	
Tiền điều kiện	
Các bước	
Kết quả mong đợi	
Kết quả quan sát	
Metric/log/video	
Pass/Fail/Blocked	
Mã lỗi và ghi chú	

Phụ lục G – Checklist an toàn trình diễn

- Bánh xe được nâng khỏi sàn khi kiểm tra dấu/FOC lần đầu.

Bảng 6.15: Mẫu tổng hợp benchmark AI

Cấu hình	Median (ms)	P95 (ms)	FPS hiệu dụng	Drop frame (%)
CPU float32				
GPU graph				
FP16				

Bảng 6.16: Mẫu tổng hợp phát hiện té ngã theo sự cố

Cấu hình	Precision	Recall	F1	Cảnh báo giả/giờ
Pose/luật				
Pose + LSTM				
Pose + LSTM + hậu kiểm				

Bảng 6.17: Mẫu tổng hợp độ trễ và an toàn điều khiển

Kịch bản	Median	P95	Max/ghi chú
Caregiver tới Robo-tApp			
RobotApp tới BLE callback			
Thời gian dừng khi mất RTDB			
Thời gian dừng khi mất BLE			

- Khu vực chạy phẳng, không có bậc thang, trẻ nhỏ, vật nuôi hoặc vật cản dễ vỡ.
- Pin, dây, driver và động cơ không quá nhiệt; có cầu chì/bảo vệ phù hợp.
- Có người giữ công tắc ngắt nguồn; STOP đã được thử trước auto-follow.
- Không dùng người cao tuổi thật để dựng lại té ngã; có đệm và giám sát.
- Không trình chiếu API key, token, email/số điện thoại thật hoặc dữ liệu sức khỏe.
- Kiểm tra RTDB Rules và dùng tài khoản/dữ liệu demo.
- Kiểm tra Internet, TURN, notification và quyền camera/micro/BLE.
- Lưu commit/model checksum và phương án khôi phục bản APK/firmware ổn định.
- Ghi log, nhưng tắt/xóa log nhạy cảm sau trình diễn.